OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 15

MANAGING AND RETRIEVING INFORMATION

SUGGESTED TIME: 18 PERIODS

IN THIS CHAPTER

This chapter introduces the principles of database management: data organisation, models, relational design, normalisation, integrity and the SQL language. Learners progress from conceptual modelling (E-R diagrams) to the practical implementation and querying of a real database using an RDBMS.

- By the end of this chapter I can distinguish between data and information and explain the advantages of database systems over flat files
- By the end of this chapter I can compare the hierarchical, network, relational and object-oriented database models
- By the end of this chapter I can describe the three-level ANSI-SPARC database architecture and data independence
- By the end of this chapter I can draw an Entity-Relationship diagram with entities, attributes, relationships and cardinalities
- By the end of this chapter I can identify primary, foreign and candidate keys and implement relationships between tables
- By the end of this chapter I can normalise a relation up to third normal form using functional dependencies

DOWNLOAD ONBUCH

Free • Google Play
AI tutor, past papers, results

GO ONBUCH+

All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE — complete, illustrated, step by step

METHODS — and worked examples

EXERCISES — exam style + solutions

SUMMARY SHEET — the essentials to remember

CONTENTS

1. Database Fundamentals and Models
2. Relational Model and Database Architecture
3. Entity-Relationship Modelling
4. Relational Design, Normalisation and Design Stages
5. Database Integrity and Integration Activity I
6. SQL: Data Definition and Data Manipulation
7. SQL: Queries, Joins and Aggregate Functions
8. RDBMS Implementation, Web and Object-Oriented Integration
9. Integration activity
10. Methods and worked examples
11. Exercises
12. Solutions to the exercises
13. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can distinguish between data and information and explain the advantages of database systems over flat files
- By the end of this chapter I can compare the hierarchical, network, relational and object-oriented database models
- By the end of this chapter I can describe the three-level ANSI-SPARC database architecture and data independence
- By the end of this chapter I can draw an Entity-Relationship diagram with entities, attributes, relationships and cardinalities
- By the end of this chapter I can identify primary, foreign and candidate keys and implement relationships between tables
- By the end of this chapter I can normalise a relation up to third normal form using functional dependencies
- By the end of this chapter I can enforce entity, referential and domain integrity in a database
- By the end of this chapter I can write SQL statements to create tables (DDL), modify data (DML) and query data (DQL)
- By the end of this chapter I can combine data from several tables using joins and summarise data with aggregate functions

- By the end of this chapter I can implement a complete small database in an RDBMS and connect it to a web or object-oriented application

PREREQUISITES

- Basic data representation: fields, records and files (Lower Sixth)
- Spreadsheet manipulation: rows, columns, sorting and filtering (Form 5 / Lower Sixth)
- Basic programming concepts: variables, data types and control structures (Lower Sixth)
- Set notions and simple logic (AND, OR, NOT) from Mathematics
- General ICT literacy: using a keyboard-driven application and saving files

1 DATABASE FUNDAMENTALS AND MODELS

Every day in Cameroon, thousands of records are created: a student registers at GBHS Bamenda, a patient is admitted at the Yaoundé Central Hospital, a customer sends money through MTN Mobile Money, and a candidate's results are processed by the GCE Board. Behind each of these operations lies a **database** that must store information without duplication, retrieve it in seconds, and protect it from errors. Imagine the chaos if the GCE Board stored candidate results in loose Excel files: two candidates with the same name could receive each other's grades! This chapter reveals how professionals design, build and interrogate databases so that information remains organised, consistent and instantly accessible.

The central problem of this section is simple to state: how should we organise large volumes of data so that they can be stored, shared and retrieved reliably? To answer it, we first clarify what data and information are, then we introduce the database and the DBMS, and finally we compare the great historical **database models** that have shaped the discipline.

1.1 Data, Information and the Limits of Traditional Filing

Data versus information.

A **datum** (plural: data) is a raw, unprocessed fact: a number, a name, a date. For example, "17", "Amina" and "2006-03-12" are data. On their own they mean little. **Information** is data that has been processed, organised or placed in context so that it becomes meaningful and useful for decision-making. If we learn that "Amina, born on 2006-03-12, scored 17/20 in Computer Science", the raw values have become information that a principal can use.

EXEMPLE **From data to information**

The GCE Board collects raw marks such as 68, 45, 72. After grading, ranking and computing subject averages, these marks become information: grades (A, B, C), pass lists and school performance statistics. The raw marks are data; the published results slip is information.

Fields, records and files.

Traditionally, data are organised in three levels:

- a **field** is a single item of data about an entity (e.g. a student's `dateOfBirth`);
- a **record** is a collection of related fields describing one entity (e.g. everything known about one student);
- a **file** is a collection of related records (e.g. the file of all Form Five students).

Limitations of paper-based and flat-file systems.

Before databases, schools kept paper registers, and early computer systems used **flat files** — one independent file per application, each with its own program. Consider a school where the principal's office keeps a student file, the bursar keeps a fee file, and each department keeps a marks file. This approach suffers from serious weaknesses:

- **Redundancy**: the same student's name and address are copied into many files, wasting storage.
- **Inconsistency**: when a student changes address, the bursar may update his file while the principal's file keeps the old address. The two copies now contradict each other.
- **Difficulty of sharing**: each file "belongs" to one program; extracting combined information (e.g. "fees paid and marks obtained") requires writing a new program each time.
- **Data dependence**: if the structure of a file changes, every program using it must be rewritten.
- **Poor security and integrity**: there is no central control over who may read or modify the data, and nothing prevents invalid entries such as a mark of 250/20.

These weaknesses are exactly what the database approach was invented to eliminate.

1.2 Databases and Database Management Systems

DEFINITION **Database and DBMS**

A **database** is an organised, shared collection of logically related data, stored so that it can be accessed, managed and updated easily, with minimal redundancy.
A **Database Management System (DBMS)** is the software that allows users to define, create, query, update and administer a database. Examples include MySQL, PostgreSQL, Microsoft Access, Oracle Database and SQLite.

A useful image: the database is the library, and the DBMS is the librarian — you never touch the shelves directly; you ask the librarian, who knows where everything is, enforces the rules and keeps the catalogue consistent.

Functions of a DBMS.

A good DBMS provides at least the following services:

- **Data storage and retrieval**: it stores data efficiently and answers queries quickly, using indexes and optimised access paths.
- **Security and access control**: it authenticates users and grants each user only the rights needed (a clerk may enter marks but not delete the whole table).
- **Concurrency control**: many users can work at the same time without corrupting data — for instance, two MTN agents updating the same account are prevented from interfering with each other through locking and transactions.
- **Backup and recovery**: the DBMS keeps backups and a log of operations, so that after a power cut or disk crash the database can be restored to a consistent state.
- **Integrity enforcement**: it automatically rejects data that violate declared rules (e.g. a negative exam mark).

PROPERTY **Advantages of the database approach over flat files**

Compared with independent flat files, the database approach offers:

1. Reduced redundancy — each fact is stored once, in one place.
2. Data consistency — since there is a single copy, updating it updates it for everyone.
3. Shared access — many users and programs query the same central data concurrently.
4. Data independence — programs are separated from the physical storage details, so storage can be reorganised without rewriting applications.
5. Improved security, integrity and recovery — centralised control of rights, validation rules, backups and transaction logs.

(Statement to know for the examination; no formal proof is required, but you must be able to illustrate each advantage with an example.)

1.3 The Hierarchical Model

The **hierarchical model**, the oldest database model (used by IBM's IMS system), organises data as a tree: each record type is a node, and links represent **parent–child** relationships. The fundamental rule is:

- each child has exactly one parent;
- each parent may have many children.

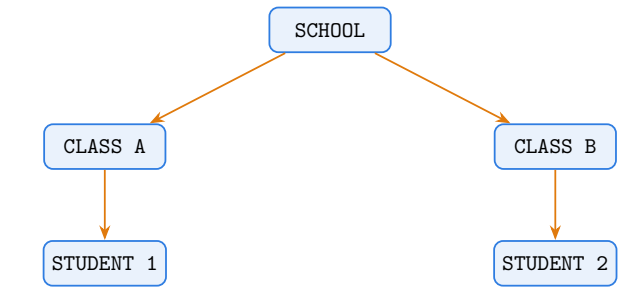
Thus the hierarchical model naturally represents **one-to-many** relationships. Example: a `SCHOOL` has many `CLASS`es, and each `CLASS` has many `STUDENT`s.

Limitations. Real life is full of many-to-many situations: a student takes many subjects, and each subject is taken by many students. In a pure tree, a child cannot have two parents, so representing this forces duplication of records — reintroducing the redundancy we wanted to avoid. Navigation is also rigid: to reach a record you must follow the path from the root.

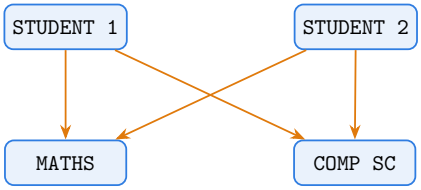
1.4 The Network Model

The **network model** (standardised by the CODASYL consortium) generalises the tree into a graph: a record may have several parents, connected by explicit **links** (called sets). This allows **many-to-many** relationships to be represented directly — a `STUDENT` record can be linked to many `SUBJECT` records, and vice versa.

Limitations. The structure is complex: the programmer must navigate the web of links manually, record by record ("pointer chasing"). Queries are hard to write and maintain, and changing the structure often means rewriting programs — weak data independence.



Hierarchical: a tree



Network: a graph (many-to-many)

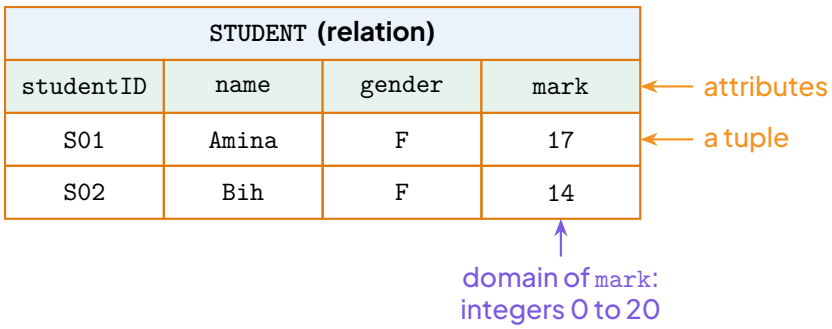
Hierarchical model (tree: one parent per child) versus network model (graph: several parents allowed).

1.5 The Relational Model

In 1970, Edgar F. Codd proposed the **relational model**: all data are stored in tables (called **relations**), and nothing else. There are no pointers and no navigation paths — relationships between tables are expressed through shared values (keys), which we study in detail later in this chapter.

DEFINITION **Relation, tuple, attribute, domain**

- A **relation** is a table with a name, a set of named columns and any number of rows (e.g. `STUDENT`).
- A **tuple** is one row of a relation: it describes one occurrence of the entity (one student).
- An **attribute** is one named column of a relation (e.g. `address`).
- A **domain** is the set of permitted values for an attribute (e.g. the domain of `gender` is {M, F}; the domain of `mark` is the integers from 0 to 20).
- The **degree** of a relation is its number of attributes (columns); the **cardinality** of a relation is its number of tuples (rows).



The relation STUDENT: degree = 4 attributes, cardinality = 2 tuples, and the domain of the attribute mark.

Why the relational model dominates today.

- Simplicity: one single concept — the table — is understood by everyone.
- Solid mathematical foundation: it rests on set theory and predicate logic, so queries have precise, predictable meanings.
- High data independence: users declare what they want (in SQL), not how to navigate to it; the DBMS chooses the access path.
- Flexibility: many-to-many relationships are handled naturally through linking tables, without duplication of the entities themselves.

WARNING **Cardinality: two different meanings**

Do not confuse the cardinality of a table with the cardinality of a relationship.

- The **cardinality of a relation** is simply its number of rows (tuples). The STUDENT table above has cardinality 2.
- The **cardinality of a relationship** (seen in Entity-Relationship modelling) describes how many instances of one entity can be associated with instances of another: one-to-many, many-to-many, etc.

Writing "the relationship STUDENT–SUBJECT has cardinality 5000 because the school has 5000 students" is a classic examination error: 5000 is the cardinality of the table, not of the relationship.

1.6 Beyond Relations: Object-Oriented and Object-Relational Models

Relational tables are excellent for numbers and short text, but modern applications must also store images, sound, maps and complex objects. Two later models answer this need:

- The **object-oriented model** stores data as objects with attributes and methods (behaviour), supporting concepts like classes and inheritance — mirroring languages such as Java or Python.

- The **object-relational model** (e.g. PostgreSQL) extends the relational model with user-defined types and inheritance, combining the rigour of tables with the richness of objects.

DID YOU KNOW? **Extension for GCE Paper 1**

You will not be asked to design object-oriented databases, but you should be able to state one advantage of each model in a comparison question: the object-oriented model handles complex multimedia objects and their behaviour naturally, while the object-relational model keeps full compatibility with existing relational (SQL) systems.

Summary comparison of the four models.

Model	Structure	Strength	Weakness
Hierarchical	Tree (one parent per child)	Fast for fixed one-to-many paths	No many-to-many without duplication; rigid navigation
Network	Graph (several parents)	Handles many-to-many directly	Complex pointer navigation; weak data independence
Relational	Tables linked by key values	Simple, flexible, strong data independence, SQL	Less natural for multimedia objects
Object-oriented	Objects with attributes and methods	Handles complex objects and behaviour	No universal query standard; less widespread

1.7 Choosing a Model: Method and Worked Example

METHOD **Choosing an appropriate database model for a scenario**

1. Identify the entities and the relationships between them (one-to-many? many-to-many? complex objects?).
2. Check the shape of the data: strictly nested (tree-like) data suggest the hierarchical model; highly interconnected data with fixed navigation paths historically suggested the network model.
3. Check flexibility needs: if queries are varied and unpredictable, and many users share the data, choose the relational model.
4. Check data types: if the system must store multimedia or complex structured objects with behaviour, consider an object-oriented or object-relational system.
5. Justify your choice in the examination by naming at least two advantages of the cho-

sen model for that scenario.

WORKED EXAMPLE 1 **Choosing a model for a school system**

Statement. GBHS Bamenda wants a system storing students, classes, subjects and marks. Students take many subjects; each subject is taken by many students. The office, the bursar and all teachers must query the data in many different, unpredictable ways. Which database model do you recommend, and why?

SOLUTION

Solution.

- Step 1: entities are `STUDENT`, `CLASS`, `SUBJECT`, `MARK`; the STUDENT–SUBJECT relationship is many-to-many.
- Step 2: the data are not a strict tree, so the hierarchical model would force duplication of student or subject records — rejected.
- Step 3: queries are varied and shared by many users; the many-to-many relationship is handled cleanly with a linking table `MARK(studentID, subjectCode, score)`. The relational model is therefore recommended.
- Step 4: no multimedia or complex objects are required, so an object-oriented model is unnecessary.
- Conclusion: a relational database managed by a DBMS such as MySQL, because it avoids redundancy, supports many-to-many links through shared keys, and lets every user formulate ad-hoc queries in SQL.

KEY POINTS **Key points of this section**

- Data are raw facts; information is processed, meaningful data. Data are organised into fields, records and files.
- Flat-file systems suffer from redundancy, inconsistency, poor sharing and data dependence.
- A database is a shared, organised collection of related data; a DBMS is the software that manages it (storage, security, concurrency, backup and recovery, integrity).
- The database approach reduces redundancy, guarantees consistency, enables shared access and provides data independence.
- Hierarchical model: tree, one parent per child, one-to-many only. Network model:

graph, several parents, many-to-many but complex to navigate. Relational model: tables linked by values — simple, rigorous, dominant today.

- Vocabulary: relation (table), tuple (row), attribute (column), domain (allowed values), degree (number of columns), cardinality of a table (number of rows) — never to be confused with the cardinality of a relationship.

EXERCISE 1 **Check your understanding**

1. A hospital keeps three separate Excel files: admissions, billing and pharmacy. Give three problems this creates, and name the database advantage that solves each.
2. A relation `PATIENT(pid, name, ward, dateAdmitted)` contains 350 rows. State its degree and its cardinality.
3. For each scenario, choose a model and justify: (a) an organisation chart where every employee reports to exactly one manager; (b) a social network where users befriend many users; (c) the GCE Board's candidate results system queried in many different ways.
4. Explain, with an example, why the hierarchical model cannot represent the STUDENT–SUBJECT relationship without redundancy.

SOLUTION **Exercise 1**

1. Redundancy (same patient typed three times) — solved by reduced redundancy; inconsistency (address updated in one file only) — solved by centralised consistency; difficult sharing (combining billing and pharmacy data needs manual work) — solved by shared access through a DBMS.
2. Degree = 4 (four attributes); cardinality = 350 (number of tuples).
3. (a) Hierarchical model: strict one-to-many tree. (b) Network model (or relational with a linking table): many-to-many links. (c) Relational model: flexible ad-hoc queries, shared access, no duplication.
4. In a tree, a child has one parent only. Since each student takes many subjects and each subject has many students, either `STUDENT` records must be repeated under each subject, or `SUBJECT` records under each student — duplication that reintroduces redundancy and inconsistency.

2 RELATIONAL MODEL AND DATABASE ARCHITECTURE

In the previous section we saw why organisations such as a school in Yaoundé or a microfinance institution in Bamenda prefer a **database** over loose files, and we surveyed the main database models (hierarchical, network, relational). We concluded that the **relational model**, proposed by E. F. Codd in 1970, dominates modern practice because of its simplicity and its solid mathematical foundation. In this section we go deeper: we describe precisely how the relational model structures data (relations, tuples, attributes, domains and keys), we introduce the formal **relational algebra** that underpins query languages, and then we study the **ANSI-SPARC three-level architecture**, which explains how a database system separates what users see from how data is physically stored.

2.1 Structure of the Relational Model

From intuition to definition.

Imagine the register of Upper Sixth Science A in a Cameroonian college. It is naturally a table: each line concerns one student, each column records one fact (name, sex, date of birth, class). The relational model simply formalises this everyday idea.

DEFINITION **Relation, tuple, attribute, domain**

A **relation** is a two-dimensional table with a unique name, consisting of:

- Rows called **tuples** (or records): each tuple describes one entity or one occurrence of a fact;
- Columns called **attributes** (or fields): each attribute has a unique name within the relation and a **domain**, i.e. the set of permitted values (e.g. `Sex` has domain {`M`, `F`}; `DoB` has domain `DATE`).

The degree (or arity) of a relation is its number of attributes; its cardinality is its number of tuples.

EXEMPLE **The relation STUDENT**

StudentID	FullName	Sex	DoB	ClassID
S01	Acha Marie	F	12/03/2007	C2
S02	Ndi Paul	M	25/07/2006	C1
S03	Bih Comfort	F	03/11/2007	C2
S04	Tabi Ernest	M	30/01/2007	C1

This relation has degree 5 and cardinality 4. The domain of `Sex` is {`M`, `F`}; the domain of `StudentID` might be strings of the form `S` followed by two digits.

PROPERTY **Properties of a relation – examinable as direct recall**

A well-formed relation must satisfy:

1. Unique name: no two relations in the same database share a name; attribute names are unique within a relation.
2. Atomic values (First Normal Form): each cell contains a single, indivisible value (never a list such as "French, English" or a composite structure).
3. No duplicate tuples: two tuples cannot be identical in every attribute; the relation is a mathematical set of tuples.
4. Order is irrelevant: the order of rows and of columns carries no semantic information.

WARNING **Atomicity trap**

Storing "Mathematics, Physics" in a single `Subject` cell violates atomicity. Searching for all Physics students then becomes unreliable and requires string parsing. The correct solution is one row per (student, subject) pair — an idea we shall formalise during normalisation (Section 4).

DEFINITION **Relation schema and instance**

The **relation schema** (or intension) is the static description: the relation name and its list of attributes with their domains, e.g. `STUDENT(StudentID, FullName, Sex, DoB, ClassID)`. The **relation instance** (or extension) is the set of tuples at a given moment; it changes as data is inserted, deleted or updated.

2.2 Keys in the Relational Model

Why keys?

Since duplicate tuples are forbidden, there must always be a way to distinguish two tuples. Keys are the attributes that guarantee this identification — exactly as your GCE candidate number identifies you far more reliably than your name.

DEFINITION **Superkey and candidate key**

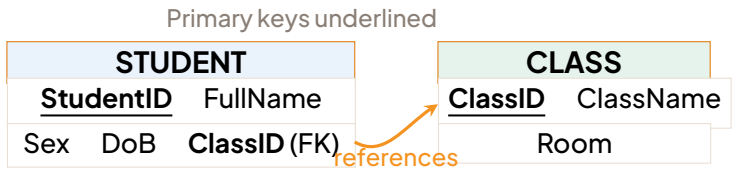
A **superkey** is any set of one or more attributes whose values uniquely identify each tuple of a relation. A **candidate key** is a minimal superkey: removing any attribute from it destroys the uniqueness property.

DEFINITION **Primary key and alternate key**

The **primary key** (PK) is the candidate key chosen by the designer to identify tuples; it is usually underlined in the schema notation, e.g. `STUDENT(StudentID, FullName, Sex, DoB, ClassID)`. The other candidate keys (if any) are called **alternate keys**.

DEFINITION **Foreign key and referential integrity**

A **foreign key** (FK) is an attribute (or set of attributes) of a relation R_2 (the referencing relation) that references the primary key of a relation R_1 (the referenced relation, possibly the same relation). The values of the foreign key must either match an existing primary key value in R_1 or be wholly `NULL` (if the FK allows nulls). This rule is called **referential integrity**.



A foreign key `ClassID` in `STUDENT` referencing the primary key of `CLASS`.

PROPERTY **Characteristics of a good primary key**

A good primary key must be:

1. Unique: no two tuples ever share its value (entity integrity);
2. Minimal: it contains no unnecessary attribute;
3. Stable: its value never changes over the lifetime of the entity;
4. Non-null: every tuple has a value for it (no component of the PK may be `NULL`).

WARNING **Common mistake: names and phone numbers as keys**

Never use a name as a primary key: two students may both be called “Ndi Paul”. Never use a phone number: a subscriber can change operator, share a family phone, or have no phone, so the value is neither stable nor always unique nor non-null. Prefer an artificial identifier (`StudentID`, candidate number, `NIH_Number`) that is meaningless but unique, minimal, stable and never null.

METHOD **Identifying keys in a sample table**

1. List all attributes of the table.
2. For each attribute, ask: "Can two rows ever have the same value here?" If yes, it cannot be a key alone.
3. Test combinations of attributes until uniqueness is achieved; then remove any attribute that is not needed (minimality) to obtain candidate keys.
4. Choose as primary key the candidate that is shortest, most stable and never null; mark the others as alternate keys.
5. Look for attributes whose values match the primary key of another table: these are foreign keys.

WORKED EXAMPLE 2 **Finding the keys of a relation**

Consider the table `ENROL(StudentID, StudentName, Phone, SubjectCode, Term)` recording which subject each student takes each term. Identify the candidate keys, choose a primary key, and state the foreign keys.

SOLUTION

Step 1 – single attributes. `StudentName` and `Phone` can repeat or change: rejected. `StudentID` alone repeats (a student takes several subjects); `SubjectCode` alone repeats too. Step 2 – combinations. A student takes a given subject only once per term, so `(StudentID, SubjectCode, Term)` is unique, and removing any part breaks uniqueness: it is a candidate key. If we assume a student has one phone, `(Phone, SubjectCode, Term)` would also be a superkey, but phone numbers are unstable, so it is a poor candidate. Step 3 – primary key. Choose `(StudentID, SubjectCode, Term)`: unique, minimal, stable, non-null. Step 4 – foreign keys. `StudentID` references `STUDENT(StudentID)`; `SubjectCode` references `SUBJECT(SubjectCode)`.

2.3 Relational Algebra – The Formal Manipulation Language

Observation.

The relational model is not only about static structure; Codd defined a set of operations that take one or two relations as input and produce a new relation as output. This **relational algebra** is the theoretical foundation of SQL. Every SQL query can be translated into a relational algebra expression. The main operators are:

DEFINITION **Unary relational operators**

- Selection (σ): $\sigma_{\text{condition}}(R)$ returns a relation containing only those tuples of R that satisfy the condition (horizontal subset).
- Projection (π): $\pi_{\text{attr_list}}(R)$ returns a relation containing only the listed attributes, with duplicate tuples removed (vertical subset).
- Renaming (ρ): $\rho_{\text{newName}}(R)$ or $\rho_{\text{newName(newAttrs)}}(R)$ changes the relation name and/or attribute names.

DEFINITION **Binary relational operators (set-theoretic)**

These require union compatibility: the two relations must have the same degree and corresponding attributes must have the same domains.

- Union ($\cup$): $R \cup S$ – tuples in R or S (or both).
- Intersection ($\cap$): $R \cap S$ – tuples in both R and S.
- Difference ($-$): $R - S$ – tuples in R but not in S.

DEFINITION **Binary relational operators (special)**

- Cartesian Product ($\times$): $R \times S$ concatenates every tuple of R with every tuple of S. Degree = $\deg(R) + \deg(S)$, Cardinality = $|R| \times |S|$. Rarely used directly; basis for joins.
- Join (⋈): Combines related tuples from two relations.
  - Theta Join: $R \bowtie_{\text{condition}} S = \sigma_{\text{condition}}(R \times S)$.
  - Equijoin: Theta join where condition is equality on common attributes.
  - Natural Join ($*$): Equijoin on all attributes with the same name, then projecting out one copy of each common attribute.
  - Outer Joins (Left, Right, Full): Preserve unmatched tuples from one or both sides, padding with `NULL`s.
- Division ($\div$): $R \div S$ finds tuples in R that match every tuple in S on the common attributes. Used for "for all" queries (e.g., students taking all core subjects).

EXEMPLE **Relational algebra on STUDENT and CLASS**

Let `STUDENT(StudentID, FullName, Sex, DoB, ClassID)` and `CLASS(ClassID, ClassName, Room)`.

- Names of female students: $\pi_{\text{FullName}}(\sigma_{\text{Sex='F'}}(\text{STUDENT}))$
- Students with their class names: STUDENT $*$ CLASS (Natural Join on `ClassID`)
- Students in class C1: $\pi_{\text{FullName}}(\sigma_{\text{ClassID='C1'}}(\text{STUDENT}))$

KEY POINTS **Relational Algebra – Key Points for Examination**

- Selection (σ) filters rows; Projection (π) filters columns (removes duplicates).
- Union, Intersection, Difference require union compatibility.
- Natural Join ($*$) is the most common join: equijoin on common attribute names + removal of duplicate columns.
- Division ($\div$) expresses universal quantification ("all").
- These operators are closed: input relations → output relation. This allows nesting.

2.4 Integrity Rules of the Relational Model

PROPERTY **Codd's Integrity Rules – examinable**

1. Entity Integrity: No component of a primary key may be `NULL`. (An entity must be identifiable.)
2. Referential Integrity: For every foreign key value in a referencing relation, there must exist a matching primary key value in the referenced relation, or the foreign key must be wholly `NULL` (if the relationship is optional).

WARNING **Referential actions**

When a referenced tuple is deleted or its PK updated, the DBMS must enforce referential integrity. Standard actions (defined in SQL DDL): `RESTRICT` (reject), `CASCADE` (propagate), `SET NULL`, `SET DEFAULT`. Know these for the SQL section.

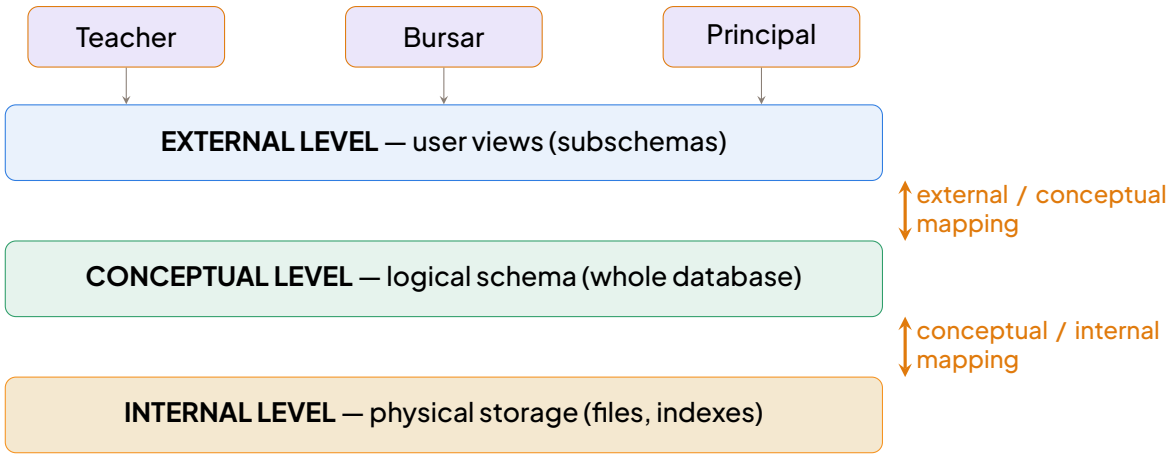
2.5 The ANSI-SPARC Three-Level Architecture

Observation.

A bursar consulting fees, a principal viewing enrolment statistics, and the hard disk holding the actual bytes all "see" the same school database very differently. In 1975 the ANSI-SPARC committee proposed an architecture in three levels so that each group works at its own level without disturbing the others.

DEFINITION **The three levels**

- External level (views): what each user group sees — a personalised **subschema** or view showing only the relevant data (e.g. a teacher sees marks, never salaries). Multiple external views can exist.
- Conceptual level (logical schema): the complete logical description of the whole database — all entities, attributes, relationships, integrity constraints — independent of any storage detail. There is exactly one conceptual schema per database.
- Internal level (physical schema): how data is actually stored: file organisations (heap, sorted, hashed), indexes (B-tree, hash), record layout, compression, partitioning, on the physical media.



The ANSI-SPARC three-level architecture with its two mappings.

DEFINITION **Schemas, subschemas and mappings**

A **schema** is the overall description of the database at one level; a **subschema** is the part visible to a particular user group (external level). A **mapping** is the correspondence the DBMS maintains between two levels:

- External/Conceptual mapping: translates each view into logical structures (e.g., a view `TeacherView` maps to a join of `STUDENT` and `MARK`).
- Conceptual/Internal mapping: translates logical structures into physical storage (e.g., `STUDENT` relation → hashed file on `StudentID` with a B-tree index on `ClassID`).

DEFINITION **Logical and physical data independence**

- **Physical data independence**: the ability to change the internal schema (e.g. add an index, move files to a faster disk, change file organisation) without changing the conceptual schema or the application programs. Achieved by modifying the conceptual/internal mapping.
- **Logical data independence**: the ability to change the conceptual schema (e.g. add a new attribute `Email` to `STUDENT`, split a relation) without forcing changes to existing external views or programs. Achieved by modifying the external/conceptual mapping.

Physical independence is easier to achieve fully; logical independence is harder, because some changes (deleting an attribute used by a view) inevitably affect that view.

EXEMPLE **Independence in practice**

The school migrates its database from a local hard disk to a solid-state server and adds indexes: thanks to physical data independence, teachers notice nothing. Later the ministry requires a new field `Region of origin`: thanks to logical data independence, the bursar's fees view continues to work unchanged (the mapping absorbs the new column).

2.6 People Around the Database: DBA and Users

DEFINITION **Database administrator and categories of users**

The **Database Administrator** (DBA) is the person (or team) responsible for the database: defining schemas, granting access rights, ensuring security, backup and recovery, tuning performance, and maintaining the data dictionary. The users include:

- Naive (end) users: interact through ready-made interfaces (a secretary entering marks on a form);
- Application programmers: write the programs (in Python, Java, PHP, …) that query the database via embedded SQL or APIs;

- Sophisticated users: analysts who write their own SQL queries directly (ad-hoc queries).

DEFINITION **Data dictionary / catalogue**

The **data dictionary** (or **system catalogue**) is the "database about the database": it stores metadata — names and structures of relations, attribute domains, keys, views, user accounts and access rights, integrity constraints, storage statistics, and mapping information. The DBMS consults it automatically before executing any query.

KEY POINTS **Key points of this section**

- A relation is a table with a unique name, atomic values (1NF), no duplicate tuples, and unordered rows/columns.
- Keys: superkey ⊇ candidate key (minimal) ⊇ chosen primary key; the rest are alternate keys; a foreign key references a primary key of another relation.
- A good primary key is unique, minimal, stable, non-null — never a name or phone number.
- Relational Algebra: $\sigma, \pi, \cup, \cap, -, \times, ⍰, *, \div$. Closed under composition. Foundation of SQL.
- Integrity: Entity Integrity (PK ≠ NULL), Referential Integrity (FK matches PK or NULL).
- ANSI-SPARC: external (views) – conceptual (logical schema) – internal (physical storage), linked by two mappings.
- Physical independence protects applications from storage changes; logical independence protects views from schema changes.
- The DBA administers the database; the data dictionary stores all metadata.

EXERCISE 2 **Keys, Algebra and Architecture**

A hospital in Douala keeps `PATIENT(NIH_Number, Name, Phone, Ward)` and `WARD(WardCode, WardName, Beds)`.

1. Identify the primary and foreign keys, justifying each choice with respect to the properties of a good key.
2. Explain why `Phone` is a bad primary key.

3. Write the relational algebra expression for: "Names of patients in Ward 'W01".
4. State which level of the ANSI-SPARC architecture changes, and which type of data independence is demonstrated, when the hospital (a) adds an index on `Name`, (b) adds a column `BloodGroup` to `PATIENT`.

SOLUTION **Exercise 1**

1. `PATIENT`: primary key `NIH_Number` (unique, minimal, stable, non-null); `Ward` is a foreign key referencing `WARD(WardCode)`. `WARD`: primary key `WardCode`.
2. A phone number can change, can be shared between relatives, and a patient may have none: it is neither stable nor reliably unique nor non-null.
3. $\pi_{\text{Name}}(\sigma_{\text{Ward='W01'}}(\text{PATIENT}))$
4. (a) Adding an index touches only the internal level: physical data independence. (b) Adding a column touches the conceptual level while existing views keep working: logical data independence.

With the structure of relations, the formal algebra, and the architecture of a DBMS now clear, we are ready in the next section to design databases from real-world descriptions using Entity–Relationship modelling.

3 ENTITY-RELATIONSHIP MODELLING

In the previous section we studied the **relational model** – tables, tuples, keys – and the **three-level architecture** that separates the way users see data from the way it is physically stored. But a natural question remains: how does a designer decide which tables to create in the first place? You would never ask a builder to start laying blocks without a plan; likewise, no serious database is built by inventing tables at random. Before writing a single `CREATE TABLE` statement, the designer first produces a **conceptual model** of the real-world situation. The most widely used conceptual modelling technique, and the one required by the GCE Advanced Level syllabus, is the **Entity–Relationship (E–R) model**, introduced by Peter Chen in 1976.

3.1 Purpose of Conceptual Modelling

Imagine you are asked to computerise the records of a secondary school in Bamenda. The principal describes the situation in plain English: "Students enrol in subjects; each teacher teaches several subjects; every candidate is registered at one examination centre." None of these sentences mentions tables, columns or keys. The first task of the designer is therefore to translate this description of the real world into a clear, precise, graphical model that everyone – principal, teachers, programmer – can discuss and validate.

DEFINITION **Conceptual model**

A **conceptual model** is a high-level description of the data requirements of an organisation, expressed in terms of entities, attributes and relationships, independent of any particular DBMS, computer or storage technology.

The E-R model answers three questions: What things do we store information about? (entities), What do we know about each thing? (attributes), and How are the things connected? (relationships).

3.2 Entities and Entity Sets

DEFINITION **Entity, entity set, attribute, relationship**

An **entity** is a distinguishable object or thing in the real world about which we wish to store information (e.g. a particular student, a particular subject). An **entity set** (or entity type) is the collection of all entities of the same kind, sharing the same attributes (e.g. STUDENT, SUBJECT).
An **attribute** is a property that describes an entity (e.g. the name and date of birth of a student).
A **relationship** is an association among two or more entities (e.g. "Amina enrols in Computer Science"). A **relationship set** is the collection of all relationships of the same kind (e.g. ENROLS between STUDENT and SUBJECT).

EXEMPLE **Entities around us**

In a Cameroonian school context, typical entity sets are: STUDENT, TEACHER, SUBJECT, CLASS, EXAM-CENTRE. Typical relationships: a student enrols in subjects; a teacher teaches subjects; a candidate is registered at an exam centre.

Strong and weak entities. Some entities can only be identified by reference to another entity. A dependant of a civil servant, or an invoice line of an invoice, has no meaning on its own.

PROPERTY **Strong vs weak entities**

A **strong entity** (regular entity) possesses its own key attribute that uniquely identifies each of its members (e.g. STUDENT identified by student_id).
A **weak entity** has no key of its own; it is identified by combining a partial key (discriminator) with the key of the owner (identifying) strong entity, through an identifying relationship. A weak entity always participates totally in its identifying relationship – it cannot exist without its owner.

Example: DEPENDANT (of an employee) is weak: its partial key dependant_name only becomes unique when combined with the owner's key employee_id. In Chen's notation, a weak entity is drawn with a double rectangle and its identifying relationship with a double diamond.

3.3 Attributes and Their Classification

Attributes are classified along three independent dimensions.

1. Simple vs composite. A **simple** (atomic) attribute cannot be meaningfully subdivided (e.g. sex, mark). A **composite** attribute can be broken into components (e.g. address = town + quarter + street; name = first name + surname).

2. Single-valued vs multi-valued. A **single-valued** attribute holds one value per entity (a student has one date of birth). A **multi-valued** attribute may hold several values (a student may have several phone numbers; a teacher may hold several qualifications). In Chen's notation a multi-valued attribute is drawn with a **double oval**.

3. Stored vs derived. A **stored** attribute is kept in the database (e.g. date_of_birth). A **derived** attribute is computed from other data (e.g. age is derived from date_of_birth and today's date). Derived attributes are drawn with a **dashed oval**.

Key attribute. A **key attribute** (identifier) is an attribute whose value uniquely identifies each entity in an entity set: student_id for STUDENT, subject_code for SUBJECT. In Chen's notation the key attribute name is **underlined**.

WARNING **Do not store what you can derive**

Storing both date_of_birth and age is a classic design error: every year every age value becomes wrong. Store the date of birth and derive the age when needed.

3.4 Relationships, Degree and Cardinality

Degree of a relationship. The **degree** is the number of entity sets participating in a relationship set:

- **Unary (recursive), degree 1:** an entity set relates to itself, e.g. a STUDENT is class prefect of other STUDENTs; an EMPLOYEE supervises EMPLOYEEs.
- **Binary, degree 2:** the most common case, e.g. STUDENT enrols in SUBJECT.
- **Ternary, degree 3:** three entity sets at once, e.g. a TEACHER teaches a SUBJECT in a CLASS – splitting this into binary relationships would lose information.

DEFINITION **Cardinality ratio and participation constraint**

The **cardinality ratio** of a binary relationship specifies the maximum number of relationship instances in which an entity can participate:

- One-to-one (1:1): each entity on each side is linked to at most one entity on the other side. Example: CANDIDATE – is assigned – CANDIDATE-NUMBER at a GCE centre.
- One-to-many (1:N): one entity on the "one" side is linked to many on the other side, but each entity on the "many" side is linked to at most one. Example: SCHOOL – registers – STUDENT: a school registers many students, but each student is registered at one school.
- Many-to-many (M:N): entities on both sides may be linked to many on the other side. Example: TEACHER – teaches – SUBJECT: a teacher teaches several subjects and a subject is taught by several teachers.

The **participation constraint** specifies the minimum: whether every entity must take part in the relationship.

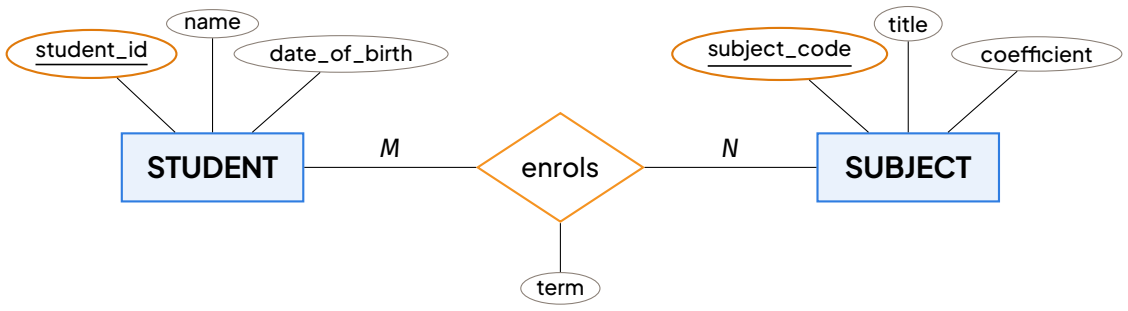
- Total participation (existence dependency): every entity must participate – drawn with a double line. Example: every CANDIDATE must be registered at an EXAM-CENTRE.
- Partial participation: some entities may not participate – drawn with a single line. Example: not every TEACHER is a class master of a CLASS.

EXEMPLE **Reading cardinality correctly**

"STUDENT enrols in SUBJECT is M:N" means: one student enrols in many subjects, and one subject is taken by many students. Always read the ratio in both directions before deciding.

3.5 E-R Diagram Notation: Chen and Crow's Foot

Concept	Chen notation	Crow's foot notation
Entity set	Rectangle	Rectangle (table-like box)
Weak entity	Double rectangle	Rectangle with double border
Relationship	Diamond	Labelled line between entities
Attribute	Oval attached to entity	Listed inside the entity box
Key attribute	Underlined oval label	Underlined or marked (PK)
Multi-valued attribute	Double oval	Usually resolved into a new entity
Cardinality	1, *N*, *M* written on edges	Symbols: circle (zero), bar (one), crow's foot (many)
Total participation	Double line	Bar symbol (mandatory)



E-R diagram in Chen notation: STUDENT enrols in SUBJECT is many-to-many; the relationship carries its own attribute term.



The same model in crow's foot notation: attributes are listed inside the entity boxes and the "crow's foot" symbol marks the many ends.

3.6 Building an E-R Diagram: A Guided Method

METHOD **Five steps to build an E-R diagram from a textual description**

1. Identify the entity sets. Underline the nouns that represent things we store data about (student, subject, centre). Give each a short singular name in capitals.
2. Identify the relationships. Look for the verbs connecting the nouns (enrols, teaches, is registered at). Name each relationship and note its degree.
3. Determine cardinality and participation. For each relationship ask, in both directions: "Can one X be linked to many Ys?" and "Must every X participate?" Write 1, *M*, *N* on the edges (Chen) or place the crow's feet.
4. Attach the attributes. List the properties of each entity, choose the key attribute for each strong entity, and mark multi-valued or derived attributes. Place attributes that describe the association (e.g. the term of an enrolment) on the relationship.
5. Validate with the users. Walk through real examples ("Amina takes 7 subjects; Computer Science is taken by 45 students") to confirm every cardinality and participation constraint.

WARNING **Common mistakes in E-R modelling**

- Placing an attribute on the wrong entity. subject_title belongs to SUBJECT, not to STUDENT – otherwise the title would be repeated for every student. Ask: "Whose property is this?"
- Omitting cardinalities. A diagram without 1, *M*, *N* (or crow's feet) is incomplete: nobody can tell whether a student may take one subject or many. Cardinalities are always required at the GCE exam.
- Confusing the direction of 1:N. In SCHOOL–registers–STUDENT the "many" side is STUDENT. Read the sentence aloud: "one school, many students."
- Modelling a multi-valued attribute as a single attribute. Several phone numbers in one field will make searching impossible; model them as multi-valued (double oval) or as a separate weak entity.

3.7 Worked Example

WORKED EXAMPLE 3 **Analysing a mini-statement**

A GCE candidate is registered at exactly one examination centre, and an examination centre registers many candidates. Every candidate must be registered at a centre; a centre may exist (on paper) before any candidate registers. For each candidate we store the candidate number (unique), name and date of birth; for each centre, the centre number (unique), name and town.
Required: state the entity sets, the relationship, its cardinality ratio, and the participation constraints.

SOLUTION

Solution.

- Entity sets: CANDIDATE (key: candidate_no) and EXAM-CENTRE (key: centre_no). Both are strong entities since each has its own key.
- Relationship: is registered at, binary, between CANDIDATE and EXAM-CENTRE.
- Cardinality ratio: 1:N – one centre registers many candidates; each candidate is registered at one centre. (Chen: write 1 on the centre edge, *N* on the candidate edge.)
- Participation: CANDIDATE participates totally (every candidate must be at a centre – double line); EXAM-CENTRE participates partially (a centre may have no candidates yet – single line).

3.8 Case Study: A School Library System

EXPERIMENT **Modelling a school library**

The librarian of a college in Yaoundé describes the system: "Each pupil borrows books. A pupil may borrow several books, and a book (a physical copy) is borrowed by at most one pupil at a time. We record the loan date and the expected return date for each borrowing. Pupils have a registration number, name and class; books have a copy number, title and author. Some books share the same title and author but are different copies."
Step 1 – Entities: PUPIL and BOOK-COPY. Since two copies can share a title, copy_no alone may not suffice globally, so the library assigns each copy a unique barcode: BOOK-COPY is strong with key barcode.
Step 2 – Relationship: borrows, binary, between PUPIL and BOOK-COPY.
Step 3 – Cardinality and participation: 1:N – one pupil borrows many copies; each copy is borrowed by at most one pupil at a time. Participation is partial on both sides (a new pupil has borrowed nothing; a new copy is on the shelf).
Step 4 – Attributes: PUPIL(reg_no, name, class); BOOK-COPY(barcode, title, author); the relationship borrows carries loan_date and return_date – these describe the event of borrowing, not the pupil or the book.
Step 5 – Validation: "Pupil Etoundi borrows 3 books; copy BC-1042 is with one pupil only." The model holds. Later, when the library wants a history of loans (a copy borrowed many times over the year), the relationship becomes M:N with a LOAN entity – an excellent discussion point for class.

EXERCISE 3 **Hospital appointment system**

A hospital in Douala keeps appointment records. Each PATIENT has a patient number, name and town. Each DOCTOR has a staff number, name and speciality. A patient may book several appointments; each appointment is with exactly one doctor and concerns exactly one patient, and records a date and a time. A doctor may receive many appointments.

1. Identify the entity sets and their key attributes.
2. Give the degree and cardinality ratio of each relationship.
3. State the participation of PATIENT in the books relationship, justifying your answer.
4. Suggest one multi-valued attribute and one derived attribute for this system.

SOLUTION **Exercise: hospital appointment system**

1. Entity sets: PATIENT (key patient_no), DOCTOR (key staff_no). Both are strong entities. The appointment may be modelled as a relationship books with attributes date and time, or as a weak entity APPOINTMENT owned by PATIENT and DOCTOR.
2. books is binary (between PATIENT and DOCTOR) and many-to-many in general (a patient sees many doctors over time; a doctor sees many patients); each individual appointment links one patient to one doctor.
3. Participation of PATIENT is partial: a patient may be registered at the hospital without currently having any appointment.
4. Multi-valued: phone_number of PATIENT (a patient may give two numbers). Derived: age of PATIENT, computed from date_of_birth.

KEY POINTS **Key points**

- The E-R model is a **conceptual** tool: entities (things), attributes (properties), relationships (associations) – independent of any DBMS.
- Attributes are simple/composite, single-valued/multi-valued, stored/derived; the **key attribute** uniquely identifies each entity.
- A **weak entity** has no key of its own; it depends on an owner entity through an identifying relationship (double rectangle, double diamond).
- Degree: unary, binary, ternary. Cardinality ratios: 1:1, 1:N, M:N. Participation: total (double line) or partial (single line).
- Two notations: Chen (rectangles, diamonds, ovals, $1/M/N$ labels) and crow's foot (attributes inside boxes, crow's foot for "many").
- Always justify cardinalities by reading the relationship in both directions, and never omit them from an exam diagram.

DID YOU KNOW? **From paper files to E-R diagrams**

Many Cameroonian institutions – schools, hospitals, councils – still keep paper registers. When such an institution computerises, the very first deliverable the analyst produces is usually an E-R diagram: it is the bridge between the bursar's exercise books and the relational tables you will design in the next section. Mastering E-R modelling means you can walk into any organisation and capture its information system on a single sheet of paper.

4 RELATIONAL DESIGN, NORMALISATION AND DESIGN STAGES

In the previous section we learnt how to model the real world with **Entity–Relationship diagrams**: entities, attributes, relationships and cardinalities. But a diagram on paper cannot be stored in a computer. This section answers the natural next question: how do we turn an E-R diagram into actual tables, and how do we make sure those tables are well designed? The answer has two parts: a set of **translation rules** from the E-R model to relations, and a **quality-control procedure** called normalisation, which removes redundancies and anomalies. We finish with the complete **stages of database design**, from the first interview with the client to the maintenance of the running system.

4.1 From E-R Diagram to Relations

The relational model speaks only one language: the **relation** (table). Translation is therefore a matter of applying three simple rules.

METHOD **Translating an E-R diagram into relations**

1. Entity → table. Each strong entity type becomes a table. Each attribute of the entity becomes a column, and the entity's identifier becomes the **primary key** (underlined).
2. One-to-many → foreign key. For a $1:N$ relationship, take the primary key of the "one" side and place it in the table of the "many" side as a **foreign key**. Example: one School registers many Candidates, so `Candidate(candNo, name, schoolNo)`.
3. Many-to-many → junction table. An $M:N$ relationship cannot be represented by a foreign key in either table; it becomes its own table (a **junction** or bridge table) whose primary key is the combination of the two foreign keys, plus any attributes of the relationship itself.

METHOD **Converting a many-to-many relationship into a junction table**

Suppose Student and Subject are linked by the relationship registers-for, which carries the attribute `grade`.

1. Create the two entity tables: `Student(studId, name)` and `Subject(subjCode, title)`.
2. Create a third table named after the relationship: `Registration`.
3. Copy the primary key of each entity into `Registration` as a foreign key: `studId`, `subjCode`.
4. Declare the pair `(studId, subjCode)` as the composite primary key of `Registration`.

5. Add the relationship's own attributes: `grade`.

Result: `Registration(studId, subjCode, grade)`. Each foreign key also enforces referential integrity: no registration can mention a student or a subject that does not exist.

WARNING **Common mistake**

Never try to store a many-to-many relationship by putting a list of values in one cell (e.g. a column `subjects` containing "Maths, Physics, ICT"). This violates atomicity (1NF) and makes searching, sorting and updating almost impossible. The junction table is the only correct solution.

4.2 Functional Dependencies

Normalisation is built on a single idea: which columns determine which other columns?

DEFINITION **Functional dependency and determinant**

Let R be a relation and A, B sets of attributes of R. We say that B is **functionally dependent** on A, written $A \to B$, if and only if each value of A is associated with exactly one value of B. In other words, knowing A is enough to know B. The attribute(s) A on the left-hand side is called the **determinant**.

EXEMPLE **Reading dependencies in a GCE context**

In a candidate-results table:

- `candNo` → `candName, centreNo`: a candidate number determines one name and one centre.
- `centreNo` → `centreName`: a centre number determines one centre name.
- `(candNo, subjCode)` → `grade`: only the combination of candidate and subject determines the grade.

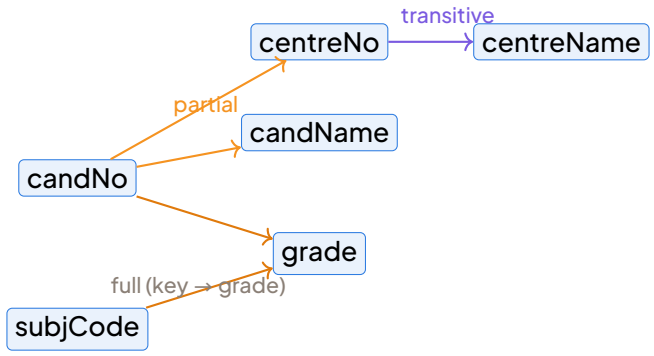
Three types of dependency matter for normalisation:

DEFINITION **Full, partial and transitive dependency**

- Full dependency: B depends on the whole of a composite key, and on no part of it

alone. Example: `grade` depends fully on `(candNo, subjCode)`.

- Partial dependency: a non-key attribute depends on only part of a composite key. Example: `candNo` → `candName` inside a table keyed by `(candNo, subjCode)`.
- Transitive dependency: a non-key attribute depends on another non-key attribute: if $A \to B$ and $B \to C$ with A the key and B non-key, then C depends transitively on A through B. Example: `candNo` → `centreNo` and `centreNo` → `centreName`, so `centreName` is transitively dependent on `candNo`.



Dependency diagram: `grade` depends fully on the composite key; `candName` and `centreNo` depend partially on `candNo` alone; `centreName` depends transitively through `centreNo`.

4.3 Why Bad Design Hurts: Update Anomalies

Consider the unnormalised table `RESULT(candNo, candName, centreNo, centreName, subjCode, grade)`. Because facts are repeated and mixed together, three classical anomalies appear.

PROPERTY **The three update anomalies**

A badly designed relation suffers from:

1. Insertion anomaly: we cannot record a fact without inventing unrelated data. We cannot register a new centre (say, GBHS Bamenda) until at least one candidate from that centre has a result row.
2. Update (modification) anomaly: a fact stored in many rows must be changed everywhere. If a centre changes its name, every row of every candidate of that centre must be updated; miss one and the database becomes inconsistent.
3. Deletion anomaly: deleting a row destroys more information than intended. If the only candidate of a centre is deleted, the centre itself disappears from the database.

(Statement examinable; the justification is by example, as above.)

4.4 The Normal Forms

Normalisation is the step-by-step decomposition of relations to remove redundancies and anomalies, while preserving all data and dependencies. For the GCE Advanced Level you must master the first three normal forms.

DEFINITION **First Normal Form (1NF)**

A relation is in **1NF** if every attribute contains only atomic (indivisible) values and there are no repeating groups: each cell holds exactly one value, and each row is uniquely identifiable by a primary key.

DEFINITION **Second Normal Form (2NF)**

A relation is in **2NF** if it is already in 1NF and every non-key attribute depends on the whole primary key — all partial dependencies on a composite key have been removed (by splitting the table). A table whose key is a single attribute is automatically in 2NF once in 1NF.

DEFINITION **Third Normal Form (3NF)**

A relation is in **3NF** if it is already in 2NF and it contains no transitive dependency: no non-key attribute depends on another non-key attribute. Every non-key attribute must depend on the key, the whole key, and nothing but the key.

METHOD **Step-by-step normalisation from UNF to 3NF**

1. UNF → 1NF: remove repeating groups; give every row a single value per cell; choose a primary key (often composite).
2. 1NF → 2NF: identify attributes that depend on only part of a composite key; move each such attribute, together with a copy of its determinant, into a new table.
3. 2NF → 3NF: in each table, find dependencies among non-key attributes ($A \rightarrow B$ where neither is the key); move B and its determinant A into a new table, leaving A behind as a foreign key.
4. Check: every table must now have all non-key attributes fully dependent on its key

alone, and every fact must be stored exactly once.

WARNING **Common mistake — stopping at 2NF**

Many candidates remove partial dependencies and stop. But if a table still contains, for example, `centreNo` → `centreName`, then `centreName` is repeated on every row and the update and deletion anomalies remain. Always ask the final question: does any non-key attribute determine another non-key attribute? If yes, you are not yet in 3NF.

WORKED EXAMPLE 4 **Normalising a GCE candidate-results table**

Statement. A school keeps results in one flat table (UNF), where each candidate sits several subjects:

candNo	candName	centreNo	centreName	subjCode	grade
C001	Amina B.	CE12	GBHS Bamenda	MATH, PHY	A, B
C002	Tabi E.	CE12	GBHS Bamenda	MATH	C

Normalise this design to 3NF, justifying each step.

SOLUTION

Step 1 — UNF to 1NF (atomic values, no repeating groups). The column `subjCode` holds lists. We explode the repeating group: one row per (candidate, subject). Primary key: `(candNo, subjCode)`.

`R1(candNo, subjCode, candName, centreNo, centreName, grade)`

Step 2 — 1NF to 2NF (remove partial dependencies). In R1, `candName`, `centreNo` and `centreName` depend only on `candNo` (part of the key), while `grade` depends on the whole key. We split:

`CANDIDATE(candNo, candName, centreNo, centreName)`

`SITS(candNo, subjCode, grade)`

Step 3 — 2NF to 3NF (remove transitive dependencies). In CANDIDATE, `candNo` → `centreNo` and `centreNo` → `centreName`: a transitive dependency. We split again:

`CANDIDATE(candNo, candName, centreNo)`

`CENTRE(centreNo, centreName)`

`SITS(candNo, subjCode, grade)`

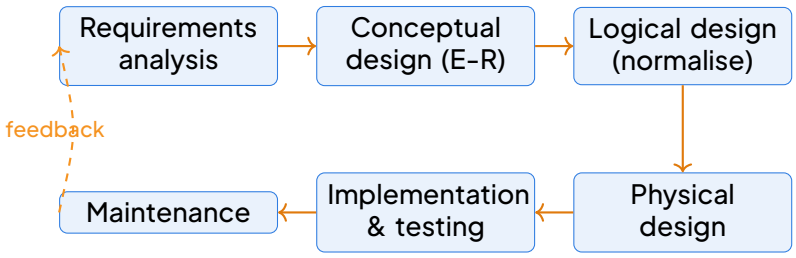
Verification. Every non-key attribute now depends on the key, the whole key, and nothing but the key. Each centre name is stored once: the three anomalies have disappeared. Note that `SITS` is exactly the junction table of the many-to-many relationship between Candidate and Subject — normalisation and E-R translation agree.

4.5 Stages of Database Design

Normalisation does not happen in isolation; it is one step inside a larger, disciplined process. Building a database for a school, a hospital or a bank follows well-defined stages.

DEFINITION **Stages of database design**

1. Requirements analysis: meet the users, collect documents and interviews, and state precisely what data must be stored and what operations must be supported.
2. Conceptual design: build an implementation-independent model of the data — typically the **E-R diagram** (entities, relationships, cardinalities).
3. Logical design: translate the conceptual model into relations, choose keys, and normalise the schema to 3NF.
4. Physical design: decide how the data will actually be stored in the chosen DBMS: data types, indexes, file organisation, storage space and access speed.
5. Implementation: create the tables (SQL `CREATE TABLE`), load the data, write the applications and test.
6. Maintenance: monitor performance, back up data, correct errors and evolve the schema as needs change.



The database design life cycle: each stage feeds the next, and maintenance can send the designers back to requirements analysis.

KEY POINTS **Key points**

- E-R translation: entity → table; $1:N$ → foreign key; $M:N$ → junction table with a composite key.
- $A \to B$ means A determines B; A is the determinant. Distinguish full, partial and transitive dependencies.

- Redundancy causes insertion, update and deletion anomalies.
- 1NF: atomic values; 2NF: no partial dependency on a composite key; 3NF: no transitive dependency — "the key, the whole key, and nothing but the key".
- Design stages: requirements → conceptual → logical → physical → implementation → maintenance.

EXERCISE 4 **Normalising a library loan table**

A school library stores loans as: `LOAN(studId, bookId, studName, class, bookTitle, author, loanDate)`.

1. List all functional dependencies, identifying the partial and transitive ones.
2. State two anomalies this table suffers from, with an example for each.
3. Decompose the table into 3NF, underlining primary keys and marking foreign keys.

SOLUTION **Exercise 1**

1. Dependencies: `(studId, bookId)` → `loanDate` (full dependency on the composite key); `studId` → `studName, class` (partial — depends on part of the key only); `bookId` → `bookTitle, author` (partial). There is no transitive dependency here, because no non-key attribute determines another non-key attribute.
2. Insertion anomaly: a new book cannot be recorded until someone borrows it. Deletion anomaly: if the only borrower of a book returns it and the row is deleted, the book's title and author are lost. (Update anomaly would also occur: if a student changes class, every loan row of that student must be updated.)
3. 3NF decomposition: `STUDENT(studId, studName, class)`; `BOOK(bookId, bookTitle, author)`; `LOAN(studId, bookId, loanDate)` with `studId` and `bookId` as foreign keys referencing STUDENT and BOOK respectively.

With well-designed, normalised relations in hand, we are ready for the next part of the chapter: protecting these tables with **integrity constraints**, and then bringing them to life with the SQL language.

5 DATABASE INTEGRITY AND INTEGRATION ACTIVITY I

In the previous section we learnt how to transform an E-R model into a well-structured rela-

tional schema, and how normalisation up to 3NF removes redundancy and update anomalies. But a normalised schema is not enough. Imagine the database of a school in Yaoundé: normalisation guarantees that each student's name is stored once, yet nothing in the tables themselves prevents a secretary from typing a mark of 250 out of 100, or from registering a candidate in a class that does not exist. **Integrity** is the missing layer of protection: it is the set of rules that keeps the data correct and meaningful throughout its life. This section defines the three classical forms of integrity, studies the cascading actions that enforce referential integrity automatically, and ends with a full integration activity.

5.1 The Concept of Data Integrity

Observation. A bank in Douala stores account balances. If a software bug lets a balance become negative when the account is not allowed to be overdrawn, or lets a transfer debit one account without crediting the other, the structure of the database is perfectly fine — it is the content that has become false. Data integrity is about protecting the content.

DEFINITION **Data Integrity**

Data integrity is the property that data in a database remains accurate (it reflects reality), consistent (it does not contradict itself anywhere in the database) and valid (it respects all the rules of the real-world domain) throughout its entire lifetime: at insertion, at update and at deletion.

Integrity is enforced through **integrity constraints**: rules declared to the DBMS, which then automatically rejects any operation that would violate them. The relational model recognises three standard kinds of constraint, plus user-defined rules.

Constraint	What it protects	Typical mechanism
Entity integrity	Identity of each row	PRIMARY KEY (not NULL, unique)
Domain integrity	Validity of each value	Data types, CHECK, NOT NULL, DEFAULT, UNIQUE
Referential integrity	Consistency between tables	FOREIGN KEY, cascading actions
User-defined rules	Business meaning	CHECK, triggers, stored procedures, assertions

5.2 Entity Integrity

Intuition. In the GCE examination, every candidate has a unique candidate number. A candidate "without a number" simply cannot be processed: nobody could tell which script belongs to him. The same logic applies to every table.

DEFINITION **Entity Integrity**

Entity integrity states that no component of a primary key may ever be `NULL`, and that

primary key values must be unique within the table. Every row must be identifiable at all times.

The reason is fundamental: the primary key is the only guaranteed way to address a row. If a key value were `NULL`, the row would become unreachable by joins and by updates, and two rows could become indistinguishable. Note that a `UNIQUE` constraint allows `NULL` values (in standard SQL), whereas a `PRIMARY KEY` does not.

EXEMPLE **Entity integrity in action**

In `STUDENT(stud_id, name, class_id)`, the insertion `INSERT INTO STUDENT VALUES (NULL, 'Amina', 3);` is rejected by the DBMS, and inserting a second row with `stud_id = 12` when it already exists is also rejected.

5.3 Domain Integrity

Intuition. A mark at the GCE is a number between 0 and 100; a phone number has a fixed format; a date of birth cannot be in the future. Each attribute draws its values from a well-defined **domain**.

DEFINITION **Domain Integrity**

Domain integrity states that every value stored in a column must belong to the declared domain of that column: correct data type, permitted range or set of values, required format, and nullability. It is enforced with data types, `NOT NULL`, `DEFAULT`, `UNIQUE` and `CHECK` constraints.

EXEMPLE **Declaring domains in SQL**

```
mark DECIMAL(5,2) CHECK (mark BETWEEN 0 AND 100),
gender CHAR(1) CHECK (gender IN ('M','F')),
name VARCHAR(40) NOT NULL,
reg_date DATE DEFAULT CURRENT_DATE
```

WARNING **Domain integrity is not referential integrity**

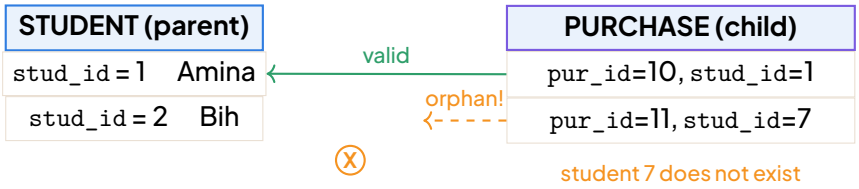
A `CHECK (class_id BETWEEN 1 AND 50)` does not guarantee that the class exists; it only checks the range. Existence across tables is the job of the foreign key (referential integrity). Do not confuse the two in the examination.

5.4 Referential Integrity

Observation. A canteen records purchases: each purchase mentions a student. If the student is later deleted from the `STUDENT` table but his purchases remain, those purchases now point to nobody. Such rows are called **orphaned records**.

DEFINITION **Referential Integrity**

Referential integrity states that a foreign key value must either be `NULL` (if nulls are allowed by the column definition) or match the value of an existing primary key (or a `UNIQUE` key) in the referenced (parent) table. A foreign key may never point to a row that does not exist.



Referential integrity violation: purchase 11 is an orphan because `stud_id=7` matches no student. Correction: either insert student 7 first, or reject the purchase.

Violations can arise in two directions:

- **Inserting/updating a child** with a foreign key that matches no parent (rejected outright).
- **Deleting/updating a parent** that still has children — this is where cascading actions decide what happens.

5.5 Cascading Actions: ON UPDATE and ON DELETE

When a referenced parent row is deleted, or its primary key is modified, the DBMS must choose a behaviour for the child rows. This behaviour is declared with the foreign key definition.

PROPERTY **Effects of cascading actions**

For a foreign key declared as `FOREIGN KEY (c) REFERENCES P(k) ON DELETE … ON UPDATE …`:

- `CASCADE`: the operation is propagated — deleting the parent deletes all its children; updating the parent's key updates the foreign key in all children.
- `SET NULL`: the foreign key of every child is set to `NULL` (the children survive but are detached). Requires the column to be nullable (no `NOT NULL`).
- `RESTRICT` (or `NO ACTION`): the operation on the parent is refused as long as at least one

child exists. This is the safest, default behaviour in most DBMS.

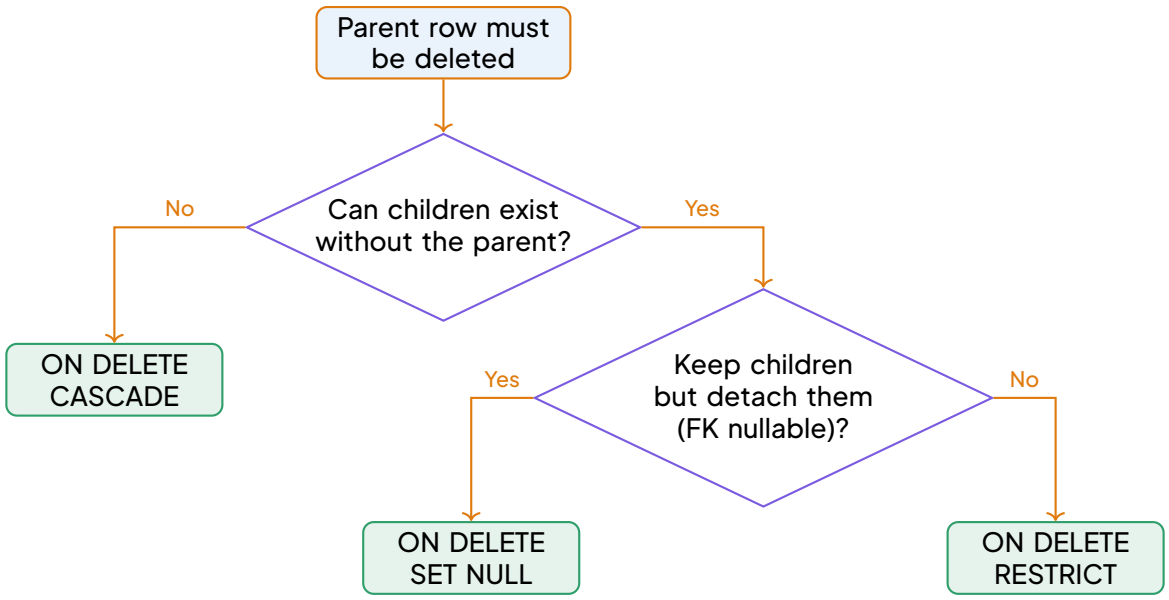
- `SET DEFAULT`: the foreign key is reset to its default value (less common, requires a `DEFAULT` clause on the column).

EXEMPLE **Worked example: deleting a class**

`CLASS(class_id, label)` and `STUDENT(stud_id, name, class_id)` with `FOREIGN KEY (class_id) REFERENCES CLASS ON DELETE CASCADE`.
Deleting class `3` automatically deletes every student of class 3 — possibly hundreds of rows and, transitively, their purchases if those also cascade. With `ON DELETE RESTRICT`, the deletion is refused until the students are moved elsewhere; with `SET NULL`, students survive with `class_id = NULL`.

WARNING **Common mistake: creating orphans or mass deletions**

The two classic traps are: (i) deleting a parent without any declared action in a system that does not enforce foreign keys, leaving orphaned rows that corrupt reports; and (ii) choosing `ON DELETE CASCADE` carelessly, so that one innocent `DELETE` silently wipes out an entire branch of the database. Always declare foreign keys explicitly, and choose `CASCADE` only when a child genuinely cannot exist without its parent (e.g., an invoice line without its invoice, a payment without its purchase).



Decision flowchart for choosing the ON DELETE action of a foreign key.

5.6 User-Defined Integrity Rules and Business Rules

Beyond the three standard constraints, every organisation has its own **business rules**: "a candidate's mark must lie between 0 and 100", "a student may not borrow more than 3 books", "an order's delivery date must be after its order date". These are **user-defined integrity rules**, implemented with `CHECK` constraints (for single-row, static rules), triggers (for multi-row, cross-table, or dynamic rules), or stored procedures (for complex logic encapsulated in an API).

EXEMPLE **Business rules for a school database**

```
CHECK (mark BETWEEN 0 AND 100)
CHECK (date_birth < date_registration)
CHECK (fees_paid <= fees_due)
```

METHOD **Checklist for validating the integrity of a relational design**

1. Every table has a primary key; no key attribute is nullable (entity integrity).
2. Every column has an appropriate data type, length, nullability, and `CHECK`/`DEFAULT` where needed (domain integrity).
3. Every foreign key references an existing primary (or unique) key, with matching data type and size (referential integrity).

4. For every foreign key, `ON DELETE` and `ON UPDATE` actions are deliberately chosen (CASCADE / SET NULL / RESTRICT), not left to chance.
5. All business rules of the situation are written down and each is implemented (CHECK, trigger, or application code).
6. Test: try inserting an orphan child, deleting a referenced parent, and entering an out-of-range value — the DBMS must react correctly each time.

WORKED EXAMPLE 5 **Applying the integrity rules**

Statement. Tables: `AUTHOR(auth_id, name)` and `BOOK(book_id, title, price, auth_id)`. (a) Which integrity rule is violated by `INSERT INTO BOOK VALUES (5,'X', 1200, 9);` when author 9 does not exist? (b) Which rule is violated by `price = -500`? (c) The librarian wants books to be detached (not deleted) when an author is removed, and any change of an author's identifier to be propagated. Write the foreign key clause.

SOLUTION

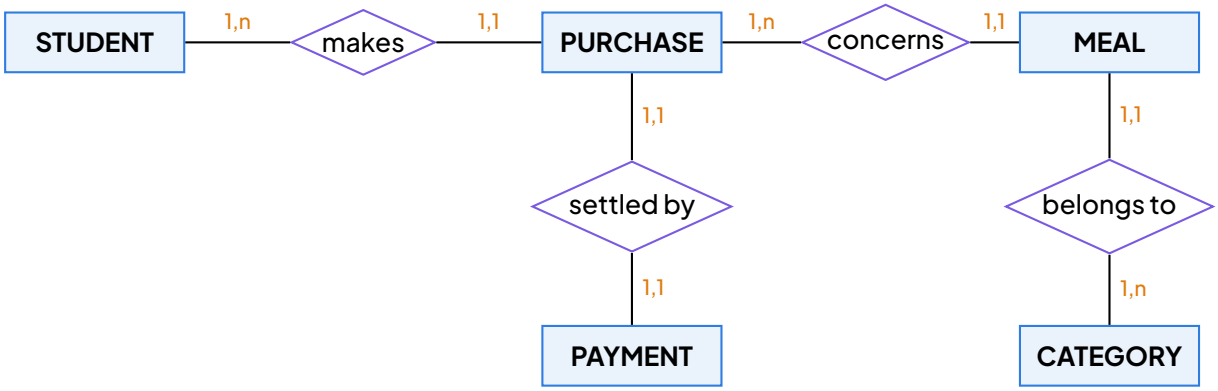
Solution. (a) Referential integrity: the foreign key `auth_id = 9` matches no existing primary key, so the insertion must be rejected. (b) Domain integrity: a negative price violates the domain rule `CHECK (price >= 0)`. (c) `FOREIGN KEY (auth_id) REFERENCES AUTHOR(auth_id) ON DELETE SET NULL ON UPDATE CASCADE` — books survive with `auth_id = NULL`, and any change of an author's identifier is propagated automatically. (Note: `auth_id` in `BOOK` must not have `NOT NULL` for `SET NULL` to work.)

5.7 Integration Activity I: College Canteen Management Database

EXPERIMENT **Case study: "College Canteen Management"**

A college in Buea wants a database for its canteen. Requirements: students buy meals; each purchase concerns one student and one meal; each meal belongs to a category (e.g., breakfast, lunch); payments are recorded per purchase. We build the complete conceptual and logical model, then verify integrity.

Step 1 — Conceptual model (E–R diagram). Entities: `STUDENT`, `MEAL`, `CATEGORY`, `PURCHASE` (the association between STUDENT and MEAL, carrying date and quantity), and `PAYMENT` linked to `PURCHASE`.



E-R diagram of the College Canteen Management database.

Step 2 — Logical (relational) model. Each entity becomes a table; each 1,n relationship carries a foreign key on the "many" side. The `PURCHASE` entity is a weak entity identified relative to `STUDENT` and `MEAL`, but we assign it a surrogate key `pur_id` for simplicity.

- `CATEGORY(cat_id, cat_name)`
- `MEAL(meal_id, meal_name, price, cat_id)`
- `STUDENT(stud_id, name, class)`
- `PURCHASE(pur_id, pur_date, quantity, stud_id, meal_id)`
- `PAYMENT(pay_id, amount, pay_date, pur_id)`

Step 3 — Normalisation check (3NF).

- **1NF:** All attributes are atomic (no list of meals in one cell, no repeating groups).
- **2NF:** In `PURCHASE`, every non-key attribute (`pur_date`, `quantity`) depends on the whole key `pur_id`, not on part of it (the key is simple, so 2NF is automatically satisfied).
- **3NF:** No transitive dependency — `price` is stored in `MEAL`, not repeated in `PURCHASE`; `cat_name` lives only in `CATEGORY`. The design is in 3NF.

Step 4 — Integrity rules.

- **Entity integrity:** All five primary keys are `NOT NULL` and unique.
- **Domain integrity:** `price >= 0`, `quantity >= 1`, `amount >= 0`, `class NOT NULL`, `pur_date <= CURRENT_DATE`.
- **Referential integrity:**
  - `MEAL.cat_id → CATEGORY` (`ON DELETE RESTRICT`: a category in use must not disappear).
  - `PURCHASE.stud_id → STUDENT` and `PURCHASE.meal_id → MEAL` (`ON DELETE RESTRICT`: keep sales history for accounting).
  - `PAYMENT.pur_id → PURCHASE` (`ON DELETE CASCADE`: a payment is meaningless without its purchase).

- **Business rules:** `amount = price × quantity` checked by trigger; a student may not have two purchases with the same `pur_id` (guaranteed by the key).

EXERCISE 5 **Integrity audit**

In the canteen design above: (1) What happens to `PAYMENT` rows if a `PURCHASE` is deleted, given `ON DELETE CASCADE`? (2) Why is `RESTRICT` preferred for `PURCHASE.stud_id`? (3) A clerk tries `INSERT INTO PURCHASE VALUES (40, '2025-02-10', 2, 99, 3);` but student 99 does not exist. Which constraint reacts, and what does the DBMS do?

SOLUTION **Exercise 1**

(1) Every payment linked to that purchase is automatically deleted — acceptable here because a payment cannot exist without its purchase. (2) Deleting a student must not erase the sales history (accounts would become wrong); `RESTRICT` forces the administrator to handle the student's purchases first (e.g., reassign or archive). (3) The foreign key constraint (referential integrity) on `stud_id` reacts; the DBMS rejects the insertion with an error, and no orphan row is created.

KEY POINTS **Key points**

- Integrity = data that stays accurate, consistent and valid over its whole lifetime, enforced automatically by the DBMS.
- Entity integrity: primary key never NULL, always unique. Domain integrity: types, ranges, formats, CHECK. Referential integrity: a foreign key matches an existing primary key or is NULL.
- On delete/update of a parent: `CASCADE` propagates, `SET NULL` detaches, `RESTRICT` refuses. Choose deliberately — careless CASCADE destroys data; missing foreign keys create orphans.
- Business rules complete the standard constraints via CHECK, triggers and procedures.
- A complete design always ends with an integrity audit: keys, domains, foreign keys with actions, and business rules — as done for the canteen case study.

6 SQL: DATA DEFINITION AND DATA MANIPULATION

In the previous section we studied **database integrity**: entity integrity, referential integrity and

domain integrity. We stated these rules in English, but a rule written in English protects nobody. The question that naturally arises is: how do we actually tell the DBMS to enforce these rules, and how do we put data into the database once the structure exists? The answer is a single, standardised language understood by virtually every relational DBMS in the world — from MySQL on a school server in Bamenda to Oracle in a bank in Douala. That language is **SQL**. In this section we learn to build tables with the Data Definition Language (DDL) and to fill and modify them with the Data Manipulation Language (DML).

6.1 Introduction to SQL

Observation. When the secretary of a college wants the list of all Form Five girls, she does not open the filing cabinet and read every folder: she states what she wants and the clerk finds it. SQL works exactly the same way. You describe what you want; the DBMS decides how to get it.

DEFINITION **SQL**

SQL (Structured Query Language) is the standard **declarative** language used to define, manipulate, query and control relational databases. Declarative means the user states the desired result, not the step-by-step procedure to obtain it.

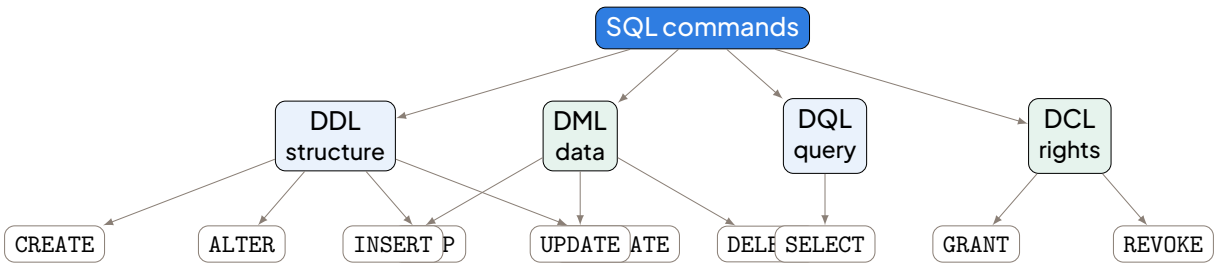
SQL was developed at IBM in the 1970s (under the name SEQUEL, linked to Codd's relational model) and later standardised by ANSI (1986) and ISO (1987). Because it is a standard, the same core commands work on MySQL, PostgreSQL, Oracle, Microsoft SQL Server, SQLite and many others — only minor details change from one product to another.

KEY POINTS **SQL Keywords are Case-Insensitive**

The SQL standard specifies that keywords (`SELECT`, `CREATE`, `WHERE`, etc.) are case-insensitive. `create table` and `CREATE TABLE` are equivalent. By convention, this course writes keywords in uppercase and user-defined names (tables, columns) in lower_case_with_underscores for readability.

SQL commands are traditionally grouped into four categories:

- **DDL** (Data Definition Language): defines and modifies the structure of the database — `CREATE`, `ALTER`, `DROP`, `TRUNCATE`.
- **DML** (Data Manipulation Language): inserts, modifies and removes data — `INSERT`, `UPDATE`, `DELETE`.
- **DQL** (Data Query Language): retrieves data — `SELECT` (studied in the next section).
- **DCL** (Data Control Language): manages access rights — `GRANT`, `REVOKE`.



Classification of SQL commands into DDL, DML, DQL and DCL.

DEFINITION **DDL and DML**

The **Data Definition Language** (DDL) is the subset of SQL used to create, modify and delete database objects (tables, views, indexes). Its main commands are `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`, `TRUNCATE TABLE`, `CREATE VIEW` and `CREATE INDEX`. The **Data Manipulation Language** (DML) is the subset of SQL used to work on the rows stored inside tables. Its main commands are `INSERT INTO`, `UPDATE` and `DELETE`.

WARNING **DDL acts on structure, DML acts on content**

A classic GCE trap: `DROP TABLE` is DDL (it removes the table itself), whereas `DELETE FROM` is DML (it removes rows but the table survives). `TRUNCATE TABLE` is also DDL — it removes all rows and resets identity counters, but keeps the structure. Keep the three families clearly separated in your mind.

6.2 SQL Data Types

Every column of a table must be declared with a **data type**, which fixes the kind of values it can hold — this is the first level of domain integrity. The main types required at this level are:

Type	Meaning	Example
`INTEGER` (or `INT`)	Whole numbers (typically 32-bit signed)	`42`
`DECIMAL(m,n)` (or `NUMERIC`)	Exact fixed-point numbers, *m* total digits, *n* after the decimal point	`DECIMAL(5,2)`: `999.99`
`CHAR(n)`	Fixed-length text, padded with spaces to exactly *n* characters	`CHAR(2)`: `'FS'`
`VARCHAR(n)`	Variable-length text, up to *n* characters (no padding)	`VARCHAR(30)`: `'Amina'`
`DATE`	Calendar date (year, month, day)	`'2008-05-17'`
`BOOLEAN`	Logical values `TRUE` / `FALSE` (some DBMSs use `BIT`, `TINYINT(1)` or `CHAR(1)`)	`TRUE`

WARNING **Quotes and formats**

Text and date values are written between single quotes (`'Amina'`, `'2008-05-17'`); numbers are written without quotes. Writing `marks = '15'` treats 15 as text — a frequent source of errors. Dates are conventionally written `YYYY-MM-DD` (ISO 8601 format).

6.3 DDL: Creating Tables with Constraints

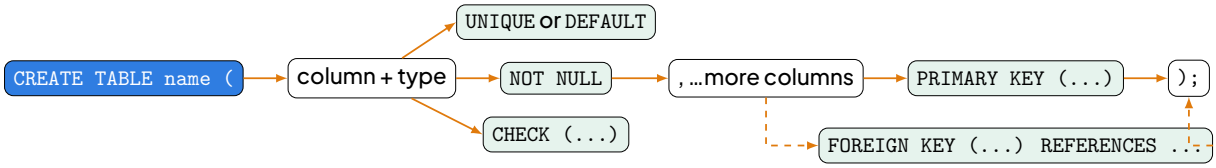
DEFINITION **Constraint**

A **constraint** is a rule attached to a column or a table that the DBMS enforces automatically on every insertion or update, in order to guarantee the integrity of the data. The six main constraint types are:

- `PRIMARY KEY`: uniquely identifies each row; implies `NOT NULL` and `UNIQUE` (entity integrity).
- `FOREIGN KEY`: a column (or set of columns) referencing the primary key of another table (referential integrity).
- `NOT NULL`: the column must always receive a value (no `NULL` allowed).
- `UNIQUE`: no two rows may share the same value in that column (or column combination).
- `CHECK`: imposes a Boolean condition that every value must satisfy (domain integrity).
- `DEFAULT`: supplies a value automatically when none is given during insertion.

METHOD Template syntax of CREATE TABLE

1. Choose a meaningful table name (singular, uppercase or lower-case_with_underscores).
2. List each column: `column_name data_type [column_constraints]`.
3. Attach column-level constraints: `NOT NULL`, `UNIQUE`, `DEFAULT value`, `CHECK (condition)`.
4. Declare the primary key (column or composite): `PRIMARY KEY (col1, col2, ...)`.
5. Declare each foreign key: `FOREIGN KEY (col) REFERENCES other_table(ref_col) [ON DELETE action] [ON UPDATE action]`.
6. End with a semicolon.



Syntax tree of the `CREATE TABLE` statement: mandatory parts in blue/white, optional constraint clauses in green.

EXEMPLE Creating the CLASS and STUDENT tables

```
CREATE TABLE CLASS (
  class_id INTEGER PRIMARY KEY,
  class_name VARCHAR(10) NOT NULL UNIQUE,
  level VARCHAR(15) DEFAULT 'Lower Sixth'
);
CREATE TABLE STUDENT (
  stud_id INTEGER PRIMARY KEY,
  full_name VARCHAR(40) NOT NULL,
  gender CHAR(1) CHECK (gender IN ('M','F')),
  birth_date DATE,
```

```
    class_id INTEGER,
    FOREIGN KEY (class_id) REFERENCES CLASS(class_id)
);
```

Notice how the integrity rules of the previous section are now enforced by the machine: the primary keys guarantee entity integrity, the foreign key guarantees referential integrity, and `NOT NULL`, `UNIQUE`, `CHECK` and `DEFAULT` guarantee domain integrity.

6.4 DDL: Modifying and Removing Structures

ALTER TABLE changes the structure of an existing table without losing its data:

```
ALTER TABLE STUDENT ADD email VARCHAR(50);                    (add a column)
ALTER TABLE STUDENT DROP COLUMN email;                        (remove a column)
ALTER TABLE STUDENT MODIFY full_name VARCHAR(60);             (change a type/size)
ALTER TABLE STUDENT ADD CONSTRAINT chk_email CHECK (email LIKE '%@%.%');   (add a constraint)
ALTER TABLE STUDENT DROP CONSTRAINT chk_email;                (remove a constraint)
```

TRUNCATE TABLE removes all rows from a table, resets any auto-increment counters, and deallocates storage — it is a DDL operation (cannot be rolled back in many DBMSs, does not fire triggers):

```
TRUNCATE TABLE STUDENT;
```

DROP TABLE removes the table completely — structure and data:

```
DROP TABLE STUDENT;
```

Views and indexes (overview). `CREATE VIEW` defines a virtual table based on a stored query, useful to show each user only what concerns them; `DROP VIEW` removes it. `CREATE INDEX` builds an auxiliary structure that speeds up searches on a column (like the index at the back of a textbook), at the cost of extra storage and slower insertions.

PROPERTY **DROP TABLE versus TRUNCATE TABLE versus DELETE FROM**

- `DELETE FROM STUDENT;` removes all the rows but the (empty) table remains in the database and can receive new rows. It is DML, can be rolled back, fires triggers.
- `TRUNCATE TABLE STUDENT;` removes all rows, resets identity/auto-increment, deallocates pages. It is DDL, typically cannot be rolled back, does not fire triggers.
- `DROP TABLE STUDENT;` removes the table definition itself together with its rows, in-

dexes, constraints, triggers; afterwards the table no longer exists and must be recreated with `CREATE TABLE` before any insertion.

(Not examinable as a proof, but frequently asked as a short-answer question.)

6.5 DML: Inserting, Updating and Deleting Rows

INSERT INTO adds new rows. The order of values must match the order of columns (explicit column list is recommended for clarity and safety):

```
INSERT INTO CLASS (class_id, class_name, level)
VALUES (1, 'USC A', 'Upper Sixth');
INSERT INTO STUDENT VALUES (101, 'Ngong Amina', 'F', '2008-03-12', 1);
```

Several rows can be inserted in one statement (multiple-row insert):

```
INSERT INTO CLASS (class_id, class_name, level) VALUES
(2, 'USC B', 'Upper Sixth'), (3, 'LSC A', 'Lower Sixth');
```

UPDATE modifies existing rows; **DELETE** removes rows. Both act on every row that satisfies the WHERE condition:

```
UPDATE STUDENT SET class_id = 2 WHERE stud_id = 101;
DELETE FROM STUDENT WHERE stud_id = 101;
```

WARNING **The fatal missing WHERE**

`UPDATE STUDENT SET class_id = 2;` (without `WHERE`) changes the class of every student in the school. `DELETE FROM STUDENT;` (without `WHERE`) empties the entire table. There is no "undo" button in basic SQL. Golden rule: before running an `UPDATE` or `DELETE`, first run the corresponding `SELECT * FROM table WHERE ...` to check exactly which rows will be affected.

DID YOU KNOW? **Transactions: COMMIT and ROLLBACK**

Most professional DBMSs group operations into transactions. Changes made inside a transaction are provisional: `COMMIT` makes them permanent, while `ROLLBACK` cancels everything since the last commit — a lifebuoy when you realise you just deleted the wrong rows. This goes slightly beyond the core syllabus but earns marks as a well-chosen extra in GCE essays.

6.6 Practical Session: Building the SCHOOL Database

EXPERIMENT **Creating and populating SCHOOL**

We build a small school database with four tables: `CLASS`, `STUDENT`, `SUBJECT` and `RESULT`.

1. Create the structure (DDL):

```
CREATE TABLE SUBJECT (
  subj_id INTEGER PRIMARY KEY,
  subj_name VARCHAR(30) NOT NULL UNIQUE);
CREATE TABLE RESULT (
  stud_id INTEGER,
  subj_id INTEGER,
  mark DECIMAL(4,1) CHECK (mark BETWEEN 0 AND 20),
  PRIMARY KEY (stud_id, subj_id),
  FOREIGN KEY (stud_id) REFERENCES STUDENT(stud_id),
  FOREIGN KEY (subj_id) REFERENCES SUBJECT(subj_id));
```

   (`CLASS` and `STUDENT` were created in the worked example above.)

2. Populate the tables (DML):

```
INSERT INTO SUBJECT VALUES (1, 'Computer Science');
INSERT INTO SUBJECT VALUES (2, 'Mathematics');
INSERT INTO RESULT VALUES (101, 1, 16.5);
INSERT INTO RESULT VALUES (101, 2, 14.0);
```

3. Test the constraints deliberately: try `INSERT INTO RESULT VALUES (101, 1, 23);` — the `CHECK` rejects it; try a `subj_id` that does not exist in `SUBJECT` — the foreign key rejects it. The DBMS itself is now guarding integrity.

WORKED EXAMPLE 6 **DDL and DML in context**

Statement. (a) Write the SQL statement that creates a table `TEACHER` with columns `teach_id` (integer, primary key), `teach_name` (text up to 40 characters, compulsory) and `salary` (decimal, must be at least 50000 FCFA). (b) Insert the teacher Mrs. Bih Comfort, id 7, salary 185000. (c) Give her a 10% raise. (d) Remove teacher number 12. What happens if the `WHERE` clause is forgotten in (d)?

SOLUTION

Solution. (a)

```
CREATE TABLE TEACHER (
  teach_id INTEGER PRIMARY KEY,
  teach_name VARCHAR(40) NOT NULL,
  salary DECIMAL(10,2) CHECK (salary >= 50000));
```

(b) `INSERT INTO TEACHER VALUES (7, 'Bih Comfort', 185000);` (c) `UPDATE TEACHER SET salary = salary * 1.10 WHERE teach_id = 7;` — the new salary is 185000 × 1.10 = 203500 FCFA. (d) `DELETE FROM TEACHER WHERE teach_id = 12;` removes only that teacher. Without `WHERE`, `DELETE FROM TEACHER;` deletes all teachers: the table remains but is completely empty.

EXERCISE 6 **Your turn**

1. Write the DDL to create a table `BOOK` (`book_id` primary key, `title` compulsory and unique, `price` with default 2500 FCFA, `pages` must be positive).
2. Insert two books of your choice in a single statement.
3. Write a statement increasing by 500 FCFA the price of every book cheaper than 3000 FCFA — including the safety check you would perform first.
4. Explain the difference between `DROP TABLE BOOK;` and `DELETE FROM BOOK;`.

SOLUTION **Exercise 1**

1. `CREATE TABLE BOOK (book_id INTEGER PRIMARY KEY, title VARCHAR(60) NOT NULL UNIQUE, price DECIMAL(8,2) DEFAULT 2500, pages INTEGER CHECK (pages > 0));`
2. `INSERT INTO BOOK (book_id, title, price, pages) VALUES (1, 'New Dawn', 3000, 180), (2, 'Hills of Buea', 2500, 120);`
3. First `SELECT * FROM BOOK WHERE price < 3000;` to preview the rows, then `UPDATE BOOK SET price = price + 500 WHERE price < 3000;`
4. `DELETE FROM BOOK;` empties the table but keeps its structure; `DROP TABLE BOOK;` destroys the table definition and its data — the table must be recreated before further use.

KEY POINTS **Key points**

- SQL is a declarative, standardised language; its commands fall into DDL (structure), DML (data), DQL (queries) and DCL (rights).
- `CREATE TABLE` declares columns with types (`INTEGER`, `DECIMAL`, `CHAR`, `VARCHAR`, `DATE`, `BOOLEAN`) and constraints (`PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`) that enforce integrity automatically.
- `ALTER TABLE` modifies structure; `DROP TABLE` destroys it; `TRUNCATE TABLE` empties it and resets counters (DDL); `DELETE FROM` only empties it (DML).
- `INSERT` adds rows; `UPDATE` and `DELETE` act on all rows matching the WHERE clause — never run them without checking the condition first.

Now that our SCHOOL database exists and is populated, the next section tackles the most used command of all: `SELECT`, together with joins and aggregate functions, to retrieve the information we have so carefully stored.

7 SQL: QUERIES, JOINS AND AGGREGATE FUNCTIONS

In the previous section we mastered the language of structure (`CREATE`, `ALTER`, `DROP`) and the language of change (`INSERT`, `UPDATE`, `DELETE`). We now turn to the most frequently used part of SQL: the **Data Query Language (DQL)**. The sole command here is `SELECT`, yet its expressive power is immense. It allows us to answer questions like "What is the average mark per class in Government Bilingual High School?" or "Which students offer both Mathematics and Physics?" This section builds your competence from simple filtering to complex joins and aggregation, exactly as required for GCE Paper 1 and Paper 2.

7.1 The SELECT Statement: Core Syntax and Execution Order

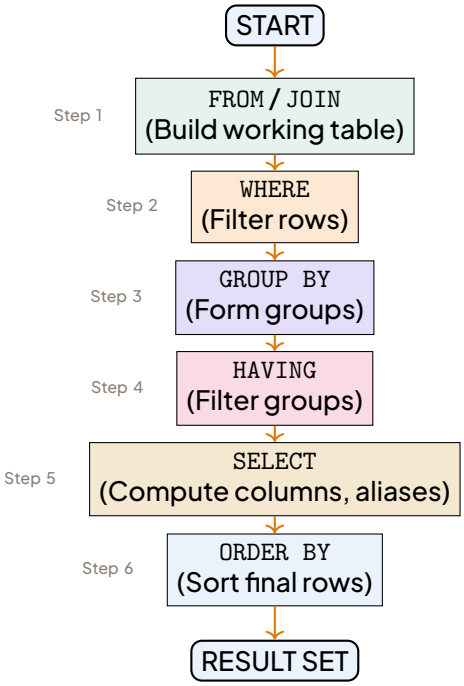
Every `SELECT` statement is logically evaluated in a fixed order, which is not the order in which we write the clauses. Understanding this order explains why you cannot use a column alias in the `WHERE` clause (the alias does not exist yet) but you can use it in `ORDER BY`.

DEFINITION **The Six Main Clauses of SELECT and Their Logical Execution Order**

The standard `SELECT` statement consists of the following clauses. The logical execution order (1 to 6) determines what data is visible to each subsequent step.

1. `FROM` / `JOIN` – Determine the working table(s) and combine rows.
2. `WHERE` – Filter individual rows before grouping.
3. `GROUP BY` – Arrange the remaining rows into groups.

4. `HAVING` – Filter groups after aggregation.
5. `SELECT` – Choose and compute the output columns (expressions, aggregates, aliases).
6. `ORDER BY` – Sort the final result set.



Logical execution order of a SELECT statement. Write clauses in the order SELECT-FROM-WHERE-GROUP BY-HAVING-ORDER BY, but the engine executes them 1–6.

EXEMPLE **Basic SELECT, Aliases and DISTINCT**

Consider a table `STUDENT(Student_ID, FirstName, LastName, Class_ID, DateOfBirth)`.

1. `SELECT FirstName, LastName FROM STUDENT;` – All names.
2. `SELECT FirstName AS GivenName, LastName AS Surname FROM STUDENT;` – **AS** renames output columns (alias).
3. `SELECT DISTINCT Class_ID FROM STUDENT;` – Unique class identifiers only (removes duplicates).

7.2 Filtering Data with the WHERE Clause

The `WHERE` clause operates on individual rows (Step 2). It accepts any boolean expression.

KEY POINTS **WHERE Condition Toolbox**

- Comparison: =, <> (or !=), >, <, >=, <=.
- Logical: `AND`, `OR`, `NOT`. Precedence: `NOT` > `AND` > `OR`. Use parentheses for clarity.
- Range: `Mark BETWEEN 10 AND 15` (inclusive).
- Set membership: `Class_ID IN ('L6A', 'L6B', 'U6A')`.
- Pattern matching: `LastName LIKE 'N%'` (starts with N), `FirstName LIKE '_a%'` (second letter is 'a'). Wildcards: % (zero or more chars), _ (exactly one char).
- Null test: `DateOfBirth IS NULL`, `GuardianPhone IS NOT NULL`. Never write `= NULL`.

EXEMPLE **Filtering Examples on STUDENT**

1. Students in Lower Sixth: `SELECT * FROM STUDENT WHERE Class_ID LIKE 'L6%';`
2. Students born after 1 Jan 2008: `SELECT * FROM STUDENT WHERE DateOfBirth > '2008-01-01';`
3. Students in L6A or L6B with a known phone: `SELECT * FROM STUDENT WHERE Class_ID IN ('L6A','L6B') AND GuardianPhone IS NOT NULL;`

WARNING **Common Mistake: Aggregate in WHERE**

You cannot write `WHERE AVG(Mark) > 10`. Aggregates are computed in Step 5 (`SELECT`) or Step 3/4 (`GROUP BY`/`HAVING`), but `WHERE` runs at Step 2 on raw rows. The engine does not know the average yet. Use `HAVING` after `GROUP BY` instead.

7.3 Sorting and Limiting Results

Sorting is the last logical step (Step 6). The result set can be ordered by column name, alias, position (1, 2, ...) or expression.

EXEMPLE **ORDER BY and LIMIT**

1. `SELECT FirstName, LastName, DateOfBirth FROM STUDENT ORDER BY DateOfBirth DESC;` – Oldest first.
2. `SELECT FirstName, LastName, Class_ID FROM STUDENT ORDER BY Class_ID,`

`LastName;` – Sort by class, then name.

3. `SELECT * FROM STUDENT ORDER BY DateOfBirth DESC LIMIT 5;` – Top 5 oldest (MySQL/PostgreSQL/SQLite).
4. `SELECT TOP 5 * FROM STUDENT ORDER BY DateOfBirth DESC;` – SQL Server / MS Access syntax.

7.4 Joining Tables: Combining Related Data

A relational database splits data into multiple tables to avoid redundancy. To answer cross-table questions we must **join** tables on common columns (usually a Primary Key / Foreign Key pair).

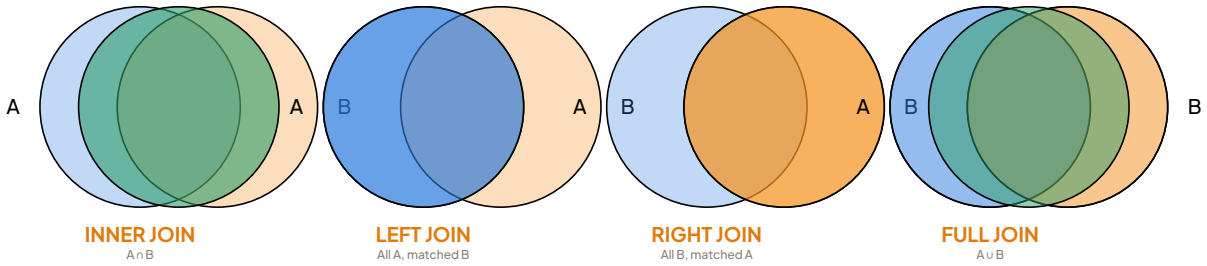
PROPERTY **The Cartesian Product Problem**

If you write `FROM TableA, TableB` without a join condition, SQL produces the **Cartesian product**: every row of A paired with every row of B. If A has 10,000 rows and B has 100 rows, the result has 1,000,000 rows – almost never what you want. Always specify a join condition.

METHOD **Choosing the Correct Join Type**

Ask: "Do I need rows from the left table even if they have no match in the right?" and vice versa.

1. INNER JOIN: Only rows with a match in both tables. (Default `JOIN`).
2. LEFT OUTER JOIN: All rows from left table; matching rows from right; `NULL`s on right side where no match.
3. RIGHT OUTER JOIN: All rows from right table; matching rows from left; `NULL`s on left side where no match.
4. FULL OUTER JOIN: All rows from both tables; `NULL`s where no match. (Not in MySQL; emulate with `UNION` of LEFT and RIGHT).
5. SELF JOIN: A table joined to itself (e.g., `STAFF` table with `SupervisorID` referencing `Staff_ID`). Use aliases.



Venn diagram illustration of join types. Shaded area = rows returned.

EXEMPLE **Joins on the SCHOOL Database**

Assume the following schema (simplified for GCE):

Table	Key Columns
STUDENT	Student_ID (PK), Name, Class_ID (FK), SchoolName
CLASS	Class_ID (PK), ClassName, FormTeacher_ID (FK)
SUBJECT	Subject_ID (PK), SubjectName
MARK	Student_ID (FK), Subject_ID (FK), Score, Term
TEACHER	Teacher_ID (PK), Name, Subject_ID (FK)

1. INNER JOIN – Students and their marks (only students who have marks).

```
1. SELECT S.Name, M.Subject_ID, M.Score
2. FROM STUDENT S
3. INNER JOIN MARK M ON S.Student_ID = M.Student_ID;
```

2. LEFT JOIN – All students, with marks if they exist (shows students with no marks yet).

```
1. SELECT S.Name, M.Subject_ID, M.Score
2. FROM STUDENT S
3. LEFT JOIN MARK M ON S.Student_ID = M.Student_ID;
```

3. SELF JOIN – Find each teacher and their head of department (assuming Teacher table has HOD_ID).

```
1. SELECT T1.Name AS Teacher, T2.Name AS HOD
2. FROM TEACHER T1
3. LEFT JOIN TEACHER T2 ON T1.HOD_ID = T2.Teacher_ID;
```

WARNING **Common Mistake: Missing Join Condition**

Writing `FROM STUDENT, MARK` without `ON` or `WHERE STUDENT.Student_ID = MARK.Student_ID` produces a Cartesian product. In an exam, this loses marks for "incorrect result set" and "inefficient query". Always verify the number of join conditions = number of tables − 1 (for a connected join graph).

7.5 Aggregate Functions and Grouping

Aggregates collapse multiple rows into a single summary value. They are computed after `GROUP BY` (Step 3) or over the whole filtered table (if no `GROUP BY`).

DEFINITION **Standard Aggregate Functions**

- `COUNT(*)` – Number of rows in group. `COUNT(column)` – Number of non-NULL values.
- `SUM(column)` – Total (numeric only).
- `AVG(column)` – Arithmetic mean (numeric only).
- `MIN(column)`, `MAX(column)` – Smallest / largest value (any comparable type).

Important: Aggregates ignore `NULL` values (except `COUNT(*)`).

PROPERTY **WHERE vs HAVING – The Fundamental Distinction**

- `WHERE` filters rows before grouping (Step 2). It cannot contain aggregates.
- `HAVING` filters groups after aggregation (Step 4). It can contain aggregates.

Proof of concept: To find classes with average mark > 12, you must first form class groups, compute the average per group, then keep groups where average > 12. This is `HAVING AVG(Score) > 12`. You cannot do it in `WHERE`.

EXEMPLE **Aggregation Worked Examples**

1. Average mark per class (GROUP BY + HAVING)

```
1. SELECT C.ClassName, AVG(M.Score) AS AverageMark
2. FROM CLASS C
3. INNER JOIN STUDENT S ON C.Class_ID = S.Class_ID
```

```
4. INNER JOIN MARK M ON S.Student_ID = M.Student_ID
5. WHERE M.Term = 'Term 1'
6. GROUP BY C.ClassName
7. HAVING AVG(M.Score) > 10
8. ORDER BY AverageMark DESC;
```

Explanation: Join three tables, filter Term 1 rows (`WHERE`), group by class name, compute average per group, keep groups with average > 10 (`HAVING`), sort descending.

2. Number of students per subject (COUNT with LEFT JOIN)

```
1. SELECT Sub.SubjectName, COUNT(DISTINCT M.Student_ID) AS NumStudents
2. FROM SUBJECT Sub
3. LEFT JOIN MARK M ON Sub.Subject_ID = M.Subject_ID
4. GROUP BY Sub.SubjectName;
```

Note: `LEFT JOIN` ensures subjects with zero students appear (count = 0). `COUNT(DISTINCT ...)` avoids double-counting if a student has multiple marks in the same subject (e.g., multiple terms).

7.6 Nested Queries (Subqueries)

A **subquery** is a `SELECT` statement nested inside another statement. In GCE Paper 1, subqueries appear in `WHERE` clauses with `IN` or comparison operators (=, >, <, etc.). The inner query executes first, producing a value or set of values used by the outer query.

KEY POINTS **Subquery Types in GCE Context**

- Single-row subquery: Returns one value (one row, one column). Use with =, >, <, etc.
- Multi-row subquery: Returns a list of values (one column, multiple rows). Use with `IN`, `ANY`, `ALL`.
- Correlated subquery: References a column from the outer query (evaluated once per outer row). Examinable in Paper 2 practical; Paper 1 usually sticks to non-correlated.

EXEMPLE **Subquery Examples**

1. Students who scored above the overall average (Single-row, = / >)

```
1. SELECT Name
```

```
2. FROM STUDENT
3. WHERE Student_ID IN (
4.   SELECT Student_ID
5.   FROM MARK
6.   WHERE Score > (SELECT AVG(Score) FROM MARK)
7. );
```

2. Subjects taught by teachers named 'Ngwa' (Multi-row, IN)

```
1. SELECT SubjectName
2. FROM SUBJECT
3. WHERE Subject_ID IN (
4.   SELECT Subject_ID
5.   FROM TEACHER
6.   WHERE Name = 'Ngwa'
7. );
```

3. Students who scored above their class average in Mathematics (Correlated subquery)

```
1. SELECT S.Name
2. FROM STUDENT S
3. INNER JOIN MARK M ON S.Student_ID = M.Student_ID
4. INNER JOIN SUBJECT Sub ON M.Subject_ID = Sub.Subject_ID
5. WHERE Sub.SubjectName = 'Mathematics'
6.   AND M.Score > (
7.   SELECT AVG(M2.Score)
8.   FROM MARK M2
9.   INNER JOIN STUDENT S2 ON M2.Student_ID = S2.Student_ID
10.  INNER JOIN SUBJECT Sub2 ON M2.Subject_ID = Sub2.Subject_ID
11.  WHERE S2.Class_ID = S.Class_ID
12.  AND Sub2.SubjectName = 'Mathematics'
13.  );
```

7.7 Integration Activity: Exam-Style Worked Questions

The following exercises use the SCHOOL database defined earlier. They combine filtering, joining, aggregation and subqueries – typical of GCE Paper 1 Section B and Paper 2 Question 1.

WORKED EXAMPLE 7 **GCE Style: School Database Queries**

Schema reminder: `STUDENT(Student_ID, Name, Class_ID, SchoolName)` `CLASS(Class_ID, ClassName)` `SUBJECT(Subject_ID, SubjectName)` `MARK(Student_ID, Subject_ID, Score, Term)` `TEACHER(Teacher_ID, Name, Subject_ID)`

Question 1: List the names of all students in "Government Bilingual High School" who scored above 15 in Mathematics in "Term 2".

SOLUTION

Solution (Counting approach):

```
1. SELECT S.Name
2. FROM STUDENT S
3. WHERE (
4.  SELECT COUNT(DISTINCT M.Subject_ID)
5.  FROM MARK M
6.  WHERE M.Student_ID = S.Student_ID
7. ) = (
8.  SELECT COUNT(*) FROM SUBJECT
9. );
```

Explanation: For each student, count distinct subjects they have marks in. Compare to total number of subjects. Correlated subquery in `WHERE`.

EXERCISE 7 **Practice Set – No Solutions Provided**

1. Write a query to display the name of each teacher and the number of students they teach (via marks), including teachers with zero students.
2. Find the class(es) with the lowest average mark in Chemistry for Term 1.
3. List students who have never scored below 10 in any subject (use `NOT EXISTS` or `NOT IN` with a subquery).
4. Produce a report: `ClassName, SubjectName, NumberOfStudents, AverageScore,`

`HighestScore, LowestScore` for Term 2.

5. Explain why `WHERE AVG(Score) > 12` is illegal and rewrite it correctly.

KEY POINTS **Examiner's Checklist for SQL Queries**

- Select list: Only columns in `GROUP BY` or aggregates. No "orphan" columns.
- Join conditions: Every table (except the first) needs an `ON` clause. Count tables = count joins + 1.
- Outer join filters: Conditions on the "optional" table go in `ON`, not `WHERE`.
- `HAVING` vs `WHERE`: Aggregate filter → `HAVING`; row filter → `WHERE`.
- Aliases: Use them for clarity (`S`, `M`, `C`). Qualify columns (`S.Name`) when ambiguity exists.
- Terminology: "List" → `SELECT`; "For each" → `GROUP BY`; "Only those where average…" → `HAVING`.

DID YOU KNOW? **SQL in Cameroon's Digital Transformation**

The `SELECT` statement is the workhorse behind every dashboard in Cameroon's growing tech ecosystem: from the `MINESUP` student tracking system (aggregating enrolment per region) to mobile money transaction logs (joining `USERS`, `AGENTS`, `TRANSACTIONS` for fraud detection). Mastering `JOIN` types and `GROUP BY`/`HAVING` is not just for the GCE – it is the skill that turns raw data into decisions for banks, telcos (MTN, Orange), and e-government portals.

8 RDBMS IMPLEMENTATION, WEB AND OBJECT-ORIENTED INTEGRATION

In the previous sections we mastered SQL: how to define tables with `CREATE TABLE`, insert and modify data with `INSERT`, `UPDATE`, `DELETE`, and how to interrogate a database with `SELECT`, joins and aggregate functions. But SQL statements do not run in the air — they need a **software package** that stores the data, executes the queries, manages concurrent users and enforces security. In this final section we bring everything together: we choose and use a real **RDBMS package**, we connect our database to the **web** using the standard three-tier architecture, we see how **object-oriented programs** talk to relational tables through Object-Relational Mapping, and we close with the essential notions of **database security** and **backup**. This is where your design on paper becomes a working system — like the canteen database of a college in Yaoundé finally going live.

8.1 RDBMS Packages: Choosing and Using One

DEFINITION **Relational Database Management System (RDBMS)**

An **RDBMS** (Relational Database Management System) is a software package that allows users to create, store, manipulate, query and secure relational databases. It implements the relational model (tables, primary/foreign keys, integrity constraints, views) and usually accepts SQL as its standard language. It also provides concurrency control, crash recovery and user management.

Many RDBMS packages exist. The most common ones you will meet in the Cameroonian context and internationally are:

Package	Characteristics	Typical use
MySQL / MariaDB	Free and open-source, fast, very popular with web servers, large community	Websites, school management systems, SME applications
PostgreSQL	Free and open-source, very powerful, highly standards-compliant, advanced data types	Large professional systems, GIS, data warehousing
Microsoft Access	Commercial, part of MS Office, graphical interface, file-based, single-user focus	Small offices, personal databases, rapid prototyping
Oracle Database	Commercial, expensive, extremely powerful, high availability features	Banks, government, big enterprises, critical systems
SQLite	Free, public domain, very light, stored in a single file, no server process needed	Mobile phones, embedded applications, local storage in browsers

METHOD **Choosing an RDBMS package**

When choosing a package for a project, weigh the following criteria systematically:

1. Cost and licensing: free/open-source (MySQL, MariaDB, PostgreSQL, SQLite) vs commercial licences (Oracle, SQL Server, Access). Consider total cost of ownership (support, hardware).

2. Data volume and concurrency: number of simultaneous users, size of database (MB vs TB), transaction rate. A single-user canteen database can live in Access or SQLite; a national examination database needs Oracle or PostgreSQL.

3. Platform and deployment: does it run on Windows, Linux, macOS? Does it require a separate server process (MySQL, PostgreSQL, Oracle) or is it embedded (SQLite, Ac-

cess)?

4. Available skills and tooling: does the team know SQL? Are graphical administration tools needed (phpMyAdmin, pgAdmin, MySQL Workbench, SSMS)? Is there an ORM library for the chosen programming language?
5. Feature requirements: stored procedures, triggers, partitioning, replication, full-text search, JSON/XML support, spatial extensions.

DID YOU KNOW? **SQLite is everywhere**

The small SQLite engine is probably the most widely deployed database in the world: it is embedded in every Android phone, in iOS, in most web browsers (Chrome, Firefox), in many desktop applications (Skype, Adobe Lightroom) and in countless IoT devices. The phone in your pocket almost certainly contains several SQLite databases right now — your contacts, messages, browser history and app settings are stored in them.

8.2 Implementing a Designed Database in an RDBMS

A database can be created in two main ways, often combined:

- **Graphical tools**: in MS Access you draw tables in Design View, choose field types from menus and click to define relationships. Tools such as phpMyAdmin (for MySQL/MariaDB), pgAdmin (for PostgreSQL), MySQL Workbench or DBeaver offer similar point-and-click interfaces for creating tables, indexes and foreign keys visually.
- **SQL scripts**: you write the `CREATE DATABASE`, `CREATE TABLE`, `ALTER TABLE`, `CREATE INDEX`, `INSERT` and `SELECT` statements in a `.sql` text file and execute the script. This is precise, repeatable, version-controllable and easy to share — exactly what we practised in the previous sections. **This is the professional approach.**

Data already existing in a spreadsheet (for example an Excel list of students) can be **imported**: save the sheet as a CSV (comma-separated values) file, then use the import wizard of the RDBMS (phpMyAdmin "Import" tab, Access "External Data") or a bulk-load command such as `LOAD DATA INFILE` (MySQL/MariaDB) or `COPY` (PostgreSQL) to load it into a table efficiently.

METHOD **Steps to implement a designed database in an RDBMS**

1. Create the database and its schema: execute `CREATE DATABASE` and `CREATE TABLE` statements (with primary keys, foreign keys, `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT` constraints from your logical design). Create indexes on frequently searched columns.
2. Populate the tables: enter reference data with `INSERT` statements, through data-entry forms, or by importing CSV files from spreadsheets. Verify row counts and

spot-check values.

3. Query and validate: write and test the `SELECT` statements (with joins, subqueries and aggregates) that answer the users' real questions. Compare results with expected outputs from the requirements.
4. Secure the database: create user accounts with strong passwords, grant only the privileges each role needs (`GRANT`), plan and test regular backups (logical: `mysqldump`, `pg_dump`; physical: file copy, snapshots), and document recovery procedures.
5. Deploy the application: connect the front-end (forms, reports, web pages) to the database using the chosen driver/connector (JDBC, ODBC, ADO.NET, PDO, mysqli, psycopg2, etc.).

8.3 Building User Interfaces: Forms, Reports and Queries

End users should never type raw SQL. A good database application offers friendly interfaces that separate concerns:

- **Forms**: screen windows with labelled boxes for entering or editing one record at a time (e.g. a "New Student" form with validation rules). In MS Access, forms are built with a wizard and bound directly to a table or query. In web applications, forms are HTML `<form>` elements that send data to a server-side script.
- **Reports**: formatted, printable outputs (e.g. the list of all Form 5 students who paid fees, grouped by class, with totals and page breaks). Access reports support grouping, sorting and automatic sums; web reports are generated as HTML, PDF or Excel.
- **Saved queries / Views**: frequently used `SELECT` statements stored under a name (as a **View** in the database or a saved query in Access), so a secretary simply clicks "Fees not paid" instead of writing SQL. Views also provide a security layer by restricting visible columns/rows.

The same trio — forms for input, queries/views for logic, reports for output — exists in every database application, whether desktop, client-server or web.

8.4 Web Database Integration: The Three-Tier Architecture

When you check your GCE results online, buy airtime via Mobile Money, or consult a library catalogue, a web page is talking to a database behind the scenes. The standard organisation is the **three-tier client-server architecture**.

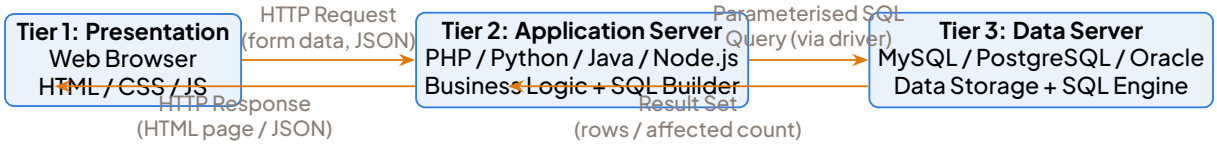
DEFINITION **Three-tier web architecture**

The **three-tier architecture** separates a web database application into three logical layers, often running on different physical machines:

1. Presentation tier (client): the web browser displays pages (HTML/CSS/JavaScript) and collects user input through forms. It knows nothing about SQL or the database

schema.

2. Application tier (application/web server): a program written in a server-side language such as PHP, Python (Django, Flask), Java (Spring, Servlets), C# (ASP.NET) or Node.js receives the HTTP request, applies the business logic (validation, calculations, workflow), builds parameterised SQL queries and sends them to the database server. It formats the results as HTML, JSON or XML for the browser.
3. Data tier (database server): the RDBMS (e.g. MySQL, PostgreSQL, Oracle) stores the data, enforces integrity constraints, executes the SQL queries, manages concurrency and returns result sets to the application server.



Three-tier web database architecture: the browser never talks to the database directly; the application server builds parameterised SQL queries and enforces business rules.

The role of **PHP** (with `mysqli` or `PDO`), **Python** (with `psycopg2` or `SQLAlchemy`), **Java** (with `JDBC`) or **C#** (with `ADO.NET`) is precisely that of the middle tier: they connect to the RDBMS using a **database driver**, send the SQL string, fetch the resulting rows and format them for the presentation tier. A minimal PHP/PDO fragment using a **parameterised query** (prepared statement) looks like this:

- `$pdo = new PDO("mysql:host=localhost;dbname=canteen", "cashier", "c4shPass");`
- `$stmt = $pdo->prepare("SELECT name, balance FROM STUDENT WHERE paid = ?");`
- `$stmt->execute(["no"]);`
- `while ($row = $stmt->fetch()) { echo "<li>$row[name] owes $row[balance] FCFA</li>"; }`

WARNING **Why parameterised queries?**

The `?` placeholder (or `:name` in named style) ensures that the user-supplied value `"no"` is sent to the database separately from the SQL command text. The database engine treats it purely as data, never as executable SQL. This completely prevents SQL injection (see security section). Always use prepared statements for any query that incorporates external input.

8.5 Object-Oriented Database Integration

Modern applications are often written in an **object-oriented** language (Java, Python, C#, PHP, C++), where data and behaviour are encapsulated in **objects** described by **classes**. A relational database, however, stores data in **tables** (relations) with no behaviour. The two worlds must be connected — this is the **object-relational impedance mismatch**.

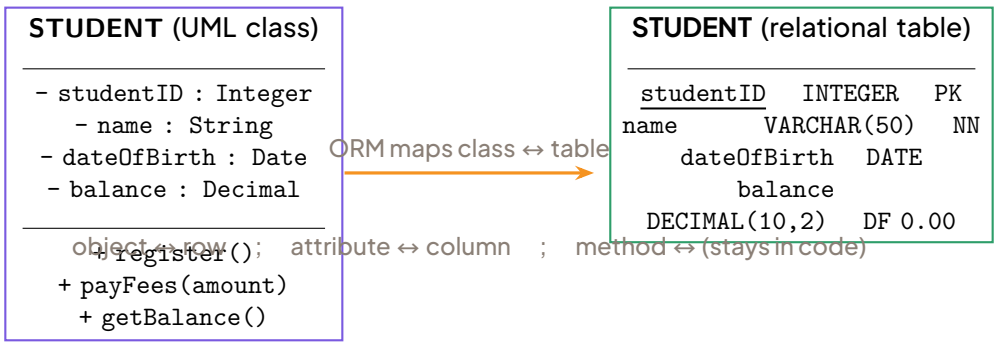
The natural structural correspondence is:

- a **class** corresponds to a **table** (or sometimes a view);
- an **attribute** (field) of the class corresponds to a **column**;
- an **object** (instance) corresponds to a **row** (tuple);
- a **collection** of objects corresponds to a **result set**;
- **inheritance** and **polymorphism** have no direct relational equivalent (handled by ORM strategies: single table, class table, concrete table).

DEFINITION **Object-Relational Mapping (ORM) and persistence**

Object-Relational Mapping (ORM) is a technique (and a family of software libraries/frameworks) that automatically converts objects of an object-oriented program into rows of relational tables and back, so the programmer manipulates objects while the ORM generates the underlying SQL (DDL for schema, DML for data). **Persistence** is the property of an object to survive the end of the program that created it, because its state has been saved in the database and can be reloaded later. An object that can be saved and restored is called a **persistent object**.

Popular ORM frameworks include: **Hibernate / JPA** (Java), **Entity Framework Core** (C#/.NET), **SQLAlchemy / Django ORM** (Python), **Doctrine** (PHP), **Sequelize / TypeORM** (Node.js).



ORM concept: the UML class STUDENT is mapped to the relational table STUDENT; each object becomes one row, each attribute one column. Methods (behaviour) remain in the program code.

WARNING **Objects and tables are not identical**

A class also contains methods (behaviour) and may use inheritance, interfaces and polymorphism, which tables do not have directly. The ORM handles only the data part (attributes → columns, object identity → primary key, associations → foreign keys); behaviour remains in the program code. Do not write methods as columns of a table.

DID YOU KNOW? **ORM in the real world**

When you use a modern web framework (Django, Laravel, Spring Boot, ASP.NET Core), you rarely write raw SQL for basic CRUD operations. You define a model class (e.g. `class Student(Model): ...`) and the ORM generates the `CREATE TABLE`, handles `student.save()`, `Student.objects.filter(paid=False)`, etc. However, for complex reports and performance-critical queries, developers still write raw SQL or use the ORM's low-level query API. Understanding both worlds is essential for the GCE exam and for professional work.

8.6 Basic Database Security

A database containing student records, examination marks or canteen money must be protected. Security operates at multiple levels: network, operating system, RDBMS, application and user behaviour. Here we focus on the RDBMS and application levels.

PROPERTY **Controlling privileges with GRANT and REVOKE**

In SQL, the database administrator (DBA) gives permissions with `GRANT` and withdraws them with `REVOKE`. The syntax varies slightly between RDBMS; the MySQL/MariaDB/PostgreSQL standard form is:

- `GRANT SELECT, INSERT ON CANTEEN.STUDENT TO 'clerk'@'localhost';`
- `GRANT ALL PRIVILEGES ON CANTEEN.* TO 'admin'@'localhost';`
- `REVOKE INSERT ON CANTEEN.STUDENT FROM 'clerk'@'localhost';`

The principle of **least privilege** applies: each user (or application role) receives only the privileges needed for the job — a canteen clerk may `SELECT` and `INSERT` payments, but only the bursar may `DELETE` or `UPDATE` prices. (Knowing the purpose and syntax of `GRANT`/`REVOKE` is examinable.)

PROPERTY **Database backup and recovery**

A complete security policy includes regular backups and a tested recovery plan:

- Logical backup: `mysqldump` (MySQL), `pg_dump` (PostgreSQL) — produces SQL scripts to recreate schema and data. Portable, slower for huge databases.
- Physical backup: copy of data files (cold backup: server stopped; hot backup: with tools like Percona XtraBackup, `pg_basebackup`). Faster, larger files.
- Point-in-time recovery (PITR): combine full backup + transaction logs (binary log in MySQL, WAL in PostgreSQL) to restore to any second.
- Test restores regularly — an untested backup is not a backup.

The second great danger concerns web applications. If a server-side script builds a query by **concatenating user input directly into the SQL string**, an attacker can type SQL code into a form field. This attack is called **SQL injection** (SQLi).

WARNING **Common mistake: concatenating user input into SQL (SQL injection)**

Suppose a login form asks for a username and the program builds the query by string concatenation:

- `query ← "SELECT * FROM USERS WHERE username = '" + input + "'"`

If a malicious user types `' OR '1'='1` (or `admin' --`), the resulting SQL becomes:

- `SELECT * FROM USERS WHERE username = '' OR '1'='1'`

The condition is always true; the attacker logs in as the first user (often admin) without a password — or worse, injects `; DROP TABLE STUDENT; --` to destroy data. Never concatenate raw user input into SQL. The remedy is twofold:

1. Use parameterised queries (prepared statements) everywhere — the input is treated purely as data, never as SQL syntax.
2. Validate and sanitise all input on the server side (whitelist allowed characters, length, type).

This is a classic exam question: recognise the risk, show the attack, state the remedy.

WARNING **Other common web-database vulnerabilities**

- Excessive privileges: the web application connects as `root` or a user with `DROP`, `GRANT` rights. Fix: create a dedicated account with only `SELECT, INSERT, UPDATE, DELETE` on the needed tables.
- Exposed database ports: MySQL port 3306 open to the internet. Fix: firewall rules, bind-address = 127.0.0.1, VPN/SSH tunnel for admin.

- Weak passwords / default accounts: always change defaults, enforce strong password policy.
- Unencrypted connections: use TLS/SSL for database connections (especially over network).

8.7 Worked Exercise and Integration Activity II

WORKED EXAMPLE 8 **Securing the College Canteen database**

Statement. The College Canteen database `CANTEEN` contains the tables `STUDENT(studentID, name, form, balance)`, `MEAL(mealID, description, price)` and `PAYMENT(paymentID, studentID, date, amount)`. (a) Create an account `cook` that may only read the `MEAL` table. (b) Create an account `cashier` that may read `STUDENT` and insert rows into `PAYMENT`. (c) The web page showing a student's balance builds its query by concatenating the student name typed in a form: `query = "SELECT balance FROM STUDENT WHERE name = '" + formInput + "'"`. Identify the risk and the remedy.

SOLUTION

Solution.

- (a) Create the user and grant minimal privilege:
  - `CREATE USER 'cook'@'localhost' IDENTIFIED BY 'm3alPass2025';`
  - `GRANT SELECT ON CANTEEN.MEAL TO 'cook'@'localhost';`
  - `FLUSH PRIVILEGES;` (MySQL)
- (b) Create the user and grant exactly the needed privileges:
  - `CREATE USER 'cashier'@'localhost' IDENTIFIED BY 'c4shPass2025';`
  - `GRANT SELECT ON CANTEEN.STUDENT TO 'cashier'@'localhost';`
  - `GRANT INSERT ON CANTEEN.PAYMENT TO 'cashier'@'localhost';`
  - `FLUSH PRIVILEGES;`
- (c) Risk: SQL injection — a user could type `' OR '1'='1` into the name field, making the condition always true and exposing all students' balances; or type `'; DROP TABLE STUDENT; --` to destroy the table. Remedy: use a parameterised query (prepared statement), e.g. in PHP/PDO:
  - `$stmt = $pdo->prepare("SELECT balance FROM STUDENT WHERE name = ?");`
  - `$stmt->execute([$formInput]);`

The input is sent separately and treated as data only. Also validate that `$formInput` contains only expected characters (letters, spaces, hyphens).

EXPERIMENT **Integration Activity II: the complete College Canteen system**

Work in groups to bring the whole chapter to life. This activity synthesises Lessons 71–88.

1. Choose an RDBMS (MySQL/MariaDB with phpMyAdmin, PostgreSQL with pgAdmin, or MS Access) and justify the choice in writing using at least three criteria from the selection method (cost, data size, concurrency, platform, team skills).
2. Create the `CANTEEN` database with the tables designed earlier (`STUDENT`, `MEAL`, `PAYMENT`), including primary keys, foreign keys (`studentID` in `PAYMENT` references `STUDENT`), `NOT NULL`, `CHECK (balance >= 0)` and `DEFAULT 0.00` constraints. Execute via SQL script.
3. Populate the tables: type a few records manually, then import the class list (50+ students) from a spreadsheet saved as CSV using the RDBMS import tool or `LOAD DATA INFILE`. Verify row counts.
4. Query: write, test and save the following queries (as Views or stored queries):
   - Daily meal count per meal type (join `PAYMENT` + `MEAL` + `GROUP BY`).
   - Students with unpaid balances > 0 (join `STUDENT` + `PAYMENT` + aggregate `SUM`).
   - Total income per week (aggregate `SUM(amount)` with `WEEK(date)`).
5. Secure: create the `cook` and `cashier` accounts with `GRANT` as in the worked exercise. Test that `cook` cannot insert into `PAYMENT` and `cashier` cannot drop tables.
6. Backup: perform a logical backup (`mysqldump canteen >canteen_backup.sql`) and verify the file contains `CREATE TABLE` and `INSERT` statements.
7. Expose one query on the web: write a small PHP (or Python/Flask) page on the school server that runs `SELECT name, balance FROM STUDENT WHERE balance >0` using a parameterised query (even though no user input here, build the habit), and displays the result as an HTML table in the browser — a real three-tier application. Screenshot the browser output.
8. Document the whole process in a short report: ER diagram, relational schema, normalisation check, SQL script, test results, security matrix (user × privilege), backup/restore test, web page screenshot.

KEY POINTS **Key points of the section**

- An RDBMS (MySQL/MariaDB, PostgreSQL, Access, Oracle, SQLite) is chosen according to cost, data size, concurrency, platform and team skills.
- Implementation follows five steps: create (DDL script), populate (INSERT/import), query/validate (SELECT, views), secure (users, GRANT/REVOKE, least privilege, backups), deploy (connect front-end).
- Users interact through forms (input), queries/views (logic) and reports (output).
- Web database applications use the three-tier architecture: browser → application server (PHP/Python/Java/Node.js) → database server. The middle tier builds parameterised SQL and enforces business rules.
- ORM (Hibernate, Entity Framework, SQLAlchemy, Django ORM, Doctrine) maps classes to tables, objects to rows, attributes to columns, giving objects persistence. Methods stay in code.
- GRANT/REVOKE control privileges (least privilege principle); backups (logical/physical) and tested recovery are mandatory.
- Never concatenate user input into SQL — use parameterised queries (prepared statements) to prevent SQL injection. Validate all input server-side.

EXERCISE 8 **Practice**

1. A small hairdressing salon in Bamenda wants a database for appointments on a single computer with no internet. Which RDBMS package would you recommend and why (give two criteria from the selection method)?
2. Draw and label the three-tier architecture for an online school results checking system, indicating what travels on each arrow (HTTP request, SQL query, result set, HTML page).
3. Given the class BOOK(isbn, title, author, price) in a library program, give the corresponding `CREATE TABLE` statement (with appropriate types and primary key) and state what one object of the class becomes in the database.
4. Write the SQL statement(s) that allow user `'secretary'@'localhost'` to read and update the table `STUDENT` in database `CANTEEN`, but not to delete rows or change the schema.
5. A web form builds the query `"DELETE FROM MEAL WHERE id = " + userInput` where `userInput` comes directly from a text field. Explain the exact danger with a concrete malicious input example, and show the corrected PHP/PDO code using a prepared statement.

SOLUTION **Exercise 1**

1. SQLite or MS Access. Justification: (1) Size/Concurrency criterion: the database is small and single-user (no concurrent access needed), so a serverless/embedded engine is sufficient. (2) Cost criterion: both are free (SQLite) or already licensed with Office (Access), avoiding extra expenditure. No server administration required.

2. Tier 1 (Browser): student enters index number in HTML form $\xrightarrow{\text{HTTP Request (GET/POST)}}$ Tier 2 (Application Server, e.g. PHP): validates input, builds parameterised SQL $\xrightarrow{\text{SQL Query (via PDO)}}$ Tier 3 (Database Server, e.g. MySQL): executes query, returns rows $\xrightarrow{\text{Result Set}}$ Tier 2: formats as HTML $\xrightarrow{\text{HTTP Response (HTML page)}}$ Tier 1: displays results.

3. `CREATE TABLE BOOK (isbn CHAR(13) PRIMARY KEY, title VARCHAR(100) NOT NULL, author VARCHAR(80), price DECIMAL(8,2) CHECK (price >= 0));` — one object of class BOOK becomes one row (tuple) of the table `BOOK`.

4. `GRANT SELECT, UPDATE ON CANTEEN.STUDENT TO 'secretary'@'localhost';` (No `DELETE`, `INSERT`, `ALTER`, `DROP`.)

5. Danger: SQL injection. If the user types `1 OR 1=1`, the query becomes `DELETE FROM MEAL WHERE id = 1 OR 1=1`, which deletes every row in `MEAL`. If they type `1; DROP TABLE STUDENT; --`, multiple statements may execute. Fix (PHP/PDO):

   - `$stmt = $pdo->prepare("DELETE FROM MEAL WHERE id = ?");`
   - `$stmt->execute([$userInput]);` // input treated as data only

   Also validate that `$userInput` is a positive integer before executing.

9 INTEGRATION ACTIVITY

9.1 Aims of the activity

This integration activity closes the chapter by making you mobilise, in one single project, all the skills acquired in Lessons 71 to 85. By the end of the activity, you should be able to:

- analyse the requirements of a real information problem and identify the entities, attributes and relationships involved;
- draw a complete **Entity–Relationship diagram** with cardinalities;
- convert the E-R model into a set of **relations**, assign primary and foreign keys, and justify that the design is in **third normal form (3NF)**;
- state the **entity, referential and domain integrity** rules that protect the data;
- implement the database with **SQL DDL** (`CREATE TABLE`) and populate it with `INSERT` statements;
- answer realistic business questions using `SELECT` queries with **joins**, **aggregate functions** and `GROUP BY`;
- work in a team, present and defend a technical solution.

KEY POINTS **Skills assessed**

E-R modelling → relational design → normalisation → integrity → DDL implementation → data insertion → querying with joins and aggregates → communication. Each stage is marked on the assessment grid at the end.

9.2 Situation

The canteen of Government Bilingual High School Buea.

Mrs. Eposi Njie, the bursar of GBHS Buea, manages the school canteen entirely on paper. Every morning the cook writes the day's menu on a chalkboard; students pay cash at the counter and the cashier records the sales in a large notebook. At the end of each term, Mrs. Njie cannot say how much money the canteen really earned, which students still owe money for meals taken on credit, or which supplier delivered the rice used last Tuesday. Pages of the notebook are sometimes lost or illegible, and parents regularly dispute the amounts claimed.

Mrs. Njie asks your Computer Science team to design and implement a small database. After interviewing her, you collect the following facts:

- The school has about 800 **students**. Each student has a unique matriculation number (e.g. `GBHS2025-014`), a name, a class (Form 1 to Upper Sixth) and a telephone contact of a parent.
- The canteen offers **meals** (e.g. Eru and water-fufu, Fried rice and chicken, Beans and puff-puff). Each meal has a code, a description and a fixed unit price in FCFA.

- Each school day, the cook publishes a **daily menu**: a date plus the list of meals available that day. The same meal can appear on many different days.
- When a student buys lunch, a **sale** is recorded: the date, the student, the meal, the quantity, and whether it was paid immediately or taken on credit.
- **Payments** are recorded separately: a student may pay later for one or several credit sales; each payment has a date and an amount in FCFA.
- The canteen buys ingredients from **suppliers** (name, address, telephone). Each supplier supplies one or more ingredients; each ingredient may come from several suppliers, each with its own unit cost.

Sample data provided by Mrs. Njie:

Meal code	Description	Price (FCFA)
M01	Eru and water-fufu	500
M02	Fried rice and chicken	700
M03	Beans and puff-puff	350

Matricule	Name	Class	Parent tel.
GBHS2025-014	Tabot Arrey	Form 5	677 45 21 08
GBHS2025-031	Ngum Blessing	Lower Sixth	690 12 34 56
GBHS2025-102	Ekane Martin	Upper Sixth	653 98 76 54

On 12 January 2026 the menu offered M01 and M02. Tabot bought one plate of M01 paid cash; Blessing took two plates of M02 on credit; Martin bought one plate of M01 cash. On 14 January, Blessing paid 1000 FCFA towards her debt. The supplier Buea Food Depot (tel. 233 32 10 10) supplies rice at 28000 FCFA per 50 kg bag and beans at 45000 FCFA per bag; Tiko Agro Supply supplies rice at 27500 FCFA per bag.

Business questions Mrs. Njie wants answered:

1. What is the total revenue collected on a given day?
2. Which students have unpaid balances, and how much does each owe?
3. Which meal generated the highest total sales this term?
4. How many plates of each meal were sold on each day?
5. Which suppliers provide rice, and at what cost?

9.3 Tasks

Work in teams of three or four. Produce a written dossier plus the working database.

1. **Requirements analysis.** From the situation, list the entities, their attributes, and underline a candidate identifier for each entity. Justify why payments must be stored separately from sales.

2. **E-R modelling.** Draw the complete E-R diagram (entities, attributes, relationships, cardinalities). Pay attention to the many-to-many relationships: menu–meal (a daily menu offers several meals) and supplier–ingredient (with the attribute unit cost).
3. **Relational design.** Convert the E-R diagram into relations. Show how each many-to-many relationship becomes a table with a composite primary key. State all primary and foreign keys.
4. **Normalisation.** For each relation, list its functional dependencies and prove that it is in 3NF (no partial dependency on part of a composite key, no transitive dependency).
5. **Integrity rules.** State the entity, referential and domain constraints (e.g. price > 0, quantity ≥ 1, payment amount > 0, sale status in {'cash','credit'}).
6. **Implementation (DDL).** Write the `CREATE TABLE` statements for all nine tables (`Student`, `Meal`, `DailyMenu`, `MenuMeal`, `Sale`, `Payment`, `Supplier`, `Ingredient`, `Supplies`), with appropriate data types and key constraints.
7. **Population (DML).** Write the `INSERT` statements loading the sample data of the situation.
8. **Business queries (DQL).** Write and test the five queries answering Mrs. Njie's questions, using joins, `SUM`, `COUNT`, `GROUP BY` and `HAVING` where needed.
9. **Presentation.** In ten minutes, present your E-R diagram, demonstrate two queries live, and explain one design decision you are proud of.

WARNING **Common traps**

Do not store the meal price inside the `Sale` table and in `Meal` without justification (redundancy breaks 3NF); do not put a list of meals in one cell of `DailyMenu` (violates 1NF); a payment must reference an existing student (referential integrity); cardinalities must reflect optionality (e.g. a student may have zero sales).

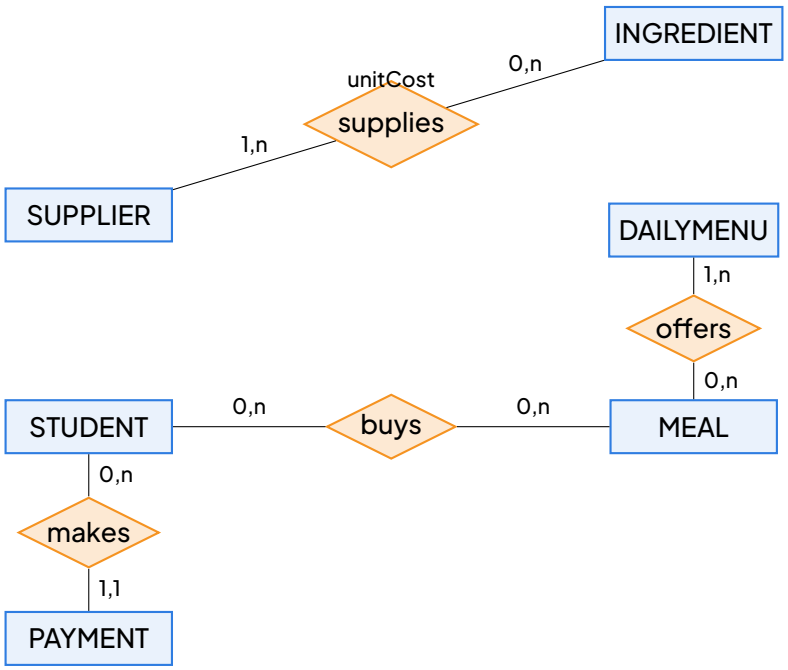
9.4 Model answer

1. Entities and identifiers.

- `Student`(matricule, name, class, parentTel)
- `Meal`(mealCode, description, price)
- `DailyMenu`(menuDate)
- `Sale`(saleId, saleDate, matricule, mealCode, quantity, status)
- `Payment`(paymentId, payDate, amount, matricule)
- `Supplier`(supplierId, name, address, tel)
- `Ingredient`(ingCode, ingName)

Payments are separate because one student makes several payments over time, and a payment may settle several credit sales: merging them into `Sale` would repeat student data and create update anomalies.

2. E-R diagram.



E-R diagram of the canteen database (cardinalities shown as (min,max))

3. Relations and keys.

The two many-to-many relationships become tables: `MenuMeal`(menuDate, mealCode) and `Supplies`(supp
unitCost).
Foreign keys:

- `Sale.matricule` → `Student(matricule)`
- `Sale.mealCode` → `Meal(mealCode)`
- `Payment.matricule` → `Student(matricule)`
- `MenuMeal.menuDate` → `DailyMenu(menuDate)`
- `MenuMeal.mealCode` → `Meal(mealCode)`
- `Supplies.supplierId` → `Supplier(supplierId)`
- `Supplies.ingCode` → `Ingredient(ingCode)`

4. Normalisation to 3NF.

- `Student`: PK `matricule`. FD: `matricule` → `name, class, parentTel`. No transitive dependency ⇒ 3NF.

- `Meal`: PK `mealCode`. FD: `mealCode` → `description, price`. No transitive dependency ⇒ 3NF.
- `DailyMenu`: PK `menuDate`. No non-key attributes ⇒ 3NF.
- `Sale`: PK `saleId`. FD: `saleId` → `saleDate, matricule, mealCode, quantity, status`. Single-attribute key ⇒ no partial dependency; no transitive dependency ⇒ 3NF.
- `Payment`: PK `paymentId`. FD: `paymentId` → `payDate, amount, matricule`. 3NF.
- `MenuMeal`: PK {`menuDate, mealCode`}. Only trivial FD on full key ⇒ no partial dependency ⇒ 3NF.
- `Supplier`: PK `supplierId`. FD: `supplierId` → `name, address, tel`. 3NF.
- `Ingredient`: PK `ingCode`. FD: `ingCode` → `ingName`. 3NF.
- `Supplies`: PK {`supplierId, ingCode`}. FD: {`supplierId, ingCode`} → `unitCost`. The cost depends on both supplier and ingredient, so no partial dependency. No other non-key attributes ⇒ 3NF.

All relations are in 3NF.

5. Integrity rules.

- **Entity integrity:** Every primary key is `NOT NULL` and unique.
- **Referential integrity:** A sale or payment can only reference an existing student; a menu line references an existing meal and menu date; a supply line references an existing supplier and ingredient.
- **Domain integrity:**
  - `Meal.price` > 0
  - `Sale.quantity` ≥ 1
  - `Sale.status` ∈ {'cash','credit'}
  - `Payment.amount` > 0
  - `Supplies.unitCost` > 0

6. DDL implementation (MySQL dialect).

- `CREATE TABLE Student (matricule VARCHAR(12) PRIMARY KEY, name VARCHAR(60) NOT NULL, class VARCHAR(20), parentTel VARCHAR(15));`
- `CREATE TABLE Meal (mealCode CHAR(3) PRIMARY KEY, description VARCHAR(60), price INT CHECK (price > 0));`
- `CREATE TABLE DailyMenu (menuDate DATE PRIMARY KEY);`
- `CREATE TABLE MenuMeal (menuDate DATE, mealCode CHAR(3), PRIMARY KEY (menuDate, mealCode), FOREIGN KEY (menuDate) REFERENCES DailyMenu(menuDate), FOREIGN KEY (mealCode) REFERENCES Meal(mealCode));`
- `CREATE TABLE Sale (saleId INT AUTO_INCREMENT PRIMARY KEY, saleDate DATE, matricule VARCHAR(12), mealCode CHAR(3), quantity INT CHECK (quantity >= 1), status ENUM('cash','cred`

```
FOREIGN KEY (matricule) REFERENCES Student(matricule), FOREIGN KEY (mealCode) REFERENCES
Meal(mealCode));
```

- `CREATE TABLE Payment (paymentId INT AUTO_INCREMENT PRIMARY KEY, payDate DATE, amount INT CHECK (amount > 0), matricule VARCHAR(12), FOREIGN KEY (matricule) REFERENCES Student(matricule));`
- `CREATE TABLE Supplier (supplierId INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(60) NOT NULL, address VARCHAR(100), tel VARCHAR(15));`
- `CREATE TABLE Ingredient (ingCode CHAR(3) PRIMARY KEY, ingName VARCHAR(40) NOT NULL);`
- `CREATE TABLE Supplies (supplierId INT, ingCode CHAR(3), unitCost INT CHECK (unitCost > 0), PRIMARY KEY (supplierId, ingCode), FOREIGN KEY (supplierId) REFERENCES Supplier(suppl FOREIGN KEY (ingCode) REFERENCES Ingredient(ingCode));`

7. Sample inserts.

- `INSERT INTO Meal VALUES ('M01','Eru and water-fufu',500), ('M02','Fried rice and chicken',7 ('M03','Beans and puff-puff',350);`
- `INSERT INTO Student VALUES ('GBHS2025-014','Tabot Arrey','Form 5','677452108'), ('GBHS2025- Blessing','Lower Sixth','690123456'), ('GBHS2025-102','Ekane Martin','Upper Sixth','6539876`
- `INSERT INTO DailyMenu VALUES ('2026-01-12');`
- `INSERT INTO MenuMeal VALUES ('2026-01-12','M01'), ('2026-01-12','M02');`
- `INSERT INTO Sale (saleDate,matricule,mealCode,quantity,status) VALUES ('2026-01-12','GBHS20 ('2026-01-12','GBHS2025-031','M02',2,'credit'), ('2026-01-12','GBHS2025-102','M01',1,'cash'`
- `INSERT INTO Payment (payDate,amount,matricule) VALUES ('2026-01-14',1000,'GBHS2025-031');`
- `INSERT INTO Supplier (name,address,tel) VALUES ('Buea Food Depot','Buea','233321010'), ('Tiko Agro Supply','Tiko','233445566');`
- `INSERT INTO Ingredient VALUES ('RIC','Rice'), ('BEA','Beans');`
- `INSERT INTO Supplies VALUES (1,'RIC',28000), (1,'BEA',45000), (2,'RIC',27500);`

8. The five business queries (standard SQL with MySQL-compatible syntax).

- **Q1 — Daily revenue:** `SELECT s.saleDate, SUM(m.price * s.quantity) AS revenue FROM Sale s JOIN Meal m ON s.mealCode = m.mealCode GROUP BY s.saleDate;`
- **Q2 — Unpaid balances:** `SELECT st.name, SUM(m.price * s.quantity) - COALESCE((SELECT SUM(p.amount) FROM Payment p WHERE p.matricule = st.matricule),0) AS balance FROM Sale s JOIN Meal m ON s.mealCode = m.mealCode JOIN Student st ON s.matricule = st.matricule WHERE s.status = 'credit' GROUP BY st.matricule, st.name HAVING balance > 0;`
- **Q3 — Best-selling meal (by total revenue):** `SELECT m.description, SUM(m.price * s.quantity) AS total FROM Sale s JOIN Meal m ON s.mealCode = m.mealCode GROUP BY m.mealCode, m.descript ORDER BY total DESC LIMIT 1;`

- **Q4 — Plates per meal per day:** `SELECT s.saleDate, m.description, SUM(s.quantity) AS plates FROM Sale s JOIN Meal m ON s.mealCode = m.mealCode GROUP BY s.saleDate, m.descriptio ORDER BY s.saleDate, m.description;`
- **Q5 — Rice suppliers:** `SELECT sp.name, su.unitCost FROM Supplies su JOIN Supplier sp ON su.supplierId = sp.supplierId JOIN Ingredient i ON su.ingCode = i.ingCode WHERE i.ingName = 'Rice';`

EXEMPLE **Assessment grid (20 marks)**

Requirements and entities (2) — E-R diagram with cardinalities (4) — Relational schema and keys (3) — Normalisation proof (3) — Integrity rules (2) — DDL scripts (2) — Inserts (1) — Five queries (2) — Presentation (1).

KEY POINTS **What this activity proves**

A database is never just tables: it is the final product of a disciplined chain — analyse, model, design, normalise, constrain, implement, query. Skipping any stage produces exactly the disorder Mrs. Njie suffered with her paper notebook.

10 METHODS AND WORKED EXAMPLES

11 MANAGING AND RETRIEVING INFORMATION

11.1 Database Fundamentals and Models

DEFINITION **Data, Information and Database**

Data are raw, unprocessed facts (e.g. `23`, `'Ngwa'`, `14.5`). Information is data that has been processed, organised and presented in a meaningful context (e.g. “Ngwa, aged 23, scored 14.5 in Computer Science”). A Database is a shared, integrated collection of logically related data, organised to serve multiple applications. A DBMS (Database Management System) is the software that enables users to define, create, maintain and control access to the database.

PROPERTY **Advantages of the Database Approach**

- Controlled redundancy: data stored once, reducing inconsistency.
- Data independence: logical/physical changes do not break applications.
- Concurrent access: multiple users, controlled by locking/protocols.
- Integrity & Security: constraints, views, roles, encryption.
- Backup & Recovery: transaction logs, point-in-time restore.

DEFINITION **Data Models**

A data model is a set of concepts for describing the structure of a database.

- Hierarchical Model: tree structure, one parent per child (e.g. IMS). Rigid, no M:N.
- Network Model: graph structure (CODASYL), allows M:N via sets. Complex navigation.
- Relational Model: data as relations (tables), based on set theory and predicate logic. Dominant today (SQL).
- Object-Oriented Model: objects with identity, encapsulation, inheritance. For complex data (CAD, multimedia).

KEY POINTS **Three-Level ANSI/SPARC Architecture**

- External Level: User views (views, reports). Multiple external schemas.
- Conceptual Level: Community view (ERD, relational schema). One conceptual schema. Logical data independence: change conceptual without affecting external.
- Internal Level: Physical storage (indexes, files, compression). One internal schema. Physical data independence: change storage without affecting conceptual.

Mappings between levels are maintained by the DBMS.

11.2 Skill 1: Identifying Entities, Attributes and Keys

METHOD **Building the Data Dictionary of a Situation**

1. Read the scenario and underline every **noun** that the organisation needs to store information about: these are candidate entities (e.g. STUDENT, BOOK, LOAN).
2. For each entity, list its attributes (properties), choosing atomic, single-valued attributes only.
3. Choose a primary key for each entity: an attribute (or minimal set of attributes) that is unique and never null. If none exists naturally, introduce a surrogate key (e.g. `StudentID`).
4. Mark foreign keys: attributes that reference the primary key of another entity.
5. Present the result as a data dictionary table: attribute name, type, size, key status, description.

Reflexes: a good primary key is short, stable, unique, non-null. Names and phone numbers are poor keys (duplicates, changes).

WORKED EXAMPLE 9 **Data Dictionary for a School Library**

A school library in Bamenda lends books to students. A student has an admission number, name, class and date of birth. A book has an ISBN, title, author and year of publication. A loan records which student borrowed which book, the date borrowed and the return date. Build the data dictionary, indicating primary and foreign keys.

SOLUTION

Step 1 — Entities: STUDENT, BOOK, LOAN.
Step 2 — Attributes and keys:

Entity	Attribute	Type	Key / description
STUDENT	AdmNo	Text(8)	Primary key
	StuName	Text(40)	Full name
	Class	Text(10)	e.g. Upper Sixth Sc
	DoB	Date	Date of birth
BOOK	ISBN	Text(13)	Primary key
	Title	Text(60)	Book title
	Author	Text(40)	Main author
	PubYear	Number	Year of publication
LOAN	LoanID	Number	Primary key (surrogate)
	AdmNo	Text(8)	Foreign key → STUDENT
	ISBN	Text(13)	Foreign key → BOOK
	DateOut	Date	Date borrowed
	DateRet	Date	Return date (null while out)

Justification: `StuName` is rejected as key because two students may share a name; `LoanID` is introduced because the pair (AdmNo, ISBN) is not unique — the same student may borrow the same book twice in a year.

11.3 Relational Model and Database Architecture

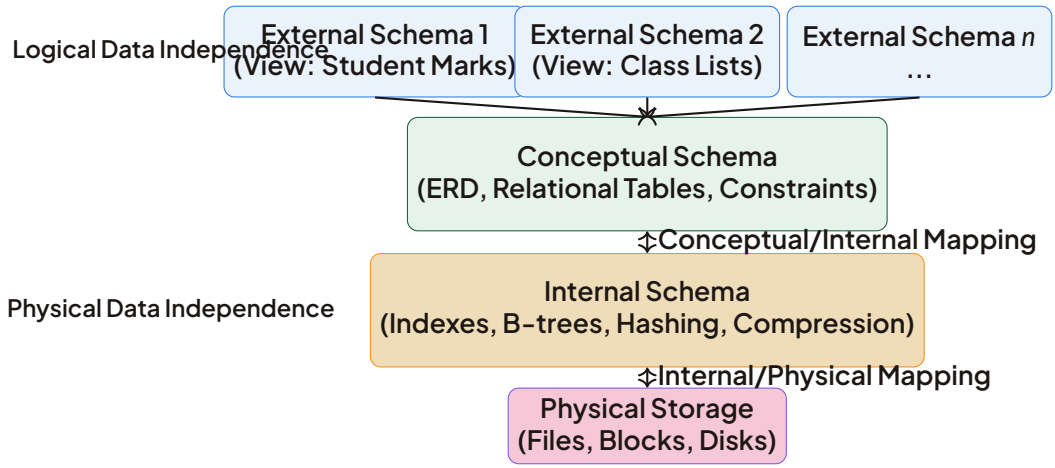
DEFINITION **Relational Model Terminology (Codd)**

- Relation: a table with rows and columns; no duplicate rows, column order insignificant, row order insignificant.
- Tuple: a row (record).
- Attribute: a column (field), drawn from a domain (set of atomic values).
- Degree: number of attributes. Cardinality: number of tuples.
- Superkey: set of attributes uniquely identifying a tuple. Candidate Key: minimal superkey. Primary Key: chosen candidate key. Alternate Key: other candidate keys.
- Foreign Key: attribute(s) in one relation referencing the primary key of another (or same) relation.

PROPERTY **Relational Integrity Rules**

1. Entity Integrity: No component of a primary key may be `NULL`.
2. Referential Integrity: For every foreign key value, there must exist a matching primary key value in the referenced relation (or the foreign key is wholly `NULL` if allowed).

KEY POINTS **Database Architecture — Three-Level Schema**



11.4 Skill 3: Translating an ERD into Relational Tables

METHOD **Mapping Rules ERD → Relations**

1. Entity → one table; its attributes become columns; the underlined key becomes the primary key.
2. 1:N relationship → copy the primary key of the "1" side into the table of the "N" side as a foreign key.
3. 1:1 relationship → place the foreign key on the side with mandatory participation (or merge the tables if both are mandatory).
4. M:N relationship → create a new table whose primary key is the combination of the two entity keys (plus any extra discriminator, e.g. date), and which carries the relationship's own attributes.
5. Write the final schema as `TABLE(PK, attr, FK*)` and state each foreign key's target.

WORKED EXAMPLE 10 **From ERD to Relations — School Canteen**

A school canteen in Yaoundé: a STUDENT (StuID, name) buys MEALs (MealID, description, price). A student can buy many meals and a meal can be bought by many students; each purchase has a date and a quantity. Each meal is supplied by exactly one SUPPLIER (SupID, supName, town); a supplier supplies many meals. Derive the relational schema.

SOLUTION

Step 1 — Entities:

- `STUDENT(StuID, name)`
- `SUPPLIER(SupID, supName, town)`

Step 2 — 1:N (SUPPLIER–MEAL): the key of the "1" side (SupID) migrates into MEAL:

- `MEAL(MealID, description, price, SupID*)` with SupID* referencing SUPPLIER.

Step 3 — M:N (STUDENT–MEAL) with attributes: new table:

- `PURCHASE(StuID*, MealID*, pDate, quantity)` where StuID* references STUDENT and MealID* references MEAL. The date is part of the key because the same student may buy the same meal on different days.

Check: 4 tables, every relationship represented, no attribute left out. This schema is the expected GCE answer format.

11.5 Entity–Relationship Modelling

DEFINITION **ER Model Core Concepts**

- Entity: a distinguishable object (strong: independent key; weak: depends on owner via identifying relationship, drawn with double rectangle).
- Attribute: Simple (atomic) vs Composite (e.g. Address → Street, City); Single-valued vs Multi-valued (drawn with double oval, e.g. PhoneNumbers); Stored vs Derived (drawn with dashed oval, e.g. Age from DoB).
- Relationship: association between entities. Degree: Unary (recursive), Binary, Ternary. Cardinality: 1:1, 1:N, M:N. Participation: Total (double line, mandatory) vs Partial (single line, optional).
- Notations: Chen (rectangles, diamonds, ovals) and Crow's Foot (boxes, lines with crow's foot for "many").

11.6 Skill 2: Drawing an Entity–Relationship Diagram

METHOD **From Scenario to ERD (Chen / Crow's Foot)**

1. Draw one rectangle per entity, named in capitals.
2. Identify relationships (verbs linking two entities) and draw them as diamonds (Chen) or labelled lines (Crow's Foot).
3. Determine the cardinality of each relationship by asking two questions: "How many *B* can one *A* be linked to?" and "How many *A* can one *B* be linked to?" — giving 1:1, 1:N or M:N.
4. Determine participation: is every entity instance required to participate? (Total = double line).
5. Attach attributes as ovals (Chen); underline the primary key; double oval for multi-valued; dashed oval for derived.
6. Check: every M:N relationship will later become a separate table in the relational design.

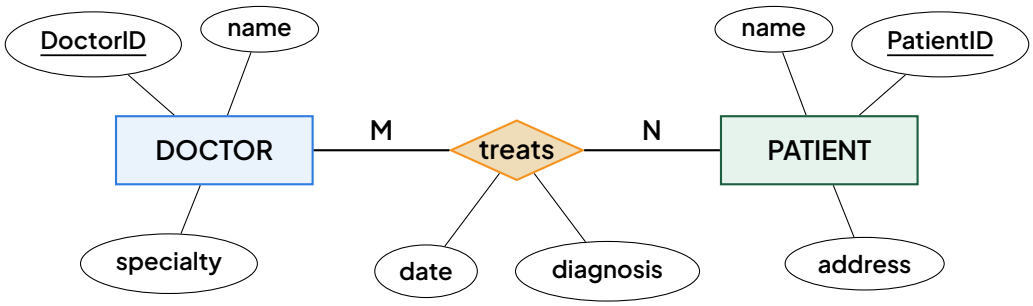
Reflex: always justify each cardinality with a sentence taken from the scenario, never guess.

WORKED EXAMPLE 11 **ERD for a Hospital**

In a clinic in Douala: a doctor (DoctorID, name, specialty) treats many patients; a patient (PatientID, name, address) may be treated by many doctors. Each treatment (date, diagnosis) concerns exactly one doctor and one patient. Draw the ERD and state the cardinalities.

SOLUTION

Cardinalities: "A doctor treats many patients" and "a patient may be treated by many doctors" ⇒ M:N relationship treats between DOCTOR and PATIENT. The attributes date and diagnosis belong to the relationship (they describe one particular treatment, not a doctor or a patient).



ERD (Chen notation) of the DOCTOR–PATIENT M:N relationship with relationship attributes.

Reading: one occurrence of treats links exactly one doctor and exactly one patient; the M:N relationship will become a table TREATMENT(DoctorID, PatientID, date, diagnosis) in the relational design.

WARNING **Common ERD Mistakes**

- Placing relationship attributes (date, quantity) on one of the entities instead of the relationship diamond.
- Forgetting to underline the primary key in attribute ovals.
- Guessing cardinalities ("a student takes many courses" ⇏ M:N; check if a course is taken by many students).
- Using multi-valued attributes (e.g. `PhoneNumbers`) without a separate entity/relationship if the model requires 1NF later.

11.7 Relational Design, Normalisation and Design Stages

DEFINITION **Functional Dependency (FD)**

Let R be a relation schema, $X, Y \subseteq R$. $X \to Y$ (X functionally determines Y) iff for any valid instance $r(R)$, whenever two tuples have the same value for X, they must have the same value for Y.

- Full FD: $X \to Y$ and no proper subset of X determines Y.
- Partial FD: $X \to Y$ but a proper subset of X determines Y (only possible with composite key).

- Transitive FD: $X \to Y$ and $Y \to Z$ with $Y \not\to X$ and $Z \not\subseteq X \cup Y$ (non-key determines non-key).

PROPERTY **Normal Forms — Definitions**

- 1NF: All attributes atomic (no repeating groups, no multi-valued attributes in a single cell).
- 2NF: 1NF + no partial dependency of any non-key attribute on any candidate key.
- 3NF: 2NF + no transitive dependency of any non-key attribute on any candidate key. (Equivalently: for every non-trivial FD $X \to A$, X is a superkey or A is prime).
- BCNF: 3NF + for every non-trivial FD $X \to A$, X is a superkey. (Stricter: handles anomalies where a non-key attribute determines part of a key).

11.8 Skill 4: Normalising a Table (1NF, 2NF, 3NF)

METHOD **Normalisation Procedure**

1. 1NF: eliminate repeating groups and multi-valued attributes — every cell must hold one atomic value; introduce a key if needed.
2. 2NF: (only relevant with a composite key) remove partial dependencies: any non-key attribute depending on only part of the key is moved, with that part, into a new table.
3. 3NF: remove transitive dependencies: if a non-key attribute A determines another non-key attribute B ($A \to B$), move B (with A as key) into a new table.
4. At each step, state the functional dependencies you used, and verify that no data is lost (the decomposition must be lossless: the tables can be re-joined on the shared key).

Reflex: write the dependencies first (e.g. `StuID` → `StuName`); they drive everything.

WORKED EXAMPLE 12 **Normalising an Exam Results Table**

The West Regional Delegation keeps results in one table: `RESULT(CandNo, SubjectCode, CandName, CentreNo, CentreName, SubjectTitle, Mark)`. Dependencies: CandNo → CandName, CentreNo; CentreNo → CentreName; SubjectCode → SubjectTitle; (CandNo, SubjectCode) → Mark. Normalise to 3NF, showing each step.

SOLUTION

1NF: all attributes are atomic; key = (CandNo, SubjectCode). Already 1NF.
2NF — remove partial dependencies: `CandName`, `CentreNo` depend only on `CandNo`; `SubjectTitle` depends only on `SubjectCode`. Decompose:

- `CANDIDATE(CandNo, CandName, CentreNo)`
- `SUBJECT(SubjectCode, SubjectTitle)`
- `MARK(CandNo*, SubjectCode*, Mark)`

3NF — remove transitive dependency: in CANDIDATE, CandNo → CentreNo → CentreName, so `CentreName` is transitively dependent on the key. Decompose further:

- `CENTRE(CentreNo, CentreName)`
- `CANDIDATE(CandNo, CandName, CentreNo*)`
- `SUBJECT(SubjectCode, SubjectTitle)`
- `MARK(CandNo*, SubjectCode*, Mark)`

Benefits: a centre's name is stored once (no update anomaly), a subject can exist before any candidate takes it (no insertion anomaly), and deleting the last candidate of a centre no longer loses the centre (no deletion anomaly).

KEY POINTS **Design Stages**

1. Requirements Analysis: interviews, documents → user requirements.
2. Conceptual Design: ERD (DBMS-independent), validation with users.
3. Logical Design: Map ERD to relational schema (target DBMS), normalise to 3NF/BCNF, define integrity constraints.
4. Physical Design: Indexes, clustering, partitioning, file organisation, storage estimation, security roles.
5. Implementation: DDL scripts, data loading, testing, tuning.

11.9 Database Integrity and Integration Activity I

DEFINITION **Integrity Constraint Types**

- Domain Integrity: Data type, `CHECK`, `DEFAULT`, `NOT NULL`, `UNIQUE`.

- Entity Integrity: Primary Key ≠ NULL, Unique.
- Referential Integrity: Foreign Key references existing PK; actions: `ON DELETE CASCADE | RESTRICT | SET NULL | SET DEFAULT, ON UPDATE ...`.
- User-defined (Business) Integrity: Complex rules via `TRIGGER` or `ASSERTION` (e.g. "total loan ≤ credit limit").

PROPERTY **Transaction Properties — ACID**

- Atomicity: All operations succeed or none (rollback).
- Consistency: Database moves from one valid state to another (constraints satisfied).
- Isolation: Concurrent transactions do not interfere (serialisability via locks/MVCC).
- Durability: Committed changes survive crashes (write-ahead log).

WARNING **Concurrency Anomalies**

- Lost Update: T1 reads, T2 reads, T1 writes, T2 writes → T1 lost.
- Dirty Read: T1 reads uncommitted write of T2; T2 rolls back.
- Unrepeatable Read: T1 reads, T2 updates+commits, T1 reads again → different value.
- Phantom Read: T1 reads set, T2 inserts+commits matching row, T1 reads set again → extra row.

Solution: Locking (Shared/Exclusive), Timestamp Ordering, Multi-Version Concurrency Control (MVCC).

EXPERIMENT **Integration Activity I — Design a Clinic Database**

Scenario: A clinic manages doctors, patients, appointments, prescriptions and medicines. A doctor has many appointments; a patient has many appointments. An appointment has date, time, diagnosis. A prescription (linked to appointment) lists medicines with dosage. Medicines have name, unit, stock level. Tasks:

1. List entities, attributes, keys (Data Dictionary).
2. Draw ERD (Chen) with cardinalities and participation.
3. Map to relational schema (show FKs).

4. Normalise any unnormalised tables to 3NF.
5. Write DDL for the final schema with all constraints.
6. Write SQL to list patients with appointments on a given date, showing doctor name and diagnosis.

11.10 SQL: Data Definition and Data Manipulation

DEFINITION **SQL Data Types (Common)**

- Character: `CHAR(n)` fixed, `VARCHAR(n)` variable, `TEXT`.
- Numeric: `INTEGER`, `SMALLINT`, `BIGINT`, `DECIMAL(p,s)` / `NUMERIC(p,s)` exact, `FLOAT`, `REAL` approximate.
- Temporal: `DATE`, `TIME`, `TIMESTAMP`, `INTERVAL`.
- Binary: `BLOB`, `BYTEA`.
- Boolean: `BOOLEAN` (TRUE/FALSE/UNKNOWN).

11.11 Skill 5: Writing DDL and DML Statements

METHOD **CREATE, INSERT, UPDATE, DELETE**

1. CREATE TABLE: name each column with its type; add `PRIMARY KEY`, `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT` and `REFERENCES` clauses to enforce integrity.
2. INSERT INTO table `(cols) VALUES (...)`: match values to columns in order and type; text and dates in single quotes.
3. UPDATE table `SET col = value WHERE condition`: never omit `WHERE` unless you really mean to change every row.
4. DELETE FROM table `WHERE condition`: same warning — without `WHERE` the whole table is emptied.
5. ALTER TABLE: `ADD COLUMN`, `DROP COLUMN`, `ALTER COLUMN TYPE`, `ADD CONSTRAINT`, `DROP CONSTRAINT`.
6. DROP TABLE [CASCADE | RESTRICT]; TRUNCATE TABLE (fast, non-transactional in some DBMS, resets identity).

Reflex: define primary keys and foreign keys at creation time; it is the DBMS, not the user, that should police integrity.

WORKED EXAMPLE 13 **DDL + DML for a Bookshop**

A bookshop in Buea needs the table BOOK(ISBN, title, author, price, stock) where ISBN is the primary key, price must be positive and stock defaults to 0. (a) Write the CREATE TABLE statement. (b) Insert "Things Fall Apart", Achebe, 2500 FCFA, 12 copies, ISBN '9780435905262'. (c) Increase all prices by 10%. (d) Delete books with zero stock.

SOLUTION

(a) Definition:

```
CREATE TABLE BOOK (
  ISBN CHAR(13) PRIMARY KEY,
  title VARCHAR(60) NOT NULL,
  author VARCHAR(40),
  price DECIMAL(8,2) CHECK (price > 0),
  stock INTEGER DEFAULT 0 );
```

(b) Insertion:

```
INSERT INTO BOOK (ISBN, title, author, price, stock)
VALUES ('9780435905262', 'Things Fall Apart', 'Achebe', 2500, 12);
```

(c) Update:

```
UPDATE BOOK SET price = price * 1.10;
```

(Here omitting `WHERE` is intentional: all prices rise.) (d) Deletion:

```
DELETE FROM BOOK WHERE stock = 0;
```

Checks: the `CHECK` clause rejects a negative price at insertion; `DEFAULT 0` means stock may be omitted in `INSERT`.

EXEMPLE **Referential Actions Demo**

```
CREATE TABLE SESSION (SessID INT PRIMARY KEY, CliID INT NOT NULL REFERENCES
CLIENT(CliID) ON DELETE RESTRICT ON UPDATE CASCADE, ...);
```

- `ON DELETE RESTRICT`: prevents deleting a client who has sessions (error raised).
- `ON UPDATE CASCADE`: if a client's `CliID` changes (rare), all session rows update automat-

ically.

- `ON DELETE SET NULL`: allowed only if FK column is nullable; sets FK to NULL when parent deleted.

11.12 SQL: Queries, Joins and Aggregate Functions

11.13 Skill 6: Querying with SELECT, WHERE, ORDER BY and Joins

METHOD **Building a SELECT Query**

1. Write `SELECT` with the required columns (or `*` for all).
2. Write `FROM` with the table(s). For several tables, add the join condition (`ON` / `WHERE` equality of foreign key and primary key) — forgetting it produces a Cartesian product.
3. Add `WHERE` for row filtering (`=`, `<`, `>`, `BETWEEN`, `LIKE '%text%'`, `IN`, `IS NULL`).
4. Add `ORDER BY col ASC|DESC` to sort; `DISTINCT` to remove duplicates.
5. Logical operators: `AND`, `OR`, `NOT` — use brackets to control priority.

Reflex: one join condition per pair of tables; n tables need at least $n - 1$ join conditions.

WORKED EXAMPLE 14 **Queries on a School Database**

Given `STUDENT(StuID, name, class, town)` and `MARK(StuID*, subject, score)`, write SQL for: (a) names of students from Bamenda, alphabetically; (b) students whose name contains "Ngong"; (c) each student's name with their Computer Science score; (d) students who have no mark recorded.

SOLUTION

(a) Filter + sort:

```
SELECT name FROM STUDENT
WHERE town = 'Bamenda' ORDER BY name ASC;
```

(b) Pattern matching:

```
SELECT * FROM STUDENT WHERE name LIKE '%Ngong%';
```

(c) Join (equi-join):

```
SELECT S.name, M.score
```

```
FROM STUDENT S JOIN MARK M ON S.StuID = M.StuID

WHERE M.subject = 'Computer Science';
```

The join condition `S.StuID = M.StuID` links each mark to the right student; without it every student would be paired with every mark (Cartesian product — a classic GCE trap). (d) Outer join + NULL test:

```
SELECT S.name FROM STUDENT S

LEFT JOIN MARK M ON S.StuID = M.StuID

WHERE M.StuID IS NULL;
```

The `LEFT JOIN` keeps students with no mark; their `M.StuID` is NULL, which the `WHERE` selects.

PROPERTY **Join Types Summary**

Join Type	Result
`INNER JOIN`	Only matching rows from both tables.
`LEFT [OUTER] JOIN`	All rows from left table + matches from right; NULLs for non-matches.
`RIGHT [OUTER] JOIN`	All rows from right table + matches from left; NULLs for non-matches.
`FULL [OUTER] JOIN`	All rows from both; NULLs where no match.
`CROSS JOIN`	Cartesian product (no ON clause).
`NATURAL JOIN`	Equi-join on all columns with same name (dangerous if schema changes).
`SELF JOIN`	Table joined to itself (use aliases).

11.14 Skill 7: Aggregate Functions and GROUP BY

METHOD **Aggregates: COUNT, SUM, AVG, MAX, MIN with GROUP BY / HAVING**

1. Choose the aggregate: `COUNT(*)` rows, `COUNT(col)` non-null values, `SUM/AVG/MAX/MIN` on numeric columns.
2. If results are needed per category, add `GROUP BY category`; every non-aggregated column in `SELECT` must appear in `GROUP BY`.
3. Filter rows before grouping with `WHERE`; filter groups after aggregation with `HAVING`

(e.g. `HAVING AVG(score) >= 10`).

4. Execution order to remember: `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`.

Trap: `WHERE AVG(score) > 10` is illegal — aggregates belong in `HAVING`.

WORKED EXAMPLE 15 **Statistics for a GCE Mock Exam**

Table `MARK(StuID, subject, score, class)`. Write SQL for: (a) the number of candidates; (b) the average, highest and lowest score per subject, best subject first; (c) subjects where the average is at least 10/20; (d) classes with more than 30 candidates.

SOLUTION

(a) Simple count:

```
SELECT COUNT(DISTINCT StuID) AS Candidates FROM MARK;
```

(`DISTINCT` avoids counting a candidate once per subject.) (b) Grouped aggregates:

```
SELECT subject, AVG(score) AS Mean, MAX(score) AS Best, MIN(score) AS Worst
FROM MARK GROUP BY subject ORDER BY Mean DESC;
```

(c) Filtering groups with HAVING:

```
SELECT subject, AVG(score) AS Mean FROM MARK
GROUP BY subject HAVING AVG(score) >= 10;
```

(d) HAVING with COUNT:

```
SELECT class, COUNT(DISTINCT StuID) AS Nb FROM MARK
GROUP BY class HAVING COUNT(DISTINCT StuID) > 30;
```

Reading of the engine's work: rows are first grouped by subject (or class); the aggregate is computed inside each group; `HAVING` then keeps only the groups satisfying the condition. Using `WHERE` instead would fail because the average does not yet exist at row-filtering time.

KEY POINTS **Subqueries**

- Scalar subquery: returns one value (e.g. `WHERE score > (SELECT AVG(score) FROM MARK)`).
- Row subquery: returns one row (e.g. `WHERE (StuID, subject) IN (SELECT StuID, subject FROM MARK WHERE score=20)`).

- Table subquery: returns multiple rows (used with `IN`, `EXISTS`, `ANY`, `ALL`).
- Correlated subquery: references a column from the outer query (executed once per outer row).

11.15 RDBMS Implementation, Web and Object-Oriented Integration

11.16 Skill 8: Enforcing Integrity and Choosing an RDBMS Implementation

METHOD **Integrity Constraints and Implementation Checklist**

1. Entity integrity: every table has a primary key, `NOT NULL` and unique.
2. Referential integrity: every foreign key declared with `REFERENCES`; decide `ON DELETE CASCADE` / `RESTRICT` behaviour.
3. Domain integrity: correct data types, `CHECK`, `DEFAULT`, `UNIQUE`.
4. User (business) integrity: validation rules from the scenario (e.g. "mark between 0 and 20").
5. For implementation: pick an RDBMS suited to the scale (SQLite/Access for a school, MySQL/PostgreSQL for a web application), create the database, run the DDL scripts, populate with DML, then test with queries.

WORKED EXAMPLE 16 **Full Mini-Implementation of a Cybercafé Billing Database**

A cybercafé in Limbe bills sessions: `CLIENT(CliID, name, phone)` and `SESSION(SessID, CliID*, sDate, minutes, amount)`. (a) Write the CREATE TABLE statements enforcing all four integrity types, including "minutes > 0" and "amount ≥ 0". (b) Explain what happens if we try to delete a client who still has sessions, with `ON DELETE RESTRICT`. (c) Write the query giving each client's total amount spent.

SOLUTION

(a) DDL with constraints:

```
CREATE TABLE CLIENT (
  CliID INTEGER PRIMARY KEY,
  name VARCHAR(40) NOT NULL,
  phone VARCHAR(15) UNIQUE );
```

```
CREATE TABLE SESSION (
  SessID INTEGER PRIMARY KEY,
  CliID INTEGER NOT NULL REFERENCES CLIENT(CliID)
    ON DELETE RESTRICT,
  sDate DATE NOT NULL,
  minutes INTEGER CHECK (minutes > 0),
  amount DECIMAL(8,2) CHECK (amount >= 0) );
```

Entity integrity: both primary keys. Referential integrity: `REFERENCES CLIENT(CliID)`. Domain integrity: types, `NOT NULL`, `CHECK`, `UNIQUE`. (b) Deletion attempt: with `ON DELETE RESTRICT`, the DBMS refuses to delete a client referenced by existing sessions and raises an error; the sessions must be deleted (or archived) first. This prevents orphan rows — sessions pointing to a non-existent client. (c) Total per client (join + aggregate):

```
SELECT C.name, SUM(S.amount) AS Total
FROM CLIENT C JOIN SESSION S ON C.CliID = S.CliID
GROUP BY C.name ORDER BY Total DESC;
```

This single query combines all the chapter's reflexes: correct join condition, grouping by the client, aggregate per group, and sorted output for the manager.

DID YOU KNOW? **RDBMS Packages in Cameroon Context**

- Microsoft Access / SQLite: file-based, zero-config, good for school projects, desktop apps, low concurrency.
- MySQL / MariaDB: most popular for web (LAMP stack), free, good replication, used by many Cameroonian SMEs.
- PostgreSQL: advanced features (JSONB, GIS, ACID, extensibility), preferred for complex data, strict standards compliance.
- Oracle / SQL Server: enterprise, high cost, advanced security, partitioning, RAC/AlwaysOn for high availability. Used in banks, telecoms (MTN, Orange), government.

Selection criteria: data volume, concurrency, budget, OS, required features (GIS, JSON, replication), vendor support.

DEFINITION **Web Integration — Connectivity**

- ODBC (Open Database Connectivity): C API, driver manager, DSN. Universal but older.
- JDBC (Java Database Connectivity): Type 4 (thin) drivers pure Java, standard for Java EE / Spring.
- ADO.NET: .NET providers (SqlClient, Npgsql, MySqlConnector), disconnected `DataSet`.
- PHP PDO: `new PDO('mysql:host=...;dbname=...', user,pass)` — supports prepared statements (prevents SQL injection).
- ORM (Object-Relational Mapping): Hibernate (Java), Entity Framework (C#), SQLAlchemy (Python), Doctrine (PHP). Maps objects to tables, generates SQL, handles lazy loading, caching.

WARNING **SQL Injection Prevention**

Never concatenate user input into SQL strings.

- Bad: `"SELECT * FROM USER WHERE login = '" + input + "'"` → input `' OR '1'='1` bypasses login.
- Good (Prepared Statement): `PreparedStatement ps = conn.prepareStatement("SELECT * FROM USER WHERE login = ?"); ps.setString(1, input);` The driver sends query plan and parameters separately; input is never interpreted as SQL.

DEFINITION **Object-Oriented and NoSQL Databases**

- OODBMS (e.g. db4o, ObjectDB): store objects directly (identity, inheritance, polymorphism). No impedance mismatch. Query by navigation (OQL). Niche: CAD, scientific, telecom.
- ORDBMS (PostgreSQL, Oracle): relational core + user-defined types, arrays, inheritance, methods. Best of both worlds.
- NoSQL (Not Only SQL): scale-out, schema-less, BASE (Basically Available, Soft state, Eventual consistency).
  - Key-Value: Redis, DynamoDB (cache, session).
  - Document: MongoDB, CouchDB (JSON/BSON, flexible schema).

  - Column-family: Cassandra, HBase (wide rows, time-series).
  - Graph: Neo4j (nodes, edges, properties; social, fraud detection).
- CAP Theorem: Consistency, Availability, Partition Tolerance — pick two. RDBMS ≈ CA/CP; NoSQL often AP.

EXPERIMENT **Integration Activity II — Full Stack Mini-Project**

Scenario: An online bookstore for Cameroon ("Book237"). Users browse books, add to cart, checkout (Mobile Money), admin manages catalogue/orders. Deliverables:

1. Conceptual ERD (Users, Books, Categories, Orders, OrderItems, Payments).
2. Normalised relational schema (3NF) with DDL (PostgreSQL).
3. Stored procedure `place_order(user_id, cart_json)` handling stock decrement, order creation, payment record (transaction).
4. PHP/PDO (or Python/FastAPI) endpoint `POST /api/orders` using prepared statements.
5. SQL queries: (a) Top 5 selling books last month. (b) Revenue per category. (c) Users with abandoned carts (cart not empty, no order > 24h).
6. Security: roles (admin, customer), password hashing (bcrypt), HTTPS, input validation.

EXERCISE 9 **GCE-Style Practice Questions**

1. Define the terms: (i) Data Independence, (ii) Functional Dependency, (iii) Transitive Dependency.
2. Given the relation `R(A,B,C,D,E)` with FDs: `A → B, B → C, C → D, D → E`. Identify all candidate keys. Normalise to 3NF.
3. Draw the ERD (Crow's Foot) for: "A university has departments. Each department offers many courses. A student enrols in one department but takes many courses. Each course is taught by one lecturer. A lecturer teaches many courses." Show cardinalities and participation.
4. Write SQL to create tables for the university above with all constraints. Insert sample data. Write a query: "List lecturers who teach more than 3 courses in the Computer Science department".

5. Explain the difference between `HAVING` and `WHERE`. Give an example where using `WHERE` with an aggregate causes an error.
6. Describe the ACID properties. How does a DBMS ensure Durability?
7. Compare `INNER JOIN`, `LEFT JOIN` and `FULL JOIN` with a Venn diagram description.
8. A school uses SQLite for its offline registration app and PostgreSQL for the central server. Explain the synchronization challenges and outline a strategy (conflict detection, resolution).

SOLUTION **Exercise 2 — Normalisation**

Candidate Key: `A` (since A → B → C → D → E, A determines all). Normalisation: Already 1NF. 2NF: no partial dep (key single). 3NF: transitive deps exist (A → B → C etc). Decompose: `R1(A,B)`, `R2(B,C)`, `R3(C,D)`, `R4(D,E)`. All BCNF.

SOLUTION **Exercise 4 — SQL Query**

```
SELECT L.name, COUNT(C.course_id) AS nb_courses FROM LECTURER L JOIN COURSE C
ON L.lec_id = C.lec_id JOIN DEPARTMENT D ON C.dept_id = D.dept_id WHERE D.name =
'Computer Science' GROUP BY L.name HAVING COUNT(C.course_id) > 3;
```

12 EXERCISES

12.1 I check my knowledge

EXERCISE 10 **Multiple choice (GCE Paper 1 style)**

For each question, choose the single correct answer.

1. In the relational model, a **tuple** is:
   (a) a column of a relation;
   (b) a row of a relation;
   (c) a set of relations;
   (d) a key of a relation.
2. Referential integrity requires that:
   (a) every primary key value is unique and not null;
   (b) every value of a foreign key either is null or matches an existing primary key value in the referenced relation;
   (c) every attribute contains atomic values only;
   (d) no relation contains a transitive dependency.
3. The SQL command used to remove all rows from a table while keeping its structure is:
   (a) `DROP TABLE;`
   (b) `DELETE;`
   (c) `REMOVE;`
   (d) `ALTER TABLE.`
4. A relation is in third normal form (3NF) if it is in 2NF and:
   (a) it has a composite primary key;
   (b) it contains no transitive dependency of a non-key attribute on the primary key;
   (c) all its attributes are multivalued;
   (d) it contains no foreign key.

EXERCISE 11 True or false — justify

State whether each assertion is true or false, and justify your answer in one or two sentences.

1. In the ANSI-SPARC architecture, the conceptual level describes how data are physically stored on the disk.
2. A foreign key in a relation must always have the same name as the primary key it references.
3. The SQL clause `GROUP BY` must be used together with an aggregate function such as `AVG` or `COUNT`.
4. A many-to-many relationship between two entities is implemented in the relational model by creating a third relation whose primary key combines the primary keys of the two entities.

EXERCISE 12 Key definitions

Define each of the following terms in one or two sentences, giving an example where appropriate:

1. **primary key**;
2. **functional dependency**;
3. **entity integrity**;
4. **logical data independence**;
5. **DBMS**.

EXERCISE 13 Reading a query

Consider the relation `RESULT(CandID, Name, Centre, Subject, Mark)` containing the rows below.

CandID	Name	Centre	Subject	Mark
C01	Acha B.	GBHS Bamenda	CSC	14
C02	Ndi K.	GBHS Bamenda	CSC	9
C03	Eta M.	GHS Limbe	CSC	16
C04	Bih T.	GHS Limbe	MTH	12

1. Give the output of the query: `SELECT Name, Mark FROM RESULT WHERE Centre = 'GHS`

```
Limbe' AND Mark > 13;
```

2. Give the output of the query: `SELECT Subject, AVG(Mark) FROM RESULT GROUP BY Subject;`
3. Give the output of the query: `SELECT COUNT(*) FROM RESULT WHERE Mark >= 10;`

12.2 I practise

EXERCISE 14 **Writing DDL statements**

A school database must store subjects and candidates' results.

1. Write the SQL statement to create the table `SUBJECT(SubCode, SubName, Coefficient)` where `SubCode` is the primary key and `Coefficient` must be strictly positive.
2. Write the SQL statement to create the table `RESULT(CandID, CandName, Centre, SubCode, Mark)` where the primary key is the pair `(CandID, SubCode)`, `SubCode` is a foreign key referencing `SUBJECT`, and `Mark` must lie between 0 and 20.
3. State which integrity constraint is enforced by each of: the primary key of `RESULT`, the foreign key, and the condition on `Mark`.

EXERCISE 15 **Writing DML statements**

Using the tables `SUBJECT` and `RESULT` of the previous exercise, write SQL statements to:

1. insert the subjects `('CSC', 'Computer Science', 3)` and `('MTH', 'Mathematics', 4)` into `SUBJECT`;
2. insert three rows into `RESULT` of your choice, respecting all constraints;
3. increase by 2 the mark of candidate `'C01'` in subject `'CSC'`;
4. delete all results of candidates from the centre `'GHS Limbe'`;
5. compute the average mark per subject, displaying the subject code and the average, using `GROUP BY`.

EXERCISE 16 **Joins**

Consider the tables `STUDENT(StudID, Name, ClassID)` and `CLASS(ClassID, Label)` with

the following contents: `STUDENT` contains `(1, 'Acha', 10), (2, 'Ndi', 10), (3, 'Eta', 20), (4, 'Bih', NULL);` `CLASS` contains `(10, 'Upper Sixth A'), (30, 'Lower Sixth B').`

1. Write a query using an `INNER JOIN` to display each student's name with the label of his or her class.
2. Write the same query using a `LEFT JOIN` from `STUDENT` to `CLASS`.
3. Give the result set of each query and explain, in two or three sentences, why the two result sets differ.

EXERCISE 17 **From E-R model to relations**

A hospital wishes to manage consultations. A doctor treats many patients; a patient may consult many doctors; each consultation involves exactly one doctor and one patient, and has a date and a prescription.

1. Identify the entities, the relationship, its cardinality, and its attributes.
2. Draw the corresponding entity-relationship diagram (entities, attributes, relationship, cardinalities).
3. Convert the diagram into relations, underlining each primary key and marking each foreign key with an asterisk (*).

12.3 I prepare for the GCE

EXERCISE 18 **Normalisation of a GCE results table — 12 marks**

The GCE Board stores results in a single unnormalised table:

CandID	CandName	Centre	CentreTown	Subject	Mark
C01	Acha B.	GBHS Bamenda	Bamenda	CSC	14
C01	Acha B.	GBHS Bamenda	Bamenda	MTH	12
C02	Ndi K.	GHS Limbe	Limbe	CSC	9

A candidate sits several subjects; each centre is located in exactly one town.

1. Identify one insertion anomaly, one update anomaly and one deletion anomaly in this table. (3 marks)
2. List all the functional dependencies that hold in this table. (3 marks)
3. Explain why the table is not in second normal form, and decompose it into 2NF. (3 marks)

4. Decompose the result into third normal form (3NF), justifying each step and indicating the primary and foreign keys of each final relation. (3 marks)

EXERCISE 19 **ANSI-SPARC architecture and data independence — 8 marks**

1. Name and describe the three levels of the ANSI-SPARC database architecture, stating clearly what each level represents. (3 marks)
2. Define **logical data independence** and **physical data independence**. (2 marks)
3. Give one concrete example of a change that is absorbed by logical data independence, and one example of a change absorbed by physical data independence, explaining in each case which levels are affected and which are protected. (3 marks)

EXERCISE 20 **Publishing examination results online — 12 marks**

A large secondary school in Yaoundé wants parents to consult end-of-term results through a web site. The data are stored in a relational database.

1. Describe the three-tier architecture (presentation tier, application tier, data tier) needed for this system, giving the role of each tier and naming one technology that could be used at each tier. (4 marks)
2. Choose an RDBMS package for this project (for example MySQL, PostgreSQL or Oracle) and justify your choice with two arguments related to the school's context (cost, size, skills available, platform). (3 marks)
3. Explain two security measures that should be put in place to protect the database, and state against which threat each measure protects. (3 marks)
4. Explain one advantage that an object-oriented or object-relational approach could bring to such a web application compared with a purely relational approach. (2 marks)

13 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — Multiple choice (GCE Paper 1 style)**

1. Answer: (b) a row of a relation. In the relational model, a **tuple** is one row (one record) of a relation; a column is called an **attribute**.
2. Answer: (b). **Referential integrity** states that a foreign key value must either be `NULL` or match an existing primary key value in the referenced relation. Option (a) describes entity integrity, (c) describes atomicity (1NF), and (d) describes 3NF.
3. Answer: (b) `DELETE`. `DELETE FROM table;` removes all rows while keeping the table structure. `DROP TABLE` removes the table itself (structure and data); `REMOVE` is not an SQL command; `ALTER TABLE` modifies the structure.
4. Answer: (b). A relation is in **3NF** if it is in 2NF and contains no **transitive dependency** of a non-key attribute on the primary key (every non-key attribute depends on the key, the whole key, and nothing but the key).

SOLUTION **Exercise 2 — True or false — justify**

1. False. In the ANSI-SPARC architecture, the physical (internal) level describes how data are stored on disk (files, indexes, blocks). The conceptual level describes the whole logical structure of the database (entities, attributes, relationships) independently of any storage detail.
2. False. A foreign key must have a compatible domain (data type) with the referenced primary key, but its name is free. For example, `RESULT(SubCode)` may reference `SUBJECT(Code)`.
3. False. `GROUP BY` is most often used with aggregate functions, but it is not compulsory: `SELECT Centre FROM RESULT GROUP BY Centre;` is valid and simply lists the distinct centres (like `DISTINCT`).
4. True. A many-to-many relationship cannot be represented directly between two relations; it is implemented by a third (junction) relation whose primary key is the combination of the two primary keys, each part being a foreign key referencing one of the entities (e.g. `CONSULT(DocID*, PatID*, Date)`).

SOLUTION **Exercise 3 — Key definitions**

1. Primary key: a minimal set of one or more attributes whose values uniquely identify each tuple of a relation; it can never be null. Example: `CandID` in `CANDIDATE`.
2. Functional dependency: a constraint written $A \rightarrow B$ meaning that each value of attribute (set) A determines exactly one value of B. Example: `Centre` → `CentreTown`, since each centre is in one town.
3. Entity integrity: the rule that no component of a primary key may be null, so that every tuple is identifiable.
4. Logical data independence: the ability to modify the conceptual schema (e.g. add an attribute or a relation) without having to rewrite the external views or the application programs that use them.
5. DBMS (Database Management System): the software layer (e.g. MySQL, PostgreSQL, Oracle) that creates, manages and controls access to a database, guaranteeing integrity, security, concurrency and data independence.

SOLUTION **Exercise 4 — Reading a query**

1. The `WHERE` clause keeps rows with `Centre = 'GHS Limbe'` and `Mark > 13`: only C03 (Eta M., 16) qualifies; C04 has mark 12 which fails the second condition.

Name	Mark
Eta M.	16

2. Rows are grouped by `Subject`; the average is computed per group. CSC: (14+9+16)/3 = 39/3 = 13; MTH: 12/1 = 12.

Subject	AVG(Mark)
CSC	13
MTH	12

3. `COUNT(*)` counts rows with `Mark >= 10`: C01 (14), C03 (16), C04 (12) qualify; C02 (9) does not. Result: 3.

SOLUTION **Exercise 5 — Writing DDL statements**

1. Creation of `SUBJECT`:

```
CREATE TABLE SUBJECT (
  SubCode CHAR(3) PRIMARY KEY,
  SubName VARCHAR(40) NOT NULL,
  Coefficient INT CHECK (Coefficient > 0)
);
```

2. Creation of `RESULT`:

```
CREATE TABLE RESULT (
  CandID CHAR(3),
  CandName VARCHAR(40) NOT NULL,
  Centre VARCHAR(40),
  SubCode CHAR(3),
  Mark DECIMAL(4,1) CHECK (Mark BETWEEN 0 AND 20),
  PRIMARY KEY (CandID, SubCode),
  FOREIGN KEY (SubCode) REFERENCES SUBJECT(SubCode)
);
```

3. Constraints enforced:
   - the primary key `(CandID, SubCode)` enforces **entity integrity** (uniqueness, no null) — a candidate has at most one mark per subject;
   - the foreign key enforces **referential integrity** — a result can only be recorded for an existing subject;
   - the `CHECK` on `Mark` enforces a **domain integrity** constraint — marks stay within the valid range $[0, 20]$.

SOLUTION **Exercise 6 — Writing DML statements**

1. Insertion of the subjects:

```
INSERT INTO SUBJECT VALUES ('CSC', 'Computer Science', 3);
INSERT INTO SUBJECT VALUES ('MTH', 'Mathematics', 4);
```

2. Three valid rows (subjects must exist, marks in $[0, 20]$, unique pairs):

```
INSERT INTO RESULT VALUES ('C01', 'Acha B.', 'GBHS Bamenda', 'CSC', 14);
```

```
INSERT INTO RESULT VALUES ('C01', 'Acha B.', 'GBHS Bamenda', 'MTH', 12);
INSERT INTO RESULT VALUES ('C02', 'Ndi K.', 'GHS Limbe', 'CSC', 9);
```

3. Update:

```
UPDATE RESULT SET Mark = Mark + 2
WHERE CandID = 'C01' AND SubCode = 'CSC';
```

(The mark becomes 16, still within [0, 20], so the `CHECK` constraint holds.)

4. Deletion:

```
DELETE FROM RESULT WHERE Centre = 'GHS Limbe';
```

5. Average per subject:

```
SELECT SubCode, AVG(Mark) AS AverageMark
FROM RESULT
GROUP BY SubCode;
```

SOLUTION **Exercise 7 — Joins**

1. Inner join:

```
SELECT STUDENT.Name, CLASS.Label
FROM STUDENT INNER JOIN CLASS
ON STUDENT.ClassID = CLASS.ClassID;
```

2. Left join:

```
SELECT STUDENT.Name, CLASS.Label
FROM STUDENT LEFT JOIN CLASS
ON STUDENT.ClassID = CLASS.ClassID;
```

3. Result sets:

`INNER JOIN` (only matching rows):

Name	Label
Acha	Upper Sixth A
Ndi	Upper Sixth A

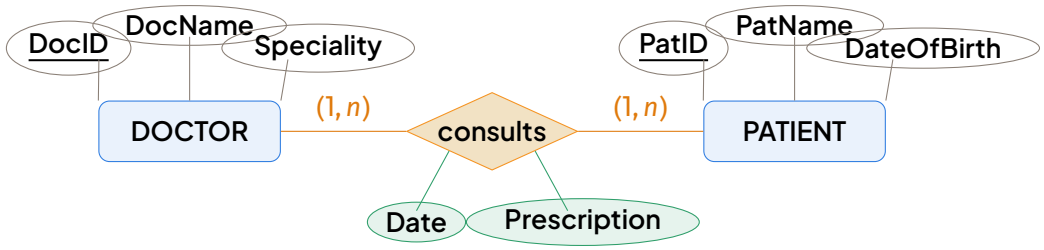
`LEFT JOIN` (all students kept):

Name	Label
Acha	Upper Sixth A
Ndi	Upper Sixth A
Eta	NULL
Bih	NULL

Explanation. The `INNER JOIN` keeps only rows where the join condition is satisfied: Eta's class 20 does not exist in `CLASS`, and Bih has a `NULL` `ClassID` which never matches, so both disappear. The `LEFT JOIN` preserves every row of the left table (`STUDENT`); when no matching class exists, the columns of `CLASS` are filled with `NULL`. Note also that class 30 (`Lower Sixth B`) appears in neither result, since no student references it.

SOLUTION **Exercise 8 — From E-R model to relations**

1. Entities: DOCTOR (DocID, DocName, Speciality) and PATIENT (PatID, PatName, DateOfBirth). Relationship: consults between DOCTOR and PATIENT. Cardinality: many-to-many (a doctor treats many patients; a patient may consult many doctors). Attributes of the relationship: `Date`, `Prescription` (they belong to the consultation itself, not to the doctor or the patient).
2. Entity-relationship diagram:



E-R diagram of the hospital consultation database.

3. Conversion into relations (primary keys underlined, foreign keys marked *):
   - `DOCTOR(DocID, DocName, Speciality)`
   - `PATIENT(PatID, PatName, DateOfBirth)`
   - `CONSULTATION(DocID*, PatID*, Date, Prescription)`

The many-to-many relationship becomes a third relation. Its primary key combines the two entity keys; `Date` is included in the key because the same patient may consult

the same doctor on different days. `DocID*` references `DOCTOR` and `PatID*` references `PATIENT`.

SOLUTION **Exercise 9 — Normalisation of a GCE results table — 12 marks**

1. Anomalies (3 marks — 1 per correct anomaly):
   - Insertion anomaly: a new centre (e.g. GBHS Douala) cannot be recorded until at least one candidate's result exists for it; likewise a new candidate cannot be registered before sitting a subject.
   - Update anomaly: the town of a centre is repeated on every row; if `GBHS Bamenda` were reclassified, every row mentioning it must be updated, and missing one creates inconsistency. Similarly `CandName` is repeated.
   - Deletion anomaly: if the single row of candidate C02 is deleted, all information about `GHS Limbe` and its town disappears from the database.
2. Functional dependencies (3 marks — 1 per correct FD):
   - `CandID` → `CandName, Centre` (a candidate has one name and one centre);
   - `Centre` → `CentreTown` (each centre is in exactly one town);
   - `(CandID, Subject)` → `Mark` (one mark per candidate per subject).

   The primary key of the table is the composite `(CandID, Subject)`.
3. Not in 2NF — decomposition (3 marks). The table is in 1NF (all values atomic) but not in 2NF, because non-key attributes depend on only part of the composite key: `CandName`, `Centre` and `CentreTown` depend on `CandID` alone (partial dependency). Only `Mark` depends on the whole key. Decomposition into 2NF:
   - `CANDIDATE(CandID, CandName, Centre, CentreTown)`
   - `RESULT(CandID*, Subject, Mark)`
4. 3NF (3 marks — 1 for identifying the transitive dependency, 1 for the decomposition, 1 for keys). `CANDIDATE` is still not in 3NF because of the transitive dependency `CandID` → `Centre` → `CentreTown`: a non-key attribute (`CentreTown`) depends on another non-key attribute. Removing it gives the final 3NF schema:
   - `CANDIDATE(CandID, CandName, Centre*)` — `Centre*` references `CENTRE`;
   - `CENTRE(Centre, CentreTown)`;
   - `RESULT(CandID*, Subject, Mark)` — `CandID*` references `CANDIDATE`.

   Each non-key attribute now depends on the key, the whole key, and nothing but the key.

Examiner expectations: anomalies must be illustrated with the given data (not just named); FDs must be written with the arrow notation; keys must be underlined and foreign keys indicated at each stage.

SOLUTION **Exercise 10 — ANSI-SPARC architecture and data independence — 8 marks**

1. The three levels (3 marks — 1 per level):
   - External level: the users' views — the part of the database seen by each user group or application (e.g. a teacher sees only his class's marks).
   - Conceptual level: the complete logical description of the whole database — all entities, attributes and relationships — independent of any physical storage.
   - Internal (physical) level: how the data are actually stored on disk: files, records, indexes, data blocks and access paths.
2. Data independence (2 marks):
   - Logical data independence: the ability to change the conceptual schema without changing the external views or the application programs.
   - Physical data independence: the ability to change the internal (storage) schema without changing the conceptual schema.
3. Examples (3 marks):
   - Logical: adding a new attribute `Gender` to the `CANDIDATE` relation, or splitting a relation during normalisation. The conceptual level changes, but existing external views (e.g. the marks-entry view) can be redefined to hide the change, so application programs keep working unchanged.
   - Physical: adding an index on `CandID` to speed up searches, or moving the database to a faster disk. Only the internal level changes; the conceptual schema and all external views are completely unaffected — queries are simply executed faster.

Examiner expectations: precise vocabulary (external / conceptual / internal), and examples that clearly state which level changes and which levels are protected.

SOLUTION **Exercise 11 — Publishing examination results online — 12 marks**

1. Three-tier architecture (4 marks — 1 per tier + 1 for coherent technologies):
   - Presentation tier: the user interface displayed in the parents' web browser — pages to log in and view results. Technology: HTML/CSS with JavaScript.

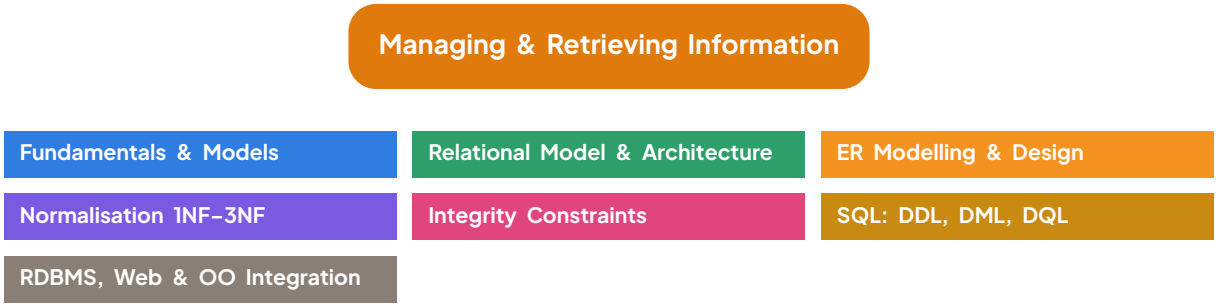
- Application tier: the web/application server that receives requests, applies the business logic (checking credentials, computing averages) and builds the pages. Technology: PHP (or Python/Node.js) running on Apache.
- Data tier: the DBMS that stores candidates, subjects and marks, executes the SQL queries and enforces integrity. Technology: MySQL (or PostgreSQL).

The tiers communicate only through well-defined interfaces: the browser never touches the database directly.

2. Choice of RDBMS (3 marks — choice 1, two justified arguments 2). Example: MySQL. Arguments: (i) it is free and open-source, which suits a school with a limited budget, and runs on ordinary hardware under Linux or Windows; (ii) it is widely taught and documented in Cameroon, so skilled administrators and community support are easy to find, and it integrates natively with PHP/Apache (the classic LAMP stack). (PostgreSQL or Oracle accepted if the arguments are coherent, e.g. Oracle for very large scale with paid support.)
3. Security measures (3 marks — 1.5 per measure with its threat):
   - Authentication and access control: each parent receives a login and password, and database user accounts are granted only the minimum privileges (read-only on their own child's results). Protects against unauthorised access to confidential marks.
   - Encryption of connections (HTTPS/SSL) and parameterised queries: HTTPS protects data travelling between the browser and the server against interception; parameterised (prepared) queries protect the database against SQL injection attacks through the login form. (Also acceptable: regular backups against data loss, firewalls against network attacks.)
4. Advantage of an object-oriented / object-relational approach (2 marks). An object-relational database can store complex objects directly — for example a Result object containing the candidate, the list of subject marks and computed methods (average, rank) — avoiding the constant decomposition/recomposition of objects into flat relational rows (the impedance mismatch). This makes the web application's code simpler, more natural to maintain, and allows richer data types (e.g. storing a scanned report card or photo) than a purely relational model.

Examiner expectations: a clearly layered description with a named technology per tier; arguments linked to the school's real context (cost, skills); each security measure explicitly matched to a threat; one precise, correctly explained object-oriented advantage.

14 SUMMARY SHEET



Mind map of the chapter: from data organisation to full database implementation with SQL.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions

- A **database** is an organised collection of related data, stored and managed so that many users can share it consistently. A **DBMS** (Database Management System) is the software that creates, manages and controls access to it (e.g. MySQL, PostgreSQL, MS Access, Oracle).
- Data organisation levels: **bit** → **byte** → **field** → **record** → **file** → **database**.
- Database models: **hierarchical** (tree), **network** (graph, many-to-many), **relational** (tables), **object-oriented** (objects with methods).
- In the **relational model**: a **relation** (table) has **tuples** (rows) and **attributes** (columns); the **domain** is the set of allowed values of an attribute; **degree** = number of attributes; **cardinality** = number of tuples.
- Keys: a **primary key** uniquely identifies each tuple (unique, never NULL); a **foreign key** is an attribute that references the primary key of another relation; a **candidate key** is any minimal superkey.
- **ANSI/SPARC architecture**: **external level** (user views), **conceptual level** (global logical structure), **internal level** (physical storage). It guarantees **logical** and **physical data independence**.
- **ER model**: entities (strong/weak), attributes (simple, composite, derived, multivalued, key), relationships with cardinality ratios (1:1, 1:N, M:N) and participation (total/partial).
- A **functional dependency** $X \rightarrow Y$: the value of X determines the value of Y. A **partial dependency** involves part of a composite key; a **transitive dependency** passes through a non-key attribute.

- **Integrity**: **entity integrity** (primary key not NULL), **referential integrity** (every foreign key matches an existing primary key or is NULL), **domain integrity** (values in the allowed domain), **user-defined integrity** (business rules).

Normal forms (must know)

- **1NF**: all attribute values are atomic (no repeating groups).
- **2NF**: 1NF and no partial dependency on part of a composite primary key.
- **3NF**: 2NF and no transitive dependency (non-key attribute depending on another non-key attribute).

SQL essentials

- **DDL**: `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`; constraints `PRIMARY KEY`, `FOREIGN KEY ... REFERENCES`, `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`.
- **DML**: `INSERT INTO ... VALUES`, `UPDATE ... SET ... WHERE`, `DELETE FROM ... WHERE`.
- **DQL**: `SELECT ... FROM ... WHERE ... ORDER BY ...`; joins: `INNER JOIN`, `LEFT/RIGHT JOIN`, or `WHERE t1.fk = t2.pk`; aggregates: `COUNT`, `SUM`, `AVG`, `MAX`, `MIN` with `GROUP BY` and `HAVING`.
- Design stages: requirements analysis → conceptual design (ER) → logical design (relations, normalisation) → physical design (implementation in an RDBMS).

Methods

- ER → relations: each entity becomes a table; a $1:N$ relationship puts the primary key of the "1" side as foreign key on the "N" side; an $M:N$ relationship becomes a new table whose primary key combines both foreign keys.
- Normalisation: list all attributes, identify the key and functional dependencies, remove repeating groups (1NF), remove partial dependencies (2NF), remove transitive dependencies (3NF).

Common mistakes

- Confusing primary key and foreign key; allowing a primary key to be NULL.
- Forgetting `WHERE` in `UPDATE`/`DELETE` (all rows are modified!).
- Using `WHERE` instead of `HAVING` to filter aggregate results.
- Storing multivalued or derived data in one field (breaks 1NF).
- Representing $M:N$ directly without a junction table.

GCE tips

- In ER diagrams, always mark cardinality and participation; in trace/design questions, state the normal form reached at each step and justify with the dependency removed.

- Quote full SQL syntax with correct clause order: `SELECT -- FROM -- WHERE -- GROUP BY -- HAVING -- ORDER BY`.
- Define terms precisely (one sentence each) before applying them; examiners reward exact vocabulary: tuple, attribute, domain, schema, instance.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+